ICPC C++ Team Notebook — Manual Completo de Algoritmos y Estructuras

Estructura de cada sección:

- **¿Cuándo usar?**: Señales en el enunciado, cotas típicas (N, M, Q) y criterio de elección.
- **Complejidad**: Tiempo y espacio en notación $\mathcal{O}(\cdot)$.
- **Plantilla C++**: Código limpio, probado y formateado a ≤ 60 caracteres por línea para impresión a **3 columnas**.
- **Métodos / Ejemplo de Input y Uso (`main`)**: Documentación puntual de métodos STL y casos de entrada completos con `main()` y salida esperada para algoritmos complejos.

Índice General

1. **Bloque 1: C++ Base, Strings STL, Bitmasks y Cambios de Base**
   - 1.1 Template Base ICPC, Pragmas, Formateo `cout` / `printf` y Comparador `EPS`
   - 1.2 Generador Aleatorio (`mt19937_64`), `shuffle` y `rd(l, r)`
   - 1.3 `std::string`: Métodos Clave (`find`, `rfind`, `substr`, `erase`, `insert`, `replace`, `stringstream`, `stoll`)
   - 1.4 Manipulación de Bits (`__builtin`, LSO, Submáscaras) y `std::bitset`
   - 1.5 Cambios de Base (Base $A \leftrightarrow B$, Binario, Base Negativa) y Números Gigantes en String
2. **Bloque 2: Contenedores STL, PBDS y sus Métodos Más Importantes**
   - 2.1 `std::vector` y `<algorithm>`: Métodos, `sort`, Búsqueda Binaria (`lower_bound` / `upper_bound`) y Compresión de Coordenadas
   - 2.2 Contenedores Lineales: `std::deque`, `std::queue` y `std::stack`
   - 2.3 Colas de Prioridad / Heaps (`std::priority_queue`): Max-Heap, Min-Heap, Custom y **Deletable Heap**
   - 2.4 Árboles Balanceados (`std::set`, `std::multiset`, `std::map`): Métodos, Búsqueda Binaria en `set` y Borrado Seguro en `multiset`
   - 2.5 Tablas Hash (`unordered_map`, `unordered_set`) con `custom_hash` Anti-Hack (SplitMix64) y Pares Iguales
   - 2.6 Extensiones GNU: `ordered_set` / `ordered_multiset` (PBDS) y `__gnu_cxx::rope`
3. **Bloque 3: Búsquedas, Two Pointers, Prefix Sum y Difference Array**
   - 3.1 Binary Search (Discreta Max/Min y Continua `double`)
   - 3.2 Ternary Search (Discreta y Continua para Funciones Unimodales)
   - 3.3 Two Pointers, Monotonic Stack (Next Greater/Smaller) y Monotonic Deque (Sliding Window Minimum)
   - 3.4 Prefix Sum 1D (Arreglos y Cadenas) y Prefix Sum 2D (Submatrices en $\mathcal{O}(1)$)
   - 3.5 Difference Array 1D, Doble Difference Array (Greg and Array) y Difference Array 2D
4. **Bloque 4: Estructuras de Datos Avanzadas (Consultas en Rango y Árboles)**
   - 4.1 Disjoint Set Union (DSU Clásico) y DSU con Rollback + Conectividad Dinámica Offline
   - 4.2 Sparse Table (RMQ e Idempotentes en $\mathcal{O}(1)$)
   - 4.3 Fenwick Tree / BIT 1D (con Binary Lifting `lower_bound`) y Fenwick Tree 2D
   - 4.4 Segment Tree Clásico (Recursivo) y Segment Tree Iterativo
   - 4.5 Segment Tree con Lazy Propagation, Segment Tree 2D y Persistent Segment Tree
   - 4.6 SQRT Decomposition y Algoritmo de Mo (con Optimización de Bloques)
   - 4.7 Treap (BST Aleatorizado) e Implicit Treap (Reversión y Suma en Rango)
   - 4.8 Consultas en Árboles: Heavy-Light Decomposition (HLD) y DSU on Tree (Sack)
   - 4.9 Link-Cut Tree (LCT — Bosques Dinámicos Online)
5. **Bloque 5: Algoritmos de Strings**
   - 5.1 Rotación Cíclica Mínima / Minimum Expression (Duval/Booth $\mathcal{O}(N)$)
   - 5.2 Double Rolling Hash Polinomial y LCP con Búsqueda Binaria
   - 5.3 KMP (Función Prefijo π, Pattern Matching, Periodo) y Autómata KMP
   - 5.4 Z-Algorithm y Algoritmo de Manacher (Palíndromos Pares e Impares)
   - 5.5 Trie (Strings y Trie Binario para Max XOR) y Aho-Corasick (Múltiples Patrones)
   - 5.6 Suffix Array + LCP, Suffix Automaton (SAM) y Palindromic Tree (Eertree)
6. **Bloque 6: Grafos, Árboles, Conectividad y Flujos**
   - 6.1 Recorridos BFS: Grid BFS (Flood Fill), Multi-Source BFS (Aristas Invertidas) y 0-1 BFS
   - 6.2 Patrones DFS: Grafo Bipartito, Ciclos, Euler Tour (`tin` / `tout`), DFS en Complemento y Conteo de Triángulos
   - 6.3 Caminos Más Cortos: Dijkstra, Bellman-Ford / SPFA (Ciclo Negativo) y Floyd-Warshall
   - 6.4 Árboles: Diámetro (Doble BFS), MST (Kruskal y Prim), LCA Binary Lifting y Centroid Decomposition
   - 6.5 Conectividad, DAGs y 2-SAT: Topological Sort, Puentes/Articulación, Biconexas, SCC (Tarjan/Kosaraju), 2-SAT y Camino Euleriano
   - 6.6 Flujos y Matching: Dinic, Min-Cost Max-Flow (MCMF), Kuhn (MCBM) + König
   - 6.7 Árbol Generador Mínimo Dirigido (Directed MST / Arborescence)
   - 6.8 Componentes Biconexas y Block-Cut Tree
   - 6.9 Camino y Ciclo Euleriano (Hierholzer)
   - 6.10 Flujos Avanzados y Matching: Push-Relabel, Algoritmo Húngaro y Blossom
7. **Bloque 7: Matemáticas, Teoría de Números, Álgebra y Juegos**
   - 7.1 Cribas (Eratóstenes, Divisores, Lineal SPF, Segmentada) y Primalidad $\mathcal{O}(\sqrt{N})$
   - 7.2 Primalidad y Factorización Grande: Miller-Rabin (64-bit) y Pollard's Rho
   - 7.3 GCD, Diferencia de Cuadrados, Euclides Extendido, Ecuaciones Diofánticas, Ecuación Modular y CRT
   - 7.4 Aritmética Modular (`mint`), Combinatoria (nCr), Teorema de Lucas y Fórmula de Legendre ($n!$)
   - 7.5 Funciones Multiplicativas: Totient de Euler (ϕ) y Función de Möbius (μ)
   - 7.6 Álgebra Lineal y Polinomios: Exponenciación de Matrices, Eliminación Gaussiana (`double`, Modular, XOR Basis), FFT, FWT, Logaritmo Discreto y Fracciones
   - 7.7 XOR en Rango $\mathcal{O}(1)$, Función `mex`, Grundy y Juego de Nim / Misère Nim
   - 7.8 Criba Segmentada (Primos en Rango $[L, R]$)
   - 7.9 Programación Lineal: Algoritmo Simplex de Dos Fases
   - 7.10 Berlekamp-Massey y Recurrencias Lineales Modulares
   - 7.11 Interpolación de Lagrange en $\mathcal{O}(D)$ y $\mathcal{O}(D^2)$
8. **Bloque 8: Programación Dinámica (DP) y Geometría Computacional**
   - 8.1 DP Clásica y Máscaras: Knapsack 0/1 y Unbounded, LCS, Edit Distance, LIS $\mathcal{O}(N \log N)$, Digit DP y SOS DP
   - 8.2 Optimizaciones de DP: Convex Hull Trick (Estático y Dinámico), Divide & Conquer DP y Knuth's Optimization
   - 8.3 Geometría 2D: `Point`, `Line`, Segmentos, `Polygon` (Área, Centroide, Punto en Polígono, Corte), `Convex Hull`, Rotating Callipers, Closest Pair
   - 8.4 Intersección de Semiplanos (Halfplane Intersection)
   - 8.5 Suma de Minkowski de Polígonos Convexos
9. **Bloque 9: Miscelánea y Parsing Especial**
   - 9.1 Evaluador de Expresiones Aritméticas (Shunting-Yard Parser)
   - 9.2 Clique Máximo (Bron-Kerbosch Branch & Bound)
10. **Bloque 10: Compendio Teórico, Teoremas, Fórmulas e Identidades**
    - 10.1 Tabla de Optimización de DP (CHT, D&Q, Knuth, WQS)
    - 10.2 Combinatoria y Sumatorias (Catalan, Stirling, Bell, Partición Pentagonal, Burnside)
    - 10.3 Teoría de Números (Diofánticas, Frobenius/McNuggets, Wilson, Euler, Legendre, Pisano)
    - 10.4 Teoría de Grafos y Redes (Kirchhoff Matrix-Tree, Cayley, Prüfer, König, Dilworth, Gale-Shapley, Flujo con Demandas)
    - 10.5 Geometría y Distancias (Pick, Herón, Incírculo, Circuncírculo, Haversine)
    - 10.6 Teoría de Juegos y XOR (Sprague-Grundy, Nim, Misère Nim, Moore Nim_k)
    - 10.7 Reglas Lógicas de 2-SAT
    - 10.8 Tabla de Constantes, Primos y Módulos Frecuentes
11. **Bloque 11: Python 3 para ICPC — Trucos, BigInt, Bases y Librería Estándar**
    - 11.1 Template Base, Fast I/O, Límites de Recursión y Digits en BigInt
    - 11.2 Conversión de Bases Arbitrarias (Base $A \leftrightarrow B$) y Formateo
    - 11.3 Aritmética Gigante (BigInt), Potencia Modular `pow()` y Raíz Entera `isqrt()`
    - 11.4 Fracciones Exactas (`Fraction`) y Decimales de Alta Precisión (`Decimal`)
    - 11.5 Manejo de Fechas y Calendarios (`datetime`)
    - 11.6 DP con Memoización Automática (`@cache` / `cache_clear`)
    - 11.7 Extracción de Números en Texto Sucio con Expresiones Regulares (`re`)
    - 11.8 Frecuencias y Álgebra de Multiconjuntos (`Counter`)
    - 11.9 Agrupación y Grafos Instantáneos (`defaultdict`)
    - 11.10 Rotación y Transposición de Matrices en 1 Línea (`zip(*mat)`)
    - 11.11 Álgebra de Conjuntos Nativos (`set`)
    - 11.12 Bits en Números Gigantes (`bit_count`, `bit_length`)
    - 11.13 Búsqueda Binaria con Proyección (`bisect` con `key`) y Ordenamiento Multi-Criterio
    - 11.14 Combinatoria y Generadores Nativos (`comb`, `perm`, `itertools`)
    - 11.15 Evaluación de Expresiones Aritméticas (`eval`)

Bloque 1: C++ Base, Strings STL, Bitmasks y Cambios de Base

1.1 Template Base ICPC, Pragmas, Formateo `cout` / `printf` y Comparador `EPS`

- **¿Cuándo usar?**: Esqueleto inicial para todo problema. Incluye optimización de I/O, impresión de `__int128`, pragmas de vectorización (usar solo si el tiempo es muy ajustado en $\mathcal{O}(N^2)$ simple) y formateo de decimales/ceros a la izquierda.
- **Complejidad**: I/O en $\mathcal{O}(1)$ amortizado.

```cpp
// Descomentar pragmas solo si se requiere optimizacion SIMD
// #pragma GCC optimize("O3,unroll-loops")
// #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using ull = unsigned long long;
using ld = long double;
using i128 = __int128_t;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;
using vll = vector<ll>;

#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()
#define pb push_back
#define fi first
#define se second

const int INF_INT = 1e9 + 7;
const ll INF = 1e18;
const ld EPS = 1e-9;
const ld PI = acos(-1.0L);

// Impresion para __int128
ostream& operator<<(ostream& os, i128 n) {
  if (n == 0) return os << 0;
  if (n < 0) { os << '-'; n = -n; }
  string s;
  while (n > 0) { s += (char)('0' + (n % 10)); n /= 10; }
  reverse(all(s));
  return os << s;
}

// Comparador seguro con EPS para set<double> o map<double, T>
struct eps_cmp {
  bool operator()(const double& a, const double& b) const {
    return a + EPS < b;
  }
};
// Uso: set<double, eps_cmp> s_dbl;

void io_formatting_tricks() {
  // 1. Imprimir punto flotante con exactamente 6 decimales:
  cout << fixed << setprecision(6) << 3.14159265 << "\n";
  // 2. Rellenar con ceros a la izquierda (ej. hora 05:09):
  int h = 5, m = 9;
  cout << setfill('0') << setw(2) << h << ":"
       << setw(2) << m << "\n";
  // 3. Imprimir en base hexadecimal, octal y volver a decimal:
  cout << hex << 255 << " " << oct << 255 << dec << "\n";
}

void solve() {
  // Logica por caso de prueba
}

int main() {
```

```
  ios_base::sync_with_stdio(false);
  cin.tie(nullptr);
  int t = 1;
  // cin >> t;
  while (t--) solve();
  return 0;
}
```

1.2 Generador Aleatorio (`mt19937_64`), `shuffle` y `rd(l, r)`

- **¿Cuándo usar?**: Cuando necesites algoritmos aleatorizados (Treap, Pollard's Rho, Miller-Rabin), barajar un arreglo para evitar peores casos, o generar casos de prueba (*stress testing*). Nunca usar `rand()` porque en Windows solo genera hasta 32767.
- **Complejidad**: $\mathcal{O}(1)$ por número generado; $\mathcal{O}(N)$ para `shuffle` .

```
mt19937_64 rng(
  chrono::steady_clock::now().time_since_epoch().count()
);

// Entero aleatorio uniforme en el rango cerrado [l, r]
ll rd(ll l, ll r) {
  return uniform_int_distribution<ll>(l, r)(rng);
}

// Real aleatorio uniforme en [l, r]
double rd_real(double l, double r) {
  return uniform_real_distribution<double>(l, r)(rng);
}

// Barajar un vector aleatoriamente:
// shuffle(all(v), rng);
```

1.3 `std::string` : Métodos Clave (`find` , `rfind` , `substr` , `erase` , `insert` , `replace` , `stringstream` , `stoll`)

- **¿Cuándo usar?**: Procesamiento directo de texto, extracción de subcadenas, reemplazos, tokenización (separar palabras por espacios o por comas/dos puntos) y conversión entre bases y números.
- **Complejidad**: `find` / `substr` / `erase` / `insert` toman $\mathcal{O}(N)$ (donde $N = |S|$).

```
// 1. Separar string por un caracter delimitador (ej. ',', ':')
vector<string> split_delim(const string& s, char delim) {
  vector<string> tokens;
  stringstream ss(s);
  string item;
  while (getline(ss, item, delim)) tokens.push_back(item);
  return tokens;
}

// 2. Separar string por espacios (ignora espacios multiples)
vector<string> split_spaces(const string& s) {
  vector<string> tokens;
  stringstream ss(s);
  string w;
  while (ss >> w) tokens.push_back(w);
  return tokens;
}

void string_methods_demo() {
  string s = "icpc_regional_icpc_2026";

  // --- BUSQUEDA (find / rfind) ---
  // Retorna indice 0-based o string::npos si NO existe
  size_t p1 = s.find("icpc");       // 0 (primera aparicion)
  size_t p2 = s.find("icpc", 1);    // 14 (busca desde idx 1)
  size_t p3 = s.rfind("icpc");      // 14 (ultima aparicion)
  if (s.find("xyz") == string::npos) { /* No esta */ }

  // Hallar TODAS las ocurrencias de un patron 'pat':
  string pat = "icpc";
  for (size_t pos = s.find(pat); pos != string::npos;
       pos = s.find(pat, pos + 1)) {
    // 'pos' es el inicio 0-based de cada coincidencia
  }

  // --- SUBCADENAS (substr) ---
  // ¡CUIDADO! substr(pos_inicio, LONGITUD)
  // Para extraer el rango [L, R] inclusivo: s.substr(L, R-L+1)
  string sub1 = s.substr(5, 8);     // "regional" (8 chars)
  string sub2 = s.substr(19);       // "2026" (hasta el final)

  // --- EDICION (erase / insert / replace) ---
  s.erase(4, 9);           // Borra 9 chars desde idx 4
  s.insert(4, "_world");   // Inserta en idx 4
  s.replace(0, 4, "acm"); // Reemplaza 4 chars desde 0 por "acm"

  // --- CONVERSIONES ---
  ll val = stoll("1234567890123");     // string -> long long
  ll from_bin = stoll("101101", 0, 2); // base 2 -> ll (45)
  ll from_hex = stoll("FF", 0, 16);    // base 16 -> ll (255)
  double d = stod("3.14159");          // string -> double
  string num_s = to_string(val);       // numero -> string

  // --- MAYUSCULAS / MINUSCULAS ---
  transform(all(s), s.begin(), ::tolower);
  transform(all(s), s.begin(), ::toupper);
}
```

Tabla de Referencia Rápida de `std::string`

Método Sintaxis y Retorno Detalle Crítico en Competencia find s.find(sub, pos = 0) Retorna size_t. Comparar siempre con string::npos. rfind s.rfind(sub) Busca de derecha a izquierda (última aparición). substr s.substr(i, len) El 2do parámetro es cantidad de caracteres, NO el índice final. erase s.erase(i, len) Borra len caracteres empezando en i en $\mathcal{O}(|S|)$. insert s.insert(i, str) Inserta str justo antes de la posición i. replace s.replace(i, len, str) Borra len chars desde i y pone str en su lugar. stoll stoll(s, nullptr, base) Convierte s en base [2, 36] a long long nativamente. getline getline(cin >> ws, s) cin >> ws consume saltos de línea previos antes de leer s.

1.4 Manipulación de Bits (`__builtin` , LSO, Submáscaras) y `std::bitset`

- **¿Cuándo usar?**:
  - **Bitmasks**: Representar subconjuntos cuando $N \leq 20$ (o $N \leq 60$ en propiedades binarias/XOR).
  - `std::bitset<MAX_N>` : Optimizar operaciones de conjuntos booleanos, alcanzabilidad en DAGs o Knapsack booleano dividiendo el tiempo y memoria entre 64: $\mathcal{O}(N/64)$.
- **Complejidad**: Operaciones de bits en $\mathcal{O}(1)$; iterar todas las submáscaras de todas las máscaras de N bits toma $\mathcal{O}(3^N)$; operaciones de `bitset` toman $\mathcal{O}(N/64)$.

```
void bitmask_tricks(ll x) {
  // 1. Funciones de hardware GCC (sufijo 'll' para 64 bits)
  int ones = __builtin_popcountll(x); // Cantidad de bits en 1
  int ctz  = __builtin_ctzll(x);      // Ceros a la derecha (x>0)
  int clz  = __builtin_clzll(x);      // Ceros a la izq (x>0)
  int msb  = 63 - __builtin_clzll(x); // floor(log2(x)) exacto

  // 2. Operaciones sobre el k-esimo bit (0-indexed)
  int k = 4;
  x |= (1LL << k);          // Prender bit k
  x &= ~(1LL << k);         // Apagar bit k
  x ^= (1LL << k);          // Invertir (flip) bit k
  bool on = (x >> k) & 1LL; // Consultar si bit k es 1

  // 3. Trucos del bit menos significativo (LSO / LSB)
  ll lso = x & -x;          // Aisla el 1 mas bajo
  ll clear_lso = x & (x-1); // Apaga el 1 mas bajo
  ll set_low0  = x | (x+1); // Prende el 0 mas bajo
  bool is_pow2 = (x > 0) && ((x & (x - 1)) == 0);

  // 4. Iterar sobre todas las submascaras no vacias de 'm'
  int m = 0b10110;
  // Orden decreciente:
  for (int sub = m; sub > 0; sub = (sub - 1) & m) {
    // procesar 'sub'
  }
  // Orden creciente:
  for (int sub = 0; (sub = (sub - m) & m); ) {
    // procesar 'sub'
  }
}

void bitset_methods_demo() {
  bitset<1005> bs;
  bs.set();         // Pone todos los bits en 1
  bs.reset();       // Pone todos los bits en 0
  bs.set(5, 1);     // Prende el bit 5 (bs[5] = 1)
  bs.reset(5);      // Apaga el bit 5 (bs[5] = 0)
  bs.flip(2);       // Invierte el bit 2
  int c = bs.count();   // Numero de bits en 1: O(N/64)
  bool a = bs.any();    // true si al menos un bit es 1
  bool n = bs.none();   // true si todos son 0

  // Recorrer SOLO los indices con bit en 1 en O(#unos + N/64):
  for (int i = bs._Find_first(); i < (int)bs.size();
       i = bs._Find_next(i)) {
    // 'i' es la posicion del bit encendido
  }
}
```

1.5 Cambios de Base (Base $A \leftrightarrow B$, Binario, Base Negativa) y Números Gigantes en String

- **¿Cuándo usar?**:
  - `to_binary_string(num, n)` : Cuando necesitas la representación binaria de longitud fija n con el bit menos significativo en `s[0]` (tu plantilla de `Notebook.md`).
  - `decimal_to_base` / `base_to_decimal` : Conversión entre cualquier par de bases $B \in [2, 36]$ o base negativa (ej. base -2).
  - `divide_large_number` / `mod_large_number` : Cuando el input es un entero gigante de 10^5 dígitos dado como `string` y necesitas dividirlo o sacarle módulo por un `int` / `long long` .
- **Complejidad**: $\mathcal{O}(\log_B(\text{num}))$ para cambios de base; $\mathcal{O}(|S|)$ para división/módulo en string.

```
// 1. Numero a string binario de longitud fija n (s[0] = LSB)
string to_binary_string(ll num, int n) {
  string s = "";
  for (int i = 0; i < n; i++) {
    s += (num % 2 == 0 ? '0' : '1');
    num /= 2;
  }
  return s; // s[0] es el bit 0 (menos significativo)
}

// 2. Conversor general entre bases [2, 36] (MSB primero)
int char_to_val(char c) {
  if (isdigit(c)) return c - '0';
  return toupper(c) - 'A' + 10;
}
char val_to_char(int v) {
  return (v < 10) ? ('0' + v) : ('A' + v - 10);
}

string decimal_to_base(ll num, int base) {
  if (num == 0) return "0";
  bool neg = (num < 0);
  if (neg) num = -num;
  string res = "";
  while (num > 0) {
    res += val_to_char(num % base);
    num /= base;
  }
  if (neg) res += '-';
  reverse(all(res));
  return res;
}

ll base_to_decimal(const string& s, int base) {
  if (s.empty()) return 0;
  ll res = 0;
  int st = (s[0] == '-' || s[0] == '+') ? 1 : 0;
  for (int i = st; i < sz(s); ++i)
    res = res * base + char_to_val(s[i]);
  return (s[0] == '-') ? -res : res;
}

// 3. Cambio a base negativa (ej. neg_base = -2)
string to_negative_base(ll n, int neg_base) {
  if (n == 0) return "0";
  string res = "";
  while (n != 0) {
    ll rem = n % neg_base;
    n /= neg_base;
    if (rem < 0) { rem += (-neg_base); n += 1; }
    res += (char)('0' + rem);
  }
  reverse(all(res));
  return res;
}

// 4. Division entera y modulo de numero gigante en string
// Ejemplo: divide_large_number("10010", 14) -> "715"
string divide_large_number(const string& s, ll n) {
```

```
  string res = "";
  ll rem = 0;
  for (char c : s) {
    rem = rem * 10 + (c - '0');
    if (!res.empty() || rem / n > 0)
      res += to_string(rem / n);
    rem %= n;
  }
  return res.empty() ? "0" : res;
}

ll mod_large_number(const string& s, ll m) {
  ll rem = 0;
  for (char c : s) rem = (rem * 10 + (c - '0')) % m;
  return rem;
}
```

Bloque 2: Contenedores STL, PBDS y sus Métodos Más Importantes

2.1 `std::vector` y `<algorithm>`: Métodos, `sort`, Búsqueda Binaria (`lower_bound` / `upper_bound`) y Compresión de Coordenadas

- **¿Cuándo usar?**:
  - **Búsqueda Binaria en `vector` ordenado**: Para hallar en $\mathcal{O}(\log N)$ el primer elemento $\geq x$ (`lower_bound`), el primero $> x$ (`upper_bound`), el último $\leq x$ o $< x$, y contar cuántos elementos caen en un rango $[L, R]$.
  - **Compresión de Coordenadas**: Cuando los valores del arreglo son enormes ($[-10^9, 10^9]$) pero solo importa su orden relativo y quieres usarlos como índices de un Fenwick Tree o Segment Tree en $[0, N-1]$.
- **Complejidad**: `sort` y Compresión en $\mathcal{O}(N \log N)$; búsquedas binarias en $\mathcal{O}(\log N)$.

```
// Compresion de Coordenadas: mapea valores arbitrarios a [0, K-1]
struct CoordComp {
  vll vals;
  void init(const vll& a) {
    vals = a;
    sort(all(vals));
    vals.erase(unique(all(vals)), vals.end());
  }
  int get_id(ll x) const { // Indice 0-based en O(log N)
    return lower_bound(all(vals), x) - vals.begin();
  }
  ll get_val(int id) const { return vals[id]; }
  int size() const { return sz(vals); }
};

void vector_and_bs_cheatsheet() {
  vi v = {1, 2, 2, 5, 5, 8, 9}; // DEBE ESTAR ORDENADO

  // 1. Primer elemento >= x (lower_bound)
  auto it_ge = lower_bound(all(v), 5); // Apunta al 1er 5 (idx 3)
  int idx_ge = it_ge - v.begin();

  // 2. Primer elemento > x (upper_bound)
  auto it_gt = upper_bound(all(v), 5); // Apunta al 8 (idx 5)
  int idx_gt = it_gt - v.begin();

  // 3. Ultimo elemento <= x (predecesor inclusivo)
  auto it_le = upper_bound(all(v), 5);
  if (it_le != v.begin()) {
    int val_le = *prev(it_le); // 5 (idx 4)
  }

  // 4. Ultimo elemento < x (predecesor estricto)
  auto it_lt = lower_bound(all(v), 5);
  if (it_lt != v.begin()) {
    int val_lt = *prev(it_lt); // 2 (idx 2)
  }

  // 5. Contar cuantos elementos estan en el rango [L, R]
  int L = 2, R = 8;
  int cnt_in_range = upper_bound(all(v), R)
                   - lower_bound(all(v), L); // 5 elementos

  // 6. Utilidades clave de <algorithm> y <numeric>
  ll suma = accumulate(all(v), 0LL); // ¡Usar 0LL, no 0!
  int mx = *max_element(all(v));
  int mn = *min_element(all(v));

  // 7. Generar todas las permutaciones en O(N! * N)
  vi p = {3, 1, 2};
  sort(all(p)); // OBLIGATORIO ordenar antes del do-while
  do {
    // procesar permutacion 'p'
  } while (next_permutation(all(p)));
}
```

2.2 Contenedores Lineales: `std::deque`, `std::queue` y `std::stack`

- **¿Cuándo usar?**:
  - `std::deque`: Cuando necesites insertar/borrar por **ambos extremos** en $\mathcal{O}(1)$ y además acceder por índice `dq[i]` en $\mathcal{O}(1)$ (ej. 0-1 BFS, Sliding Window Minimum).
  - `std::queue`: Cola FIFO para BFS estándar y Kahn TopoSort.
  - `std::stack`: Pila LIFO para Monotonic Stack, evaluación de expresiones e historiales de rollback.
- **Complejidad**: Todas las operaciones en los extremos e indexación en `deque` son $\mathcal{O}(1)$.

```
void linear_containers_demo() {
  // 1. std::deque (Doble cola + acceso aleatorio O(1))
  deque<int> dq;
  dq.push_back(20);     // [20]
  dq.push_front(10);    // [10, 20]
  int f = dq.front();   // 10
  int b = dq.back();    // 20
  int x = dq[1];        // 20 (acceso por indice O(1))
  dq.pop_front();       // Quita el 10
  dq.pop_back();        // Quita el 20

  // 2. std::queue (FIFO - Primero en entrar, primero en salir)
  queue<int> q;
  q.push(1); q.push(2);
  int primero = q.front(); // 1
  q.pop();                 // Elimina el frente (1)

  // 3. std::stack (LIFO - Ultimo en entrar, primero en salir)
  stack<int> st;
  st.push(10); st.push(20);
  int tope = st.top();     // 20
  st.pop();                // Elimina el tope (20)
}
```

2.3 Colas de Prioridad / Heaps (`std::priority_queue`): Max-Heap, Min-Heap, Custom y Deletable Heap

- **¿Cuándo usar?**:
  - **Max-Heap / Min-Heap**: Extraer repetidamente el máximo o mínimo dinámico en $\mathcal{O}(\log N)$ (Dijkstra, Prim, Greedy con eventos).
  - **Deletable Heap (Lazy Deletion)**: Cuando necesitas la velocidad de un Heap pero el problema también pide **eliminar elementos arbitrarios por valor** (más rápido en constante que `std::multiset`).
- **Complejidad**: `push`, `pop` y `erase` en $\mathcal{O}(\log N)$; `top` en $\mathcal{O}(1)$ amortizado.

```
// 1. Comparador en struct para Min-Heap (invertir el signo '<')
struct State {
  int u;
  ll cost;
  bool operator<(const State& o) const {
    return cost > o.cost; // El menor 'cost' queda en top()
  }
};

// 2. Deletable Min-Heap (Borrado arbitrario en O(log N))
template <typename T>
struct DeletableMinHeap {
  priority_queue<T, vector<T>, greater<T>> pq, del;

  void push(T x) { pq.push(x); }
  void erase(T x) { del.push(x); } // Borrado perezoso

  void clean() {
    while (!pq.empty() && !del.empty() && pq.top() == del.top()) {
      pq.pop();
      del.pop();
    }
  }
  T top() { clean(); return pq.top(); }
  void pop() { clean(); pq.pop(); }
  bool empty() { clean(); return pq.empty(); }
  int size() const { return sz(pq) - sz(del); }
};

void priority_queue_demo() {
  // Max-Heap (por defecto: saca el MAYOR primero)
  priority_queue<ll> max_pq;
  max_pq.push(10); max_pq.push(50);
  ll mayor = max_pq.top(); // 50
  max_pq.pop();

  // Min-Heap (saca el MENOR primero)
  priority_queue<ll, vector<ll>, greater<ll>> min_pq;
  min_pq.push(10); min_pq.push(50);
  ll menor = min_pq.top(); // 10
  min_pq.pop();
}
```

2.4 Árboles Balanceados (`std::set`, `std::multiset`, `std::map`): Métodos, Búsqueda Binaria en `set` y Borrado Seguro en `multiset`

- **¿Cuándo usar?**: Cuando necesites mantener un conjunto ordenado con inserciones, borrados y consultas de **sucesor** ($\geq x$, $> x$) **o predecesor** ($\leq x$, $< x$) dinámicamente en $\mathcal{O}(\log N)$.
- **Complejidad**: `insert`, `erase(it)`, `find`, `lower_bound`, `upper_bound` en $\mathcal{O}(\log N)$.

```
void set_multiset_map_cheatsheet() {
  set<int> s = {10, 20, 30, 40};
  s.insert(25);
  s.erase(20);                  // Borra el 20 en O(log N)
  int mn = *s.begin();          // Minimo actual (10) en O(1)
  int mx = *s.rbegin();         // Maximo actual (40) en O(1)

  // --- BUSQUEDA BINARIA EN SET / MULTISET ---
  // ¡PROHIBIDO usar std::lower_bound(all(s), x) porque es O(N)!
  // Usar SIEMPRE el metodo miembro s.lower_bound(x) -> O(log N):

  int x = 25;
  // 1) Primer elemento >= x:
  auto it_ge = s.lower_bound(x);
  if (it_ge != s.end()) { int val_ge = *it_ge; }

  // 2) Primer elemento > x:
  auto it_gt = s.upper_bound(x);
  if (it_gt != s.end()) { int val_gt = *it_gt; }

  // 3) Mayor elemento < x (predecesor estricto):
  auto it_lt = s.lower_bound(x);
  if (it_lt != s.begin()) { int val_lt = *prev(it_lt); }

  // 4) Mayor elemento <= x (predecesor inclusivo):
  auto it_le = s.upper_bound(x);
  if (it_le != s.begin()) { int val_le = *prev(it_le); }

  // --- TRAMPAS CLASICAS DE MULTISET ---
  multiset<int> ms = {5, 5, 5, 10};
  // Trampa 1: ms.erase(5) borra TODAS las apariciones del 5.
  // Para borrar UNA SOLA copia del 5 en O(log N):
  auto it = ms.find(5);
  if (it != ms.end()) ms.erase(it); // Queda {5, 5, 10}

  // Trampa 2: ms.count(x) es O(k + log N) donde k = repeticiones.
  // Para saber solo si existe en O(log N), usar:
  bool existe = (ms.find(5) != ms.end());

  // --- CUIDADO CON STD::MAP ---
  map<int, int> mp;
  // Hacer if (mp[key] == 0) INSERTA 'key' en el mapa y gasta RAM/tiempo.
  // Para consultar sin insertar:
  if (mp.find(100) != mp.end()) { int val = mp[100]; }
}
```

2.5 Tablas Hash (unordered_map , unordered_set) con custom_hash Anti-Hack (SplitMix64) y Pares Iguales

- **¿Cuándo usar?**: Cuando quieras contar frecuencias o buscar llaves en $\mathcal{O}(1)$ promedio sin necesitar orden, evitando los casos *anti-hash* de Codeforces/ICPC que degradan el unordered_map estándar a $\mathcal{O}(N^2)$.
- **Complejidad**: $\mathcal{O}(1)$ esperado por operación.

```
struct custom_hash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM =
      chrono::steady_clock::now().time_since_epoch().count();
    return splitmix64(x + FIXED_RANDOM);
  }
  // Permite usar pair<T1, T2> como llave en safe_umap
  template <typename T1, typename T2>
  size_t operator()(const pair<T1, T2>& p) const {
    uint64_t h1 = (*this)(p.first);
    uint64_t h2 = (*this)(p.second);
    return h1 ^ (h2 + 0x9e3779b97f4a7c15 + (h1<<6) + (h1>>2));
  }
};

template <typename K, typename V>
using safe_umap = unordered_map<K, V, custom_hash>;

template <typename K>
using safe_uset = unordered_set<K, custom_hash>;

// Aplicacion: Contar pares (i < j) con a[i] == a[j] en O(N)
// Ejemplo: a = [1, 2, 1, 1, 2] -> 3 pares de 1 + 1 par de 2 = 4
ll count_equal_pairs(const vi& a) {
  safe_umap<int, int> freq;
  freq.reserve(sz(a) * 2);
  freq.max_load_factor(0.25f);
  for (int x : a) freq[x]++;
  ll pairs = 0;
  for (auto& [val, cnt] : freq)
    pairs += 1LL * cnt * (cnt - 1) / 2;
  return pairs;
}
```

2.6 Extensiones GNU: ordered_set / ordered_multiset (PBDS) y __gnu_cxx::rope

- **¿Cuándo usar?**:
  - ordered_set : Cuando además de un std::set necesites responder en $\mathcal{O}(\log N)$ cuál es el **k-ésimo elemento más pequeño** (find_by_order(k) , 0-indexed) o **cuántos elementos son estrictamente menores que** x (order_of_key(x)).
  - rope : Cuando tengas que cortar un subarreglo/substring $[L, R]$ y pegarlo en otra posición en $\mathcal{O}(\log N)$ sin programar un Implicit Treap.
- **Complejidad**: $\mathcal{O}(\log N)$ por operación.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/rope>
using namespace __gnu_pbds;
using namespace __gnu_cxx;

// 1. Ordered Set (elementos unicos)
template <typename T>
using ordered_set = tree<
  T, null_type, less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;

// 2. Ordered Multiset (usar less_equal o pair<T, int>)
template <typename T>
using ordered_multiset = tree<
  T, null_type, less_equal<T>, rb_tree_tag,
  tree_order_statistics_node_update>;

void pbds_and_rope_demo() {
  ordered_set<int> os;
  os.insert(10); os.insert(20); os.insert(50);

  // k-esimo elemento (0-indexed):
  int elem_idx_1 = *os.find_by_order(1); // 20
  // Cantidad de elementos estrictamente < 25:
  int menores_25 = os.order_of_key(25);  // 2 (el 10 y el 20)

  // Como borrar UNA sola copia en ordered_multiset<int> oms:
  ordered_multiset<int> oms;
  oms.insert(5); oms.insert(5);
  int idx = oms.order_of_key(5);
  oms.erase(oms.find_by_order(idx));

  // 3. Uso de rope<int> o crope (rope<char>)
  // Cortar el rango 0-based [l, r] y moverlo al inicio:
  rope<int> rp;
  for (int i = 1; i <= 5; ++i) rp.push_back(i); // [1,2,3,4,5]
  int l = 1, r = 3, len = r - l + 1;
  rope<int> sub = rp.substr(l, len); // Extrae [2, 3, 4]
  rp.erase(l, len);                  // Queda [1, 5]
  rp.insert(rp.mutable_begin(), sub);// Queda [2, 3, 4, 1, 5]
}
```

Bloque 3: Búsquedas, Two Pointers, Prefix Sum y Difference Array

3.1 Binary Search (Discreta Max/Min y Continua double)

- **¿Cuándo usar?**: Cuando el problema pide *"maximizar el mínimo"* o *"minimizar el máximo"*, o cuando verificar si una respuesta X es válida (check(X)) es fácil y la función de validez es **monótona** (ej. V, V, V, F, F o F, F, F, V, V).
- **Complejidad**: $\mathcal{O}(\log(\text{high} - \text{low}) \cdot \text{costo(check)})$. En double , $80 - 100$ iteraciones logran precisión de 10^{-15}.

```
// 1. Maximizar respuesta discreta: [True, True, ..., False, False]
template <typename F>
ll bs_max(ll low, ll high, F check) {
  ll ans = low - 1; // Centinela si ninguno cumple
  while (low <= high) {
    ll mid = low + (high - low) / 2;
    if (check(mid)) {
      ans = mid;      // mid sirve, probar mayores
      low = mid + 1;
    } else {
      high = mid - 1;
    }
  }
  return ans;
}

// 2. Minimizar respuesta discreta: [False, False, ..., True, True]
template <typename F>
ll bs_min(ll low, ll high, F check) {
  ll ans = high + 1;
  while (low <= high) {
    ll mid = low + (high - low) / 2;
    if (check(mid)) {
      ans = mid;      // mid sirve, probar menores
      high = mid - 1;
    } else {
      low = mid + 1;
    }
  }
  return ans;
}

// 3. Binary Search continua en double (iteraciones fijas)
template <typename F>
double bs_double(double l, double r, F check, int it = 90) {
  for (int i = 0; i < it; ++i) {
    double mid = l + (r - l) / 2.0;
    if (check(mid)) l = mid; // Si maximiza (cambiar a r=mid si minimiza)
    else r = mid;
  }
  return l;
}
```

Ejemplo de Uso (Cortar maderas / Agresive Cows):

```
// Ejemplo: Dado un arreglo de troncos, hallar la mayor longitud L
// tal que se puedan sacar al menos K pedazos de longitud L.
int main() {
  vll a = {10, 15, 20};
  ll k = 4;
  auto check = [&](ll L) {
    ll cnt = 0;
    for (ll x : a) cnt += x / L;
    return cnt >= k;
  };
  ll best_L = bs_max(1, 20, check); // Retorna 10
}
```

3.2 Ternary Search (Discreta y Continua para Funciones Unimodales)

- **¿Cuándo usar?**: Cuando necesitas encontrar el **mínimo o máximo global** de una función **estrictamente unimodal** $f(x)$ (una función en forma de U que primero decrece y luego crece, o en forma de montaña que primero crece y luego decrece), donde NO puedes derivar fácilmente.
- **Complejidad**: $\mathcal{O}(\log_{3/2}(R - L) \cdot \text{costo}(f))$.

```
// 1. Ternary Search Continua (halla el MINIMO en [l, r])
// Para hallar el MAXIMO: cambiar f(m1) < f(m2) por f(m1) > f(m2)
template <typename F>
double ternary_continuous(double l, double r, F f, int it = 100) {
  for (int i = 0; i < it; ++i) {
    double m1 = l + (r - l) / 3.0;
    double m2 = r - (r - l) / 3.0;
    if (f(m1) < f(m2)) r = m2;
    else l = m1;
  }
  return f(l); // Retorna el valor minimo (o 'l' para la posicion)
}

// 2. Ternary Search Discreta en enteros [low, high] (MINIMO)
template <typename F>
ll ternary_discrete(ll low, ll high, F f) {
  while (high - low >= 3) {
    ll m1 = low + (high - low) / 3;
    ll m2 = high - (high - low) / 3;
    if (f(m1) < f(m2)) high = m2;
    else low = m1;
  }
  ll best = f(low);
  for (ll x = low + 1; x <= high; ++x)
    best = min(best, f(x));
  return best;
}
```

3.3 Two Pointers, Monotonic Stack (Next Greater/Smaller) y Monotonic Deque (Sliding Window Minimum)

- **¿Cuándo usar?**:
  - **Two Pointers**: Subarreglo más largo/corto que cumple una condición monótona (ej. suma $\leq S$ con elementos ≥ 0, o a lo sumo K distintos).
  - **Monotonic Stack**: Hallar para cada posición i el primer índice a su derecha/izquierda que sea estrictamente mayor o menor (ej. mayor rectángulo en un histograma, contribución de cada elemento como mínimo/máximo de subarreglos).
  - **Monotonic Deque**: Hallar el **mínimo o máximo de todas las subventanas de tamaño fijo** K en tiempo lineal $\mathcal{O}(N)$ sin usar Segment Tree ni multiset .
- **Complejidad**: $\mathcal{O}(N)$ tiempo y $\mathcal{O}(N)$ espacio (cada elemento entra y sale de la pila/cola a lo sumo una vez).

```
// 1. Two Pointers: Subarreglo mas largo con suma <= max_sum (a[i] >= 0)
int max_subarray_sum_bound(const vll& a, ll max_sum) {
  int n = sz(a), max_len = 0, l = 0;
  ll cur_sum = 0;
  for (int r = 0; r < n; ++r) {
    cur_sum += a[r];
    while (cur_sum > max_sum && l <= r) {
      cur_sum -= a[l++];
    }
    max_len = max(max_len, r - l + 1);
  }
  return max_len;
}

// 2. Monotonic Stack: Indice del primer elemento > a[i] a la derecha
// Retorna -1 si no existe. (Para < a[i], cambiar '<' por '>')
vi next_greater_right(const vll& a) {
  int n = sz(a);
  vi nxt(n, -1);
  stack<int> st;
  for (int i = 0; i < n; ++i) {
```

```
    while (!st.empty() && a[st.top()] < a[i]) {
      nxt[st.top()] = i;
      st.pop();
    }
    st.push(i);
  }
  return nxt;
}

// 3. Monotonic Deque: Minimo de cada ventana de longitud K
// Retorna vector con N - K + 1 respuestas. (Para Maximo: cambiar >= por <=)
vll sliding_window_min(const vll& a, int k) {
  int n = sz(a);
  deque<int> dq; // Guarda indices en orden creciente de valor
  vll ans;
  for (int i = 0; i < n; ++i) {
    if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
    while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1) ans.push_back(a[dq.front()]);
  }
  return ans;
}
```

Ejemplo de Input y Uso (Monotonic Stack & Sliding Window):

```
Input:
N = 6, K = 3
A = [4, 2, 5, 1, 3, 6]
```

```
int main() {
  vll a = {4, 2, 5, 1, 3, 6};
  vi ngr = next_greater_right(a); // [2, 2, 5, 4, 5, -1]
  vll win = sliding_window_min(a, 3); // [2, 1, 1, 1]
}
```

3.4 Prefix Sum 1D (Arreglos y Cadenas) y Prefix Sum 2D (Submatrices en $\mathcal{O}(1)$)

- **¿Cuándo usar?**: Cuando el arreglo o matriz es **estático** (no hay actualizaciones entre consultas) y necesitas responder Q consultas de suma de un rango $[L, R]$ o de un rectángulo $(r_1, c_1) \to (r_2, c_2)$ en $\mathcal{O}(1)$.
- **Complejidad**: Preprocesamiento $\mathcal{O}(N)$ en 1D y $\mathcal{O}(N \cdot M)$ en 2D; cada consulta `query` toma $\mathcal{O}(1)$.

```
// 1. Prefix Sum 1D (1-indexed en las consultas [l, r])
struct PrefixSum {
  vll pref;
  PrefixSum(const vll& a) : pref(sz(a) + 1, 0) {
    for (int i = 0; i < sz(a); ++i)
      pref[i + 1] = pref[i] + a[i];
  }
  // Constructor para contar caracteres adyacentes iguales (CF 313B)
  PrefixSum(const string& s) : pref(sz(s) + 1, 0) {
    for (int i = 0; i + 1 < sz(s); ++i)
      pref[i + 1] = pref[i] + (s[i] == s[i + 1]);
    if (!s.empty()) pref[sz(s)] = pref[sz(s) - 1];
  }
  ll query(int l, int r) const { // 1-based [l, r]
    return (l > r) ? 0 : pref[r] - pref[l - 1];
  }
};

// 2. Prefix Sum 2D (Submatrices 1-based [r1..r2][c1..c2])
struct PrefixSum2D {
  vector<vll> p;
  PrefixSum2D(const vector<vll>& mat) {
    int n = sz(mat), m = n ? sz(mat[0]) : 0;
    p.assign(n + 1, vll(m + 1, 0));
    for (int i = 1; i <= n; ++i)
      for (int j = 1; j <= m; ++j)
        p[i][j] = mat[i - 1][j - 1] + p[i - 1][j]
                + p[i][j - 1] - p[i - 1][j - 1];
  }
  ll query(int r1, int c1, int r2, int c2) const {
    if (r1 > r2 || c1 > c2) return 0;
    return p[r2][c2] - p[r1 - 1][c2]
         - p[r2][c1 - 1] + p[r1 - 1][c1 - 1];
  }
};
```

3.5 Difference Array 1D, Doble Difference Array (Greg and Array) y Difference Array 2D

- **¿Cuándo usar?**: Cuando tienes muchas actualizaciones de suma en rango (`sumar V a todo [L, R]` o a toda una submatriz) pero **todas las consultas ocurren al final** (offline). Evita programar un Segment Tree con Lazy Propagation.
- **Complejidad**: Cada actualización de rango/submatriz toma $\mathcal{O}(1)$; reconstruir el arreglo final toma $\mathcal{O}(N)$ (o $\mathcal{O}(N \cdot M)$ en 2D).

```
// 1. Difference Array 1D (1-indexed [1..n])
struct DiffArray1D {
  int n;
  vll diff;
  DiffArray1D(int n) : n(n), diff(n + 2, 0) {}
  void add_range(int l, int r, ll val) { // 1-based [l, r]
    diff[l] += val;
    diff[r + 1] -= val;
  }
  vll build() const {
    vll res(n + 1, 0);
    for (int i = 1; i <= n; ++i) res[i] = res[i - 1] + diff[i];
    return res; // res[1..n]
  }
};

// 2. Doble Difference Array (Operaciones de rango sobre operaciones)
// Se tienen M operaciones (sumar d_i en [l_i, r_i]) y K consultas
// que aplican todas las operaciones desde la x_j hasta la y_j (CF 295A).
struct Op { int l, r; ll d; };
vll apply_double_diff(int n, int m, const vll& base_arr,
                      const vector<Op>& ops,
                      const vector<pii>& queries) {
  DiffArray1D op_diff(m);
  for (auto [x, y] : queries) op_diff.add_range(x, y, 1);
  vll op_cnt = op_diff.build();

  DiffArray1D arr_diff(n);
  for (int i = 1; i <= m; ++i)
    if (op_cnt[i] > 0)
      arr_diff.add_range(ops[i].l, ops[i].r, ops[i].d * op_cnt[i]);

  vll delta = arr_diff.build(), ans(n + 1, 0);
  for (int i = 1; i <= n; ++i) ans[i] = base_arr[i] + delta[i];
  return ans;
}

// 3. Difference Array 2D (Sumar 'val' al rectangulo [r1..r2][c1..c2])
struct DiffArray2D {
  int n, m;
  vector<vll> d;
  DiffArray2D(int n, int m) : n(n), m(m), d(n + 2, vll(m + 2, 0)) {}
  void add_rect(int r1, int c1, int r2, int c2, ll val) {
    d[r1][c1] += val;
    d[r1][c2 + 1] -= val;
    d[r2 + 1][c1] -= val;
    d[r2 + 1][c2 + 1] += val;
  }
  vector<vll> build() const {
    vector<vll> res(n + 1, vll(m + 1, 0));
    for (int i = 1; i <= n; ++i)
      for (int j = 1; j <= m; ++j)
        res[i][j] = d[i][j] + res[i - 1][j]
                  + res[i][j - 1] - res[i - 1][j - 1];
    return res;
  }
};
```

Bloque 4: Estructuras de Datos Avanzadas (Consultas en Rango y Árboles)

4.1 Disjoint Set Union (DSU Clásico) y DSU con Rollback + Conectividad Dinámica Offline

- **¿Cuándo usar?**:
  - **DSU Clásico**: Mantener componentes conexas al **agregar** aristas, consultar si dos nodos están conectados, conocer el tamaño de cada componente o detectar ciclos al armar un MST (Kruskal).
  - **DSU con Rollback y Conectividad Dinámica**: Cuando en el grafo **se agregan y también se ELIMINAN aristas**, y te piden el número de componentes conexas tras cada operación (offline en $\mathcal{O}(Q \log Q \log N)$ dividiendo en el árbol de tiempo).
- **Complejidad**: DSU clásico: $\mathcal{O}(\alpha(N)) \approx \mathcal{O}(1)$ por operación. DSU con Rollback (sin compresión de caminos, solo unión por rango): $\mathcal{O}(\log N)$ por unión, $\mathcal{O}(1)$ por rollback.

```
// 1. DSU Clasico (Path Compression + Union by Size)
struct DSU {
  vi parent, sz;
  int num_comp;
  DSU(int n) : parent(n + 1), sz(n + 1, 1), num_comp(n) {
    iota(all(parent), 0);
  }
  int find(int i) {
    return (parent[i] == i) ? i : (parent[i] = find(parent[i]));
  }
  bool unite(int i, int j) {
    int ri = find(i), rj = find(j);
    if (ri == rj) return false;
    if (sz[ri] < sz[rj]) swap(ri, rj);
    parent[rj] = ri;
    sz[ri] += sz[rj];
    num_comp--;
    return true;
  }
  bool same(int i, int j) { return find(i) == find(j); }
  int get_size(int i) { return sz[find(i)]; }
};

// 2. DSU con Rollback + Dynamic Connectivity Offline
struct RollbackDSU {
  vi p, r, hist;
  int comp;
  RollbackDSU(int n) : p(n + 1), r(n + 1, 1), comp(n) {
    iota(all(p), 0);
  }
  int find(int i) const { return (p[i] == i) ? i : find(p[i]); }
  void unite(int i, int j) {
    i = find(i); j = find(j);
    if (i == j) return;
    if (r[i] > r[j]) swap(i, j);
    p[i] = j; r[j] += r[i];
    hist.pb(i); comp--;
  }
  int snapshot() const { return sz(hist); }
  void rollback(int snap) {
    while (sz(hist) > snap) {
      int i = hist.back(); hist.pop_back();
      r[p[i]] -= r[i]; p[i] = i; comp++;
    }
  }
};

struct DynCon {
  enum { ADD, DEL, QUERY };
  struct Ev { int type, u, v; };
  vector<Ev> q;
  RollbackDSU dsu;
  vi mt, ans;
  map<pii, int> active;

  DynCon(int n) : dsu(n) {}
  void add_edge(int u, int v) {
    if (u > v) swap(u, v);
    active[{u, v}] = sz(q);
    q.pb({ADD, u, v}); mt.pb(-1);
  }
  void rem_edge(int u, int v) {
    if (u > v) swap(u, v);
    int pr = active[{u, v}];
    mt[pr] = sz(q);
    q.pb({DEL, u, v}); mt.pb(pr);
  }
  void add_query() { q.pb({QUERY, -1, -1}); mt.pb(-1); }

  void go(int l, int r) {
    if (l + 1 == r) {
      if (q[l].type == QUERY) ans.pb(dsu.comp);
      return;
    }
    int snap = dsu.snapshot(), mid = (l + r) / 2;
    for (int i = mid; i < r; ++i)
      if (mt[i] != -1 && mt[i] < l) dsu.unite(q[i].u, q[i].v);
    go(l, mid); dsu.rollback(snap);
    for (int i = l; i < mid; ++i)
      if (mt[i] >= r) dsu.unite(q[i].u, q[i].v);
    go(mid, r); dsu.rollback(snap);
  }
  vi solve() {
    if (q.empty()) return {};
    for (int i = 0; i < sz(q); ++i)
```

```
      if (q[i].type == ADD && mt[i] == -1) mt[i] = sz(q);
    go(0, sz(q));
    return ans;
  }
};
```

Ejemplo de Input y Uso (Dynamic Connectivity):

```
Input: N = 4 nodos.
1) Conectar (1, 2) y (2, 3) -> Query (esperado: 2 componentes)
2) Borrar (1, 2) -> Query (esperado: 3 componentes)
```

```
int main() {
  DynCon dc(4);
  dc.add_edge(1, 2);
  dc.add_edge(2, 3);
  dc.add_query();
  dc.rem_edge(1, 2);
  dc.add_query();
  vi res = dc.solve(); // res = {2, 3}
}
```

4.2 Sparse Table (RMQ e Idempotentes en $\mathcal{O}(1)$)

- **¿Cuándo usar?**: Cuando el arreglo es **100% estático** (sin modificaciones) y tienes muchas consultas ($Q \leq 10^6$) de **Mínimo, Máximo, GCD, AND o OR** en un rango $[L, R]$. Responder en $\mathcal{O}(1)$ puro evita el factor $\log N$ de un Segment Tree.
- **Complejidad**: Construcción $\mathcal{O}(N \log N)$ tiempo y memoria; cada consulta `query(L, R)` toma $\mathcal{O}(1)$.

```
struct SparseTable {
  int n, K;
  vector<vll> st;
  vi lg;

  ll op(ll a, ll b) const {
    return min(a, b); // Cambiar a max(a, b) o std::gcd(a, b)
  }

  SparseTable(const vll& a) {
    n = sz(a);
    K = __lg(max(1, n)) + 1;
    st.assign(K, vll(n));
    lg.assign(n + 1, 0);
    for (int i = 2; i <= n; ++i) lg[i] = lg[i / 2] + 1;
    st[0] = a;
    for (int j = 1; j < K; ++j)
      for (int i = 0; i + (1 << j) <= n; ++i)
        st[j][i] = op(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
  }

  // Consulta en rango 0-indexed [L, R] inclusivo en O(1)
  ll query(int L, int R) const {
    int j = lg[R - L + 1];
    return op(st[j][L], st[j][R - (1 << j) + 1]);
  }
};
```

4.3 Fenwick Tree / BIT 1D (con Binary Lifting `lower_bound`) y Fenwick Tree 2D

- **¿Cuándo usar?**:
  - **Fenwick 1D**: Actualizaciones puntuales (`a[i] += delta`) y sumas de rango $[L, R]$ dinámicas. Es $3\times$ más rápido y usa $4\times$ menos memoria que un Segment Tree. Además permite hallar el primer prefijo con suma $\geq K$ en $\mathcal{O}(\log N)$.
  - **Fenwick 2D**: Actualizaciones puntuales en una matriz $N \times M$ (`mat[x][y] += delta`) y suma de submatrices rectangulares en $\mathcal{O}(\log N \log M)$ (para $N, M \leq 2500$).
- **Complejidad**: 1D: $\mathcal{O}(\log N)$ tiempo y $\mathcal{O}(N)$ memoria. 2D: $\mathcal{O}(\log N \log M)$ tiempo y $\mathcal{O}(N \cdot M)$ memoria.

```
// 1. Fenwick Tree 1D (1-indexed: 1 <= idx <= n)
struct Fenwick {
  int n;
  vll bit;
  Fenwick(int n) : n(n), bit(n + 1, 0) {}

  void add(int idx, ll delta) {
    for (; idx <= n; idx += idx & -idx) bit[idx] += delta;
  }
  ll query(int idx) const { // Suma en [1..idx]
    ll sum = 0;
    for (; idx > 0; idx -= idx & -idx) sum += bit[idx];
    return sum;
  }
  ll query(int l, int r) const { // Suma en [l..r]
    return (l > r) ? 0 : query(r) - query(l - 1);
  }
  // Menor indice idx tal que query(1..idx) >= target (valores >= 0)
  int lower_bound(ll target) const {
    int idx = 0;
    ll cur = 0;
    for (int pw = 1 << __lg(max(1, n)); pw > 0; pw >>= 1) {
      if (idx + pw <= n && cur + bit[idx + pw] < target) {
        idx += pw;
        cur += bit[idx];
      }
    }
    return idx + 1;
  }
};

// 2. Fenwick Tree 2D (1-indexed: 1 <= r <= n, 1 <= c <= m)
struct Fenwick2D {
  int n, m;
  vector<vll> bit;
  Fenwick2D(int n, int m) : n(n), m(m), bit(n + 1, vll(m + 1, 0)) {}

  void add(int r, int c, ll delta) {
    for (int i = r; i <= n; i += i & -i)
      for (int j = c; j <= m; j += j & -j)
        bit[i][j] += delta;
  }
  ll query(int r, int c) const { // Suma en [1..r][1..c]
    ll sum = 0;
    for (int i = r; i > 0; i -= i & -i)
      for (int j = c; j > 0; j -= j & -j)
        sum += bit[i][j];
    return sum;
  }
  ll query(int r1, int c1, int r2, int c2) const {
    return query(r2, c2) - query(r1 - 1, c2)
         - query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
  }
};
```

4.4 Segment Tree Clásico (Recursivo) y Segment Tree Iterativo

- **¿Cuándo usar?**: Cuando necesitas actualizaciones puntuales (`a[pos] = val`) y consultas en rango $[L, R]$ de **cualquier operación asociativa** (mínimo, máximo, suma, GCD, XOR, máxima suma de subarreglo, multiplicación de matrices).
  - Usa el **Iterativo** si quieres escribir pocas líneas y máxima velocidad en tiempo de ejecución ($2N$ de memoria).
  - Usa el **Recursivo** si necesitas caminar sobre el árbol (*walk on Segment Tree*) o extenderlo a Lazy.
- **Complejidad**: Construcción $\mathcal{O}(N)$; `update` y `query` en $\mathcal{O}(\log N)$.

```
// 1. Segment Tree Iterativo (0-indexed, ocupa solo 2*N de memoria)
struct IterSegTree {
  int n;
  const ll NEUTRAL = INF; // INF para min, -INF para max, 0 para suma
  vll t;
  ll op(ll a, ll b) const { return min(a, b); }

  IterSegTree(const vll& a) : n(sz(a)), t(2 * sz(a), NEUTRAL) {
    for (int i = 0; i < n; ++i) t[n + i] = a[i];
    for (int i = n - 1; i >= 1; --i)
      t[i] = op(t[i << 1], t[i << 1 | 1]);
  }
  void update(int p, ll val) { // Asigna a[p] = val (0-indexed)
    for (t[p += n] = val; p > 1; p >>= 1)
      t[p >> 1] = op(t[p], t[p ^ 1]);
  }
  ll query(int l, int r) const { // Rango CERRADO 0-indexed [l, r]
    ll vl = NEUTRAL, vr = NEUTRAL;
    for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
      if (l & 1) vl = op(vl, t[l++]);
      if (r & 1) vr = op(t[--r], vr);
    }
    return op(vl, vr);
  }
};

// 2. Segment Tree Recursivo Clasico (0-indexed)
struct SegTree {
  int n;
  const ll NEUTRAL = 0;
  vll t;
  ll merge(ll a, ll b) const { return a + b; }

  SegTree(const vll& a) : n(sz(a)), t(4 * sz(a), NEUTRAL) {
    if (n > 0) build(1, 0, n - 1, a);
  }
  void build(int v, int l, int r, const vll& a) {
    if (l == r) { t[v] = a[l]; return; }
    int m = (l + r) / 2;
    build(v * 2, l, m, a);
    build(v * 2 + 1, m + 1, r, a);
    t[v] = merge(t[v * 2], t[v * 2 + 1]);
  }
  void update(int v, int l, int r, int pos, ll val) {
    if (l == r) { t[v] = val; return; }
    int m = (l + r) / 2;
    if (pos <= m) update(v * 2, l, m, pos, val);
    else update(v * 2 + 1, m + 1, r, pos, val);
    t[v] = merge(t[v * 2], t[v * 2 + 1]);
  }
  ll query(int v, int l, int r, int ql, int qr) const {
    if (qr < l || r < ql) return NEUTRAL;
    if (ql <= l && r <= qr) return t[v];
    int m = (l + r) / 2;
    return merge(query(v * 2, l, m, ql, qr),
                 query(v * 2 + 1, m + 1, r, ql, qr));
  }
  void update(int pos, ll val) { update(1, 0, n - 1, pos, val); }
  ll query(int ql, int qr) const { return query(1, 0, n - 1, ql, qr); }
};
```

4.5 Segment Tree con Lazy Propagation, Segment Tree 2D y Persistent Segment Tree

- **¿Cuándo usar?**:
  - **Lazy Propagation**: Cuando tan pronto actualizas **todo un rango** $[L, R]$ (ej. sumar `val` a cada elemento en $[L, R]$) también debes intercalar consultas de suma/mínimo/máximo en rango en $\mathcal{O}(\log N)$.
  - **Segment Tree 2D**: Actualizaciones puntuales y consultas de suma/mínimo/GCD en submatrices $(x_0..x_1, y_0..y_1)$ cuando la operación no tiene inverso (si es suma, prefiere Fenwick 2D).
  - **Persistent Segment Tree**: Cuando necesitas **conservar todas las versiones históricas** del arreglo tras cada actualización puntual, o para hallar el K-**-ésimo menor elemento en un subarreglo** $[L, R]$ sin actualizaciones en $\mathcal{O}(\log N)$.
- **Complejidad**: Lazy y Persistente: $\mathcal{O}(\log N)$ por operación (Persistente usa $\mathcal{O}((N + Q) \log N)$ de memoria). 2D: $\mathcal{O}(\log N \log M)$.

```
// 1. Segment Tree con Lazy Propagation (Suma en rango + Query Suma y Min)
struct LazySegTree {
  int n;
  vll sum_t, min_t, lazy;

  LazySegTree(const vll& a)
    : n(sz(a)), sum_t(4 * n, 0), min_t(4 * n, 0), lazy(4 * n, 0) {
    if (n > 0) build(1, 0, n - 1, a);
  }
  void apply(int v, int l, int r, ll val) {
    sum_t[v] += val * (r - l + 1);
    min_t[v] += val;
    lazy[v] += val;
  }
  void push(int v, int l, int r) {
    if (lazy[v] != 0) {
      int m = (l + r) / 2;
      apply(v * 2, l, m, lazy[v]);
      apply(v * 2 + 1, m + 1, r, lazy[v]);
      lazy[v] = 0;
    }
  }
  void pull(int v) {
    sum_t[v] = sum_t[v * 2] + sum_t[v * 2 + 1];
    min_t[v] = min(min_t[v * 2], min_t[v * 2 + 1]);
  }
  void build(int v, int l, int r, const vll& a) {
    if (l == r) { sum_t[v] = min_t[v] = a[l]; return; }
```

```
    int m = (l + r) / 2;
    build(v * 2, l, m, a);
    build(v * 2 + 1, m + 1, r, a);
    pull(v);
  }
  void add_range(int v, int l, int r, int ql, int qr, ll val) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) { apply(v, l, r, val); return; }
    push(v, l, r);
    int m = (l + r) / 2;
    add_range(v * 2, l, m, ql, qr, val);
    add_range(v * 2 + 1, m + 1, r, ql, qr, val);
    pull(v);
  }
  ll query_sum(int v, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return sum_t[v];
    push(v, l, r);
    int m = (l + r) / 2;
    return query_sum(v * 2, l, m, ql, qr)
         + query_sum(v * 2 + 1, m + 1, r, ql, qr);
  }
  ll query_min(int v, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return INF;
    if (ql <= l && r <= qr) return min_t[v];
    push(v, l, r);
    int m = (l + r) / 2;
    return min(query_min(v * 2, l, m, ql, qr),
               query_min(v * 2 + 1, m + 1, r, ql, qr));
  }
  void add_range(int ql, int qr, ll val) {
    add_range(1, 0, n - 1, ql, qr, val);
  }
  ll query_sum(int ql, int qr) { return query_sum(1, 0, n - 1, ql, qr); }
  ll query_min(int ql, int qr) { return query_min(1, 0, n - 1, ql, qr); }
};

// 2. Segment Tree 2D Iterativo (0-indexed: [x0..x1][y0..y1])
struct SegTree2D {
  int n, m;
  const ll NEUTRAL = 0;
  vector<vll> st;
  ll op(ll a, ll b) const { return a + b; }

  SegTree2D(const vector<vll>& a)
    : n(sz(a)), m(n ? sz(a[0]) : 0), st(2 * n, vll(2 * m, 0)) {
    for (int i = 0; i < n; ++i)
      for (int j = 0; j < m; ++j) st[i + n][j + m] = a[i][j];
    for (int i = 0; i < n; ++i)
      for (int j = m - 1; j > 0; --j)
        st[i + n][j] = op(st[i + n][j << 1], st[i + n][j << 1 | 1]);
    for (int i = n - 1; i > 0; --i)
      for (int j = 0; j < 2 * m; ++j)
        st[i][j] = op(st[i << 1][j], st[i << 1 | 1][j]);
  }
  void update(int x, int y, ll v) {
    st[x + n][y + m] = v;
    for (int j = y + m; j > 1; j >>= 1)
      st[x + n][j >> 1] = op(st[x + n][j], st[x + n][j ^ 1]);
    for (int i = x + n; i > 1; i >>= 1)
      for (int j = y + m; j > 0; j >>= 1)
        st[i >> 1][j] = op(st[i][j], st[i ^ 1][j]);
  }
  ll query(int x0, int x1, int y0, int y1) const {
    ll res = NEUTRAL;
    for (int i0 = x0 + n, i1 = x1 + n + 1; i0 < i1; i0 >>= 1, i1 >>= 1) {
      int rows[2], q = 0;
      if (i0 & 1) rows[q++] = i0++;
      if (i1 & 1) rows[q++] = --i1;
      for (int k = 0; k < q; ++k)
        for (int j0 = y0 + m, j1 = y1 + m + 1; j0 < j1; j0 >>= 1, j1 >>= 1)
{
          if (j0 & 1) res = op(res, st[rows[k]][j0++]);
          if (j1 & 1) res = op(res, st[rows[k]][--j1]);
        }
    }
    return res;
  }
};

// 3. Persistent Segment Tree (Suma en versiones historicas)
struct PersistentST {
  struct Node { ll val; int l, r; };
  int n;
  vector<Node> t;
  vi roots;

  PersistentST(const vll& a) : n(sz(a)) {
    t.reserve(n * 22);
    roots.pb(build(0, n - 1, a));
  }
  int build(int tl, int tr, const vll& a) {
    int v = sz(t); t.pb({0, 0, 0});
    if (tl == tr) { t[v].val = a[tl]; return v; }
    int tm = (tl + tr) / 2;
    int left = build(tl, tm, a);
    int right = build(tm + 1, tr, a);
    t[v] = {t[left].val + t[right].val, left, right};
    return v;
  }
  int update(int prv, int tl, int tr, int pos, ll new_val) {
    int v = sz(t); t.pb(t[prv]);
    if (tl == tr) { t[v].val = new_val; return v; }
    int tm = (tl + tr) / 2;
    if (pos <= tm) {
      int nl = update(t[prv].l, tl, tm, pos, new_val);
      t[v].l = nl;
    } else {
      int nr = update(t[prv].r, tm + 1, tr, pos, new_val);
      t[v].r = nr;
    }
    t[v].val = t[t[v].l].val + t[t[v].r].val;
    return v;
  }
  ll query(int v, int tl, int tr, int ql, int qr) const {
    if (qr < tl || tr < ql) return 0;
    if (ql <= tl && tr <= qr) return t[v].val;
    int tm = (tl + tr) / 2;
    return query(t[v].l, tl, tm, ql, qr)
         + query(t[v].r, tm + 1, tr, ql, qr);
  }
  // Crea una nueva version a partir de 'ver' y retorna su ID
  int create_version(int ver, int pos, ll val) {
    int new_root = update(roots[ver], 0, n - 1, pos, val);
    roots.pb(new_root);
    return sz(roots) - 1;
  }
  ll query_version(int ver, int ql, int qr) const {
    return query(roots[ver], 0, n - 1, ql, qr);
  }
};
```

Ejemplo de Input y Uso (Lazy SegTree y Persistent SegTree):

```
Input: A = [1, 2, 3, 4, 5] (0-indexed)
Lazy: sumar +10 en [1, 3] -> A = [1, 12, 13, 14, 5]. Query suma [0, 2] ->
26, min [1, 4] -> 5.
Persistente: version 0 tiene A original. Version 1 cambia pos 2 a 100.
```

```
int main() {
  vll a = {1, 2, 3, 4, 5};
  LazySegTree lst(a);
  lst.add_range(1, 3, 10);
  cout << lst.query_sum(0, 2) << " " << lst.query_min(1, 4) << "\n"; // 26 5

  PersistentST pst(a);              // Version 0: [1, 2, 3, 4, 5]
  int v1 = pst.create_version(0, 2, 100); // Version 1: [1, 2, 100, 4, 5]
  cout << pst.query_version(0, 0, 4) << "\n"; // 15
  cout << pst.query_version(v1, 0, 4) << "\n"; // 112
}
```

4.6 SQRT Decomposition y Algoritmo de Mo (con Optimización de Bloques)

- **¿Cuándo usar?**:
  - **SQRT Decomposition**: Cuando la propiedad por bloques es más fácil de mantener en $\mathcal{O}(\sqrt{N})$ que en un Segment Tree (ej. contar cuántos elementos $\geq X$ hay en $[L, R]$ con actualizaciones puntuales manteniendo bloques ordenados).
  - **Algoritmo de Mo**: Cuando tienes Q **consultas en rango** $[L, R]$ **offline (sin actualizaciones)** de una función difícil de combinar en un Segment Tree (ej. cantidad de elementos distintos, suma de frecuencias al cuadrado $\sum cnt[x]^2$, moda del rango), pero **es fácil agregar o quitar 1 elemento en los bordes** (`add(idx)` / `del(idx)` en $\mathcal{O}(1)$).
- **Complejidad**: $\mathcal{O}((N+Q)\sqrt{N} \cdot \text{costo(add/del)})$.

```
// 1. SQRT Decomposition en Bloques (0-indexed)
struct SqrtDecomp {
  int n, B;
  vll a, b;
  SqrtDecomp(const vll& arr)
    : n(sz(arr)), B((int)sqrt(n) + 1), a(arr), b(B, 0) {
    for (int i = 0; i < n; ++i) b[i / B] += a[i];
  }
  void update(int pos, ll val) { // O(1)
    b[pos / B] += val - a[pos];
    a[pos] = val;
  }
  ll query(int l, int r) const { // O(sqrt(N))
    ll sum = 0;
    int bl = l / B, br = r / B;
    if (bl == br) {
      for (int i = l; i <= r; ++i) sum += a[i];
    } else {
      for (int i = l, end = (bl + 1) * B - 1; i <= end; ++i) sum += a[i];
      for (int i = bl + 1; i < br; ++i) sum += b[i];
      for (int i = br * B; i <= r; ++i) sum += a[i];
    }
    return sum;
  }
};

// 2. Algoritmo de Mo (Offline con ordenamiento serpenteante par/impar)
const int BLOCK_SIZE = 450; // ~ N / sqrt(Q) para N, Q <= 2e5
struct MoQuery {
  int l, r, idx;
  bool operator<(const MoQuery& o) const {
    int b1 = l / BLOCK_SIZE, b2 = o.l / BLOCK_SIZE;
    if (b1 != b2) return b1 < b2;
    return (b1 & 1) ? (r < o.r) : (r > o.r); // Optimiza el puntero R
  }
};

// Ejemplo: Contar cantidad de elementos distintos en cada rango [l, r]
vll solve_mo(const vi& a, vector<MoQuery>& queries) {
  int max_val = *max_element(all(a));
  vi freq(max_val + 1, 0);
  int distinct_cnt = 0;

  auto add = [&](int idx) {
    if (++freq[a[idx]] == 1) distinct_cnt++;
  };
  auto del = [&](int idx) {
    if (--freq[a[idx]] == 0) distinct_cnt--;
  };

  sort(all(queries));
  vll ans(sz(queries));
  int cur_l = 0, cur_r = -1;

  // ¡IMPORTANTE! Primero expandir (cur_l--, cur_r++), luego contraer
  for (const auto& q : queries) {
    while (cur_l > q.l) add(--cur_l);
    while (cur_r < q.r) add(++cur_r);
    while (cur_l < q.l) del(cur_l++);
    while (cur_r > q.r) del(cur_r--);
    ans[q.idx] = distinct_cnt;
  }
  return ans;
}
```

Ejemplo de Input y Uso (Algoritmo de Mo):

```
Input (0-indexed):
A = [1, 2, 1, 3, 2]
Q = 2 consultas: q0 = [0, 2] (distintos en {1,2,1} -> 2), q1 = [1, 4] (en
{2,1,3,2} -> 3)
```

```
int main() {
  vi a = {1, 2, 1, 3, 2};
  vector<MoQuery> q = {{0, 2, 0}, {1, 4, 1}};
  vll res = solve_mo(a, q); // res = {2, 3}
}
```

4.7 Treap (BST Aleatorizado) e Implicit Treap (Reversión y Suma en Rango)

- **¿Cuándo usar?**:
  - **Implicit Treap (Treap con clave implícita por posición)**: Representa un **arreglo dinámico** donde puedes **cortar y pegar subarreglos, invertir el orden de un rango** $[L, R]$ (`reverse`), **sumar un valor a un rango** $[L, R]$, **insertar o borrar elementos en cualquier posición, y consultar la suma en** $[L, R]$, todo en $\mathcal{O}(\log N)$.
- **Complejidad**: $\mathcal{O}(\log N)$ esperado por `split`, `merge`, `insert`, `reverse_range`, `add_range` y `query_sum`.

```
struct ImplicitTreap {
  struct Node {
    ll val, sum, lazy_add;
```

```
    int pr, cnt;
    bool rev;
    Node *l = nullptr, *r = nullptr;
    Node(ll v)
      : val(v), sum(v), lazy_add(0), pr( (int)rng() ), cnt(1), rev(false) {}
  };
  using pnode = Node*;
  pnode root = nullptr;

  int cnt(pnode t) const { return t ? t->cnt : 0; }
  ll sum(pnode t) const { return t ? t->sum : 0; }

  void apply_add(pnode t, ll add_v) {
    if (!t) return;
    t->val += add_v;
    t->sum += add_v * t->cnt;
    t->lazy_add += add_v;
  }
  void apply_rev(pnode t) {
    if (!t) return;
    t->rev ^= 1;
    swap(t->l, t->r);
  }
  void push(pnode t) {
    if (!t) return;
    if (t->rev) {
      apply_rev(t->l);
      apply_rev(t->r);
      t->rev = false;
    }
    if (t->lazy_add != 0) {
      apply_add(t->l, t->lazy_add);
      apply_add(t->r, t->lazy_add);
      t->lazy_add = 0;
    }
  }
  void pull(pnode t) {
    if (!t) return;
    t->cnt = 1 + cnt(t->l) + cnt(t->r);
    t->sum = t->val + sum(t->l) + sum(t->r);
  }
  // Divide 't' en 'l' (primeros k elementos) y 'r' (el resto)
  void split(pnode t, pnode& l, pnode& r, int k) {
    if (!t) { l = r = nullptr; return; }
    push(t);
    if (cnt(t->l) >= k) {
      split(t->l, l, t->l, k);
      r = t;
    } else {
      split(t->r, t->r, r, k - cnt(t->l) - 1);
      l = t;
    }
    pull(t);
  }
  void merge(pnode& t, pnode l, pnode r) {
    push(l); push(r);
    if (!l || !r) t = l ? l : r;
    else if (l->pr > r->pr) { merge(l->r, l->r, r); t = l; }
    else { merge(r->l, l, r->l); t = r; }
    pull(t);
  }
  void push_back(ll val) { merge(root, root, new Node(val)); }

  // Invierte el subarreglo 0-indexed [ql, qr]
  void reverse_range(int ql, int qr) {
    pnode t1, t2, t3;
    split(root, t1, t2, ql);
    split(t2, t2, t3, qr - ql + 1);
    apply_rev(t2);
    merge(root, t1, t2);
    merge(root, root, t3);
  }
  // Suma 'val' al subarreglo 0-indexed [ql, qr]
  void add_range(int ql, int qr, ll val) {
    pnode t1, t2, t3;
    split(root, t1, t2, ql);
    split(t2, t2, t3, qr - ql + 1);
    apply_add(t2, val);
    merge(root, t1, t2);
    merge(root, root, t3);
  }
  // Consulta suma en 0-indexed [ql, qr]
  ll query_sum(int ql, int qr) {
    pnode t1, t2, t3;
    split(root, t1, t2, ql);
    split(t2, t2, t3, qr - ql + 1);
    ll res = sum(t2);
    merge(root, t1, t2);
    merge(root, root, t3);
    return res;
  }
};
```

Ejemplo de Input y Uso (Implicit Treap):

```
Input: Arreglo inicial [10, 20, 30, 40, 50] (0-indexed)
1) Invertir rango [1, 3] -> [10, 40, 30, 20, 50]
2) Consultar suma en [0, 2] -> 10 + 40 + 30 = 80
```

```
int main() {
  ImplicitTreap tr;
  for (ll x : {10, 20, 30, 40, 50}) tr.push_back(x);
  tr.reverse_range(1, 3);
  cout << tr.query_sum(0, 2) << "\n"; // 80
}
```

4.8 Consultas en Árboles: Heavy-Light Decomposition (HLD) y DSU on Tree (Sack)

- **¿Cuándo usar?**:
  - **Heavy-Light Decomposition (HLD)**: Cuando tienes un árbol con valores en los nodos (o aristas) y debes responder **actualizaciones y consultas a lo largo del camino simple entre cualquier par de nodos** (u, v) y/o en el **subárbol completo de** u. Descompone cualquier camino del árbol en a lo sumo $\mathcal{O}(\log N)$ intervalos contiguos de un Segment Tree.
  - **DSU on Tree (Sack / Small-to-Large)**: Cuando tienes **consultas offline sobre subárboles** (sin actualizaciones) donde contar frecuencias por color/valor es fácil (ej. *"¿cuántos vértices en el subárbol de u tienen el color dominante?"*).
- **Complejidad**: HLD: Construcción $\mathcal{O}(N)$, consulta/actualización en camino $\mathcal{O}(\log^2 N)$, en subárbol $\mathcal{O}(\log N)$. Sack: $\mathcal{O}(N \log N)$ total.

```
// 1. Heavy-Light Decomposition (HLD) + Segment Tree Iterativo
struct HLD {
  int n, timer = 0;
  vector<vi> adj;
  vi par, dep, heavy, head, pos, sz_sub;
  IterSegTree st;

  HLD(int n, int root, const vector<vi>& g, const vll& vals)
    : n(n), adj(g), par(n + 1), dep(n + 1, 0), heavy(n + 1, -1),
      head(n + 1), pos(n + 1), sz_sub(n + 1, 1), st(vll(n, 0)) {
    par[root] = root;
    dfs_sz(root);
    dfs_hld(root, root);
    for (int u = 1; u <= n; ++u) st.update(pos[u], vals[u]);
  }
  void dfs_sz(int u) {
    int max_c = 0;
    for (int v : adj[u]) {
      if (v == par[u]) continue;
      par[v] = u;
      dep[v] = dep[u] + 1;
      dfs_sz(v);
      sz_sub[u] += sz_sub[v];
      if (sz_sub[v] > max_c) {
        max_c = sz_sub[v];
        heavy[u] = v;
      }
    }
  }
  void dfs_hld(int u, int h) {
    head[u] = h;
    pos[u] = timer++;
    if (heavy[u] != -1) dfs_hld(heavy[u], h);
    for (int v : adj[u])
      if (v != par[u] && v != heavy[u])
        dfs_hld(v, v);
  }
  void update_node(int u, ll val) { st.update(pos[u], val); }

  // Consulta en el camino entre u y v (minimo en este ejemplo)
  ll query_path(int u, int v) const {
    ll res = st.NEUTRAL;
    for (; head[u] != head[v]; v = par[head[v]]) {
      if (dep[head[u]] > dep[head[v]]) swap(u, v);
      res = st.op(res, st.query(pos[head[v]], pos[v]));
    }
    if (dep[u] > dep[v]) swap(u, v);
    res = st.op(res, st.query(pos[u], pos[v]));
    // NOTA: Si los pesos estan en las ARISTAS (guardados en el hijo),
    // cambiar la linea anterior por:
    // if (u != v) res = st.op(res, st.query(pos[u] + 1, pos[v]));
    return res;
  }
  // Consulta en todo el subarbol de u en O(log N):
  ll query_subtree(int u) const {
    return st.query(pos[u], pos[u] + sz_sub[u] - 1);
  }
};

// 2. DSU on Tree (Sack) — Consultas de frecuencia en subarboles en O(N log N)
const int MAX_SACK = 200005;
vi adj_s[MAX_SACK];
int col_s[MAX_SACK], cnt_s[MAX_SACK], big_s[MAX_SACK];
int tin_s[MAX_SACK], tout_s[MAX_SACK], ver_s[MAX_SACK], t_sack = 0;

int pre_sack(int u, int p) {
  int sz = 1, mx = 0;
  tin_s[u] = ++t_sack;
  ver_s[t_sack] = u;
  big_s[u] = -1;
  for (int v : adj_s[u]) {
    if (v == p) continue;
    int c_sz = pre_sack(v, u);
    sz += c_sz;
    if (c_sz > mx) { mx = c_sz; big_s[u] = v; }
  }
  tout_s[u] = t_sack;
  return sz;
}
void add_node_sack(int u, int delta) {
  cnt_s[col_s[u]] += delta; // Actualizar estructura de frecuencias
}
void dfs_sack(int u, int p, bool keep) {
  for (int v : adj_s[u])
    if (v != p && v != big_s[u]) dfs_sack(v, u, false);
  if (big_s[u] != -1) dfs_sack(big_s[u], u, true);

  for (int v : adj_s[u])
    if (v != p && v != big_s[u])
      for (int t = tin_s[v]; t <= tout_s[v]; ++t)
        add_node_sack(ver_s[t], +1);
  add_node_sack(u, +1);

  // AQUI: cnt_s[c] tiene la cantidad exacta de nodos de color 'c'
  // en el subarbol de 'u'. Responder todas las queries de 'u'.

  if (!keep)
    for (int t = tin_s[u]; t <= tout_s[u]; ++t)
      add_node_sack(ver_s[t], -1);
}
```

Ejemplo de Input y Uso (Heavy-Light Decomposition):

```
Input: Arbol de 5 nodos (raiz 1): aristas (1-2), (1-3), (2-4), (2-5).
Valores en nodos 1..5: [10, 5, 20, 8, 2]
Query minimo en el camino de 4 a 3 (pasa por 4 -> 2 -> 1 -> 3): min(8, 5, 10, 20) = 5.
Update nodo 2 a valor 50 -> nuevo min en camino (4, 3) = min(8, 50, 10, 20) = 8.
```

```
int main() {
  int n = 5;
  vector<vi> g(n + 1);
  g[1] = {2, 3}; g[2] = {1, 4, 5}; g[3] = {1}; g[4] = {2}; g[5] = {2};
  vll vals = {0, 10, 5, 20, 8, 2}; // 1-indexed (vals[0] ignorado)
  HLD hld(n, 1, g, vals);
  cout << hld.query_path(4, 3) << "\n"; // 5
  hld.update_node(2, 50);
  cout << hld.query_path(4, 3) << "\n"; // 8
}
```

4.9 Link-Cut Tree (LCT — Bosques Dinámicos Online)

- **¿Cuándo usar?**: Mantener una colección de árboles disjuntos (bosque dinámico) sujeta a modificaciones en tiempo real: agregar aristas (`link`), eliminar aristas (`cut`), verificar si dos nodos están en el mismo árbol (`connected`), hallar el ancestro común más cercano (`lca`) o cambiar la raíz del árbol (`make_root`).
- **Complejidad**: $\mathcal{O}(\log N)$ amortizado por operación.

```
// Splay Tree Node. 'pp' apunta al padre del arbol representado
struct LCTNode {
  LCTNode *p = 0, *pp = 0, *c[2] = {0, 0};
  bool flip = 0;
  void fix() {
    for (int i = 0; i < 2; ++i) if (c[i]) c[i]->p = this;
  }
  int up() { return p ? (p->c[1] == this) : -1; }
  void push() {
```

```
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    for (int i = 0; i < 2; ++i) if (c[i]) c[i]->flip ^= 1;
  }
  void rot(int i, int b) {
    int h = i ^ b;
    LCTNode *x = c[i], *y = (b == 2 ? x : x->c[h]), *z = (b ? y : x);
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) x->c[h] = y->c[h ^ 1], z->c[h ^ 1] = (b ? x : this);
    y->c[i ^ 1] = (b ? this : x);
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() {
    for (push(); p;) {
      if (p->p) p->p->push();
      p->push(); push();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  LCTNode* first() {
    push();
    return c[0] ? c[0]->first() : (splay(), this);
  }
};

struct LinkCut {
  vector<LCTNode> node;
  LinkCut(int n) : node(n + 1) {}

  LCTNode* access(LCTNode* u) {
    u->splay();
    while (LCTNode* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) pp->c[1]->p = 0, pp->c[1]->pp = pp;
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
  void make_root(int u) {
    access(&node[u]); node[u].splay();
    if (node[u].c[0]) {
      node[u].c[0]->p = 0;
      node[u].c[0]->flip ^= 1;
      node[u].c[0]->pp = &node[u];
      node[u].c[0] = 0; node[u].fix();
    }
  }
  bool connected(int u, int v) {
    return access(&node[u])->first() == access(&node[v])->first();
  }
  void link(int u, int v) {
    if (connected(u, v)) return;
    make_root(u); node[u].pp = &node[v];
  }
  void cut(int u, int v) {
    make_root(v); node[u].splay();
    if (node[u].pp == &node[v]) node[u].pp = 0;
    else {
      node[u].c[0] = node[v].p = 0;
      node[u].fix();
    }
  }
};
```

Bloque 5: Algoritmos de Strings

5.1 Rotación Cíclica Mínima / Minimum Expression (Duval/Booth $\mathcal{O}(N)$)

- **¿Cuándo usar?**: Cuando puedes rotar una cadena circularmente (ej. mover el primer carácter al final cualquier cantidad de veces) y necesitas encontrar **el índice de inicio que produce la cadena lexicográficamente menor** en $\mathcal{O}(N)$ (sirve también para canonizar collares/ciclos isomorfos).
- **Complejidad**: $\mathcal{O}(N)$ tiempo y memoria.

```
// Retorna el indice 0-based donde empieza la menor rotacion ciclica
// Ejemplo: s = "bbaaca" -> retorna 2 (rotacion "aacabb")
int min_cyclic_rotation(string s) {
  int n = sz(s);
  s += s;
  int i = 0, j = 1, k = 0;
  while (i < n && j < n && k < n) {
    if (s[i + k] == s[j + k]) {
      k++;
    } else {
      if (s[i + k] > s[j + k]) i += k + 1;
      else j += k + 1;
      if (i == j) j++;
      k = 0;
    }
  }
  return min(i, j);
}
```

5.2 Double Rolling Hash Polinomial y LCP con Búsqueda Binaria

- **¿Cuándo usar?**: Cuando necesites **comparar si dos subcadenas $S[l_1..r_1]$ y $S[l_2..r_2]$ son iguales en $\mathcal{O}(1)$**, verificar si una subcadena es palíndromo en $\mathcal{O}(1)$ (hasheando S y $\text{rev}(S)$), o **comparar lexicográficamente dos subcadenas en $\mathcal{O}(\log N)$** hallando su Prefijo Común Más Largo (LCP) con Búsqueda Binaria. Usa **dos primos simultáneos** para que la probabilidad de colisión sea $\approx 10^{-18}$.
- **Complejidad**: Preprocesamiento $\mathcal{O}(N)$; `get_hash(l, r)` en $\mathcal{O}(1)$; `lcp` y comparación lexicográfica en $\mathcal{O}(\log N)$.

```
struct PolyHash {
  static const ll M1 = 1000000007LL;
  static const ll M2 = 1000000009LL;
  static const ll BASE = 911382323LL;
  int n;
  vll h1, h2, p1, p2;
  string s;

  PolyHash(const string& str)
    : n(sz(str)), h1(n + 1, 0), h2(n + 1, 0),
      p1(n + 1, 1), p2(n + 1, 1), s(str) {
    for (int i = 0; i < n; ++i) {
      p1[i + 1] = (p1[i] * BASE) % M1;
      p2[i + 1] = (p2[i] * BASE) % M2;
      h1[i + 1] = (h1[i] * BASE + s[i]) % M1;
      h2[i + 1] = (h2[i] * BASE + s[i]) % M2;
    }
  }
  // Hash de s[l..r] (0-indexed, inclusivo) en O(1)
  pll get(int l, int r) const {
    ll r1 = (h1[r + 1] - h1[l] * p1[r - l + 1]) % M1;
    if (r1 < 0) r1 += M1;
    ll r2 = (h2[r + 1] - h2[l] * p2[r - l + 1]) % M2;
    if (r2 < 0) r2 += M2;
    return {r1, r2};
  }
  // Longitud del mayor prefijo comun entre s[i..n-1] y s[j..n-1] en O(log N)
  int lcp(int i, int j) const {
    int low = 1, high = n - max(i, j), ans = 0;
    while (low <= high) {
      int mid = (low + high) / 2;
      if (get(i, i + mid - 1) == get(j, j + mid - 1)) {
        ans = mid; low = mid + 1;
      } else high = mid - 1;
    }
    return ans;
  }
  // Retorna true si s[l1..r1] < s[l2..r2] lexicograficamente en O(log N)
  bool cmp_substr(int l1, int r1, int l2, int r2) const {
    int len1 = r1 - l1 + 1, len2 = r2 - l2 + 1;
    int common = min({lcp(l1, l2), len1, len2});
    if (common == len1 || common == len2) return len1 < len2;
    return s[l1 + common] < s[l2 + common];
  }
};
```

Ejemplo de Uso:

```
int main() {
  string s = "abracadabra";
  PolyHash ph(s);
  bool iguales = (ph.get(0, 3) == ph.get(7, 10)); // "abra" == "abra" (true)
  int pref_comun = ph.lcp(0, 7);                  // 4 ("abra")
}
```

5.3 KMP (Función Prefijo π, Pattern Matching, Periodo) y Autómata KMP

- **¿Cuándo usar?**:
  - **Función Prefijo (π) y KMP**: `pi[i]` es la longitud del mayor prefijo propio de $S[0..i]$ que también es sufijo de $S[0..i]$. Úsalo para:
    1) Encontrar todas las ocurrencias de un patrón P dentro de un texto T en $\mathcal{O}(|P| + |T|)$ determinista.
    2) Hallar el **periodo mínimo** de una cadena: $k = n - \pi[n-1]$ (si $n \bmod k == 0$, el bloque repetido es $S[0..k-1]$).
    3) Contar cuántas veces aparece cada prefijo como subcadena.
  - **Autómata KMP**: Cuando haces **Programación Dinámica construyendo un string carácter por carácter** (ej. *"¿cuántos strings de longitud N NO contienen al patrón P como subcadena?"*). `aut[estado][letra]` te dice a qué estado de KMP pasas en $\mathcal{O}(1)$.
- **Complejidad**: KMP: $\mathcal{O}(N + M)$ tiempo y $\mathcal{O}(N)$ memoria. Autómata KMP: $\mathcal{O}(|P| \cdot \Sigma)$ con $\Sigma = 26$.

```
// 1. Funcion Prefijo pi[0..n-1] en O(N)
vi prefix_function(const string& s) {
  int n = sz(s);
  vi pi(n, 0);
  for (int i = 1; i < n; ++i) {
    int j = pi[i - 1];
    while (j > 0 && s[i] != s[j]) j = pi[j - 1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
  }
  return pi;
}

// 2. Buscar todos los indices 0-based donde 'pat' ocurre en 'text'
vi kmp_search(const string& text, const string& pat) {
  vi pi = prefix_function(pat), matches;
  int m = sz(pat), j = 0;
  for (int i = 0; i < sz(text); ++i) {
    while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
    if (text[i] == pat[j]) j++;
    if (j == m) {
      matches.pb(i - m + 1);
      j = pi[j - 1];
    }
  }
  return matches;
}

// 3. Automata KMP: aut[i][c] = nuevo estado si llevamos 'i' chars de 'p'
//    coincidentes y agregamos la letra ('a' + c).
vector<vi> build_kmp_automaton(const string& p) {
  int m = sz(p);
  vi pi = prefix_function(p);
  vector<vi> aut(m + 1, vi(26, 0));
  for (int i = 0; i <= m; ++i) {
    for (int c = 0; c < 26; ++c) {
      if (i == m || (i > 0 && 'a' + c != p[i]))
        aut[i][c] = aut[pi[i - 1]][c];
      else
        aut[i][c] = i + ('a' + c == p[i]);
    }
  }
  return aut;
}
```

5.4 Z-Algorithm y Algoritmo de Manacher (Palíndromos Pares e Impares)

- **¿Cuándo usar?**:
  - **Z-Algorithm**: `z[i]` almacena directamente la **longitud del prefijo común más largo entre toda la cadena S y el sufijo que empieza en i ($S[i..n-1]$)**. Más intuitivo que KMP cuando el problema habla explícitamente de prefijos comunes con sufijos.
  - **Algoritmo de Manacher**: Calcula en $\mathcal{O}(N)$ el **radio del palíndromo más largo centrado en cada posición** i (tanto para longitud impar como par). Permite saber todos los $O(N)$ palíndromos maximales de una cadena sin usar Hashing.
- **Complejidad**: $\mathcal{O}(N)$ tiempo y memoria.

```cpp
// 1. Z-Algorithm: z[i] = LCP(s[0..n-1], s[i..n-1])
vi z_function(const string& s) {
  int n = sz(s), l = 0, r = 0;
  vi z(n, 0);
  for (int i = 1; i < n; ++i) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) { l = i; r = i + z[i]; }
  }
  return z;
}

// 2. Manacher (Unificado con '#' para pares e impares)
// Transforma "abba" -> "#a#b#b#a#".
// En el vector retornado 'p', p[i] - 1 es la longitud real del mayor
// palindromo centrado en t[i].
// Ademas, devuelve si s[l..r] (0-indexed) es palindromo en O(1)!
struct Manacher {
  vi p;
  Manacher(const string& s) {
    string t = "#";
    for (char c : s) { t += c; t += "#"; }
    int n = sz(t), l = 0, r = -1;
    p.assign(n, 0);
    for (int i = 0; i < n; ++i) {
      int k = (i > r) ? 1 : min(p[l + r - i], r - i + 1);
      while (0 <= i - k && i + k < n && t[i - k] == t[i + k]) k++;
      p[i] = k--;
      if (i + k > r) { l = i - k; r = i + k; }
    }
  }
  // Consulta en O(1) si s[l..r] (0-indexed) es palindromo:
  bool is_pal(int l, int r) const {
    int center = l + r + 1;
    int len = r - l + 1;
    return (p[center] - 1) >= len;
  }
  int max_pal_len() const {
    return *max_element(all(p)) - 1;
  }
};
```

5.5 Trie (Strings y Trie Binario para Max XOR) y Aho-Corasick (Múltiples Patrones)

- **¿Cuándo usar?**:
  - **Trie de Strings**: Insertar un diccionario de cadenas y consultar cuántas palabras tienen a P como prefijo en $\mathcal{O}(|P|)$.
  - **Trie Binario (BitTrie)**: Insertar/eliminar enteros de 60 bits y hallar en $\mathcal{O}(\log(\max A))$ qué número y del conjunto **maximiza (o minimiza)** $x \oplus y$, o cuántos cumplen $x \oplus y < K$.
  - **Aho-Corasick**: Cuando te dan un **diccionario de múltiples patrones** $\{P_1, P_2, ..., P_k\}$ y debes buscar todas sus ocurrencias simultáneamente sobre un texto T (o hacer DP sobre el autómata para contar cadenas que contienen/evitan un conjunto de palabras prohibidas).
- **Complejidad**: Trie: $\mathcal{O}(|S|)$. BitTrie: $\mathcal{O}(\text{MAX_B})$. Aho-Corasick: Construcción $\mathcal{O}(\sum |P_i| \cdot \Sigma)$, búsqueda en texto $\mathcal{O}(|T|)$.

```cpp
// 1. Trie Binario (Max XOR Pair con insercion y borrado)
struct BitTrie {
  static const int B = 60; // 30 para int, 60 para long long
  struct Node { int ch[2] = {-1, -1}, cnt = 0; };
  vector<Node> t = {Node()};

  void update(ll x, int delta) { // delta = +1 insertar, -1 borrar
    int u = 0;
    for (int b = B; b >= 0; --b) {
      int bit = (x >> b) & 1LL;
      if (t[u].ch[bit] == -1) {
        t[u].ch[bit] = sz(t);
        t.emplace_back();
      }
      u = t[u].ch[bit];
      t[u].cnt += delta;
    }
  }
  ll max_xor(ll x) const { // Retorna max(x ^ y) para 'y' en el Trie
    int u = 0;
    ll ans = 0;
    for (int b = B; b >= 0; --b) {
      int bit = (x >> b) & 1LL, want = bit ^ 1;
      if (t[u].ch[want] != -1 && t[t[u].ch[want]].cnt > 0) {
        ans |= (1LL << b);
        u = t[u].ch[want];
      } else {
        u = t[u].ch[bit];
      }
    }
    return ans;
  }
};

// 2. Aho-Corasick (Multi-Pattern Matching + Automaton en O(sum|P| * 26))
struct AhoCorasick {
  struct Node {
    int next[26], link = 0, exit_link = 0, cnt = 0;
    Node() { fill(begin(next), end(next), 0); }
  };
  vector<Node> t = {Node()};

  void insert(const string& s) {
    int u = 0;
    for (char c : s) {
      int ch = c - 'a';
      if (!t[u].next[ch]) {
        t[u].next[ch] = sz(t);
        t.emplace_back();
      }
      u = t[u].next[ch];
    }
    t[u].cnt++; // Cantidad de patrones que terminan exactamente en u
  }
  void build() {
    queue<int> q;
    for (int c = 0; c < 26; ++c)
      if (t[0].next[c]) q.push(t[0].next[c]);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      int f = t[u].link;
      t[u].cnt += t[f].cnt; // Acumula ocurrencias de sufijos
      t[u].exit_link = (t[f].cnt > 0) ? f : t[f].exit_link;
      for (int c = 0; c < 26; ++c) {
        int v = t[u].next[c];
        if (v) {
          t[v].link = t[f].next[c];
          q.push(v);
        } else {
          t[u].next[c] = t[f].next[c]; // Transicion directa de automata
        }
      }
    }
  }
  // Cuenta el total de apariciones de todos los patrones dentro de 'text'
  ll count_matches(const string& text) const {
    int u = 0;
    ll total = 0;
    for (char c : text) {
      u = t[u].next[c - 'a'];
      total += t[u].cnt;
    }
    return total;
  }
};
```

Ejemplo de Input y Uso (Aho-Corasick):

```
Input: Patrones = {"he", "she", "his", "hers"}, Texto = "ushers"
Ocurrencias dentro de "ushers": "she" (1), "he" (1), "hers" (1) -> Total = 3.
```

```cpp
int main() {
  AhoCorasick ac;
  for (string p : {"he", "she", "his", "hers"}) ac.insert(p);
  ac.build();
  cout << ac.count_matches("ushers") << "\n"; // 3
}
```

5.6 Suffix Array + LCP, Suffix Automaton (SAM) y Palindromic Tree (Eertree)

- **¿Cuándo usar?**:
  - **Suffix Array + LCP**: Ordena lexicográficamente todos los sufijos de S en $\mathcal{O}(N \log N)$ y calcula `lcp[i]` (prefijo común más largo entre sufijos adyacentes en el orden).
    - *Fórmula clave*: Número de subcadenas distintas = $\frac{N(N+1)}{2} - \sum_{i=1}^{N} \text{lcp}[i]$.
  - **Suffix Automaton (SAM)**: El arma definitiva de strings en $\mathcal{O}(N)$. Comprime todas las subcadenas de S en un DAG de a lo sumo $2N-1$ estados. Resuelve: número de subcadenas distintas ($\sum(len[u] - len[link[u]])$), cuántas veces aparece cualquier patrón P en $\mathcal{O}(|P|)$, K-ésima subcadena lexicográfica y **Subcadena Común Más Larga (LCS)** entre dos strings en $\mathcal{O}(N)$.
  - **Palindromic Tree (Eertree)**: Estructura donde cada nodo es un **palíndromo distinto** de S (a lo sumo $N+2$ nodos). Cuenta palíndromos distintos y cuántas veces aparece cada uno en $\mathcal{O}(N)$.
- **Complejidad**: Suffix Array $\mathcal{O}(N \log N)$; SAM y Eertree $\mathcal{O}(N)$.

```cpp
// 1. Suffix Array + LCP (Kasai) en O(N log N)
struct SuffixArray {
  vi sa, lcp, rank;
  SuffixArray(const string& s, int lim = 256) {
    int n = sz(s) + 1, k = 0, a, b;
    vi x(all(s) + 1), y(n), ws(max(n, lim));
    sa = lcp = rank = y;
    iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
      p = j;
      for (int i = 0; i < n; ++i)
        y[i] = (sa[i] - j >= 0 ? 0 : n) + sa[i] - j;
      fill(all(ws), 0);
      for (int i = 0; i < n; ++i) ws[x[i]]++;
      for (int i = 1; i < lim; ++i) ws[i] += ws[i - 1];
      for (int i = n; i--; ) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y); p = 1; x[sa[0]] = 0;
      for (int i = 1; i < n; ++i) {
        a = sa[i - 1]; b = sa[i];
        x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
      }
    }
    for (int i = 1; i < n; ++i) rank[sa[i]] = i;
    for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
      for (k && k--, j = sa[rank[i] - 1]; s[i + k] == s[j + k]; k++);
  }
  // Nota: sa[0] es el sufijo vacio '$'. Los sufijos reales estan en sa[1..n].
  // lcp[i] = LCP(sa[i - 1], sa[i]) para i in [1..n].
};

// 2. Suffix Automaton (SAM) en O(N)
struct SuffixAutomaton {
  struct State {
    int len = 0, link = -1;
    int next[26];
    ll cnt = 0; // Frecuencia de aparicion del estado (endpos size)
    State() { fill(begin(next), end(next), -1); }
  };
  vector<State> st;
  int last = 0;

  SuffixAutomaton(const string& s) {
    st.reserve(2 * sz(s) + 1);
    st.emplace_back();
    for (char c : s) extend(c - 'a');
    // Propagar frecuencias (cnt) de estados largos a sus suffix links:
    vi ord(sz(st) - 1);
    iota(all(ord), 1);
    sort(all(ord), [&](int a, int b) { return st[a].len > st[b].len; });
    for (int u : ord) st[st[u].link].cnt += st[u].cnt;
  }
  void extend(int c) {
    int cur = sz(st);
    st.emplace_back();
    st[cur].len = st[last].len + 1;
    st[cur].cnt = 1;
    int p = last;
    while (p != -1 && st[p].next[c] == -1) {
      st[p].next[c] = cur;
      p = st[p].link;
    }
    if (p == -1) {
      st[cur].link = 0;
    } else {
      int q = st[p].next[c];
      if (st[p].len + 1 == st[q].len) {
        st[cur].link = q;
      } else {
        int clone = sz(st);
        st.pb(st[q]);
        st[clone].len = st[p].len + 1;
        st[clone].cnt = 0; // Los clones no suman nueva ocurrencia base
        while (p != -1 && st[p].next[c] == q) {
          st[p].next[c] = clone;
          p = st[p].link;
        }
        st[q].link = st[cur].link = clone;
      }
    }
    last = cur;
  }
```

```cpp
  // Total de subcadenas distintas de S en O(|st|):
  ll count_distinct_substrings() const {
    ll ans = 0;
    for (int i = 1; i < sz(st); ++i)
      ans += st[i].len - st[st[i].link].len;
    return ans;
  }
  // Subcadena comun mas larga (LCS) entre S y otro string T en O(|T|):
  string longest_common_substring(const string& t) const {
    int v = 0, l = 0, best_len = 0, best_pos = 0;
    for (int i = 0; i < sz(t); ++i) {
      int c = t[i] - 'a';
      while (v != 0 && st[v].next[c] == -1) {
        v = st[v].link;
        l = st[v].len;
      }
      if (st[v].next[c] != -1) { v = st[v].next[c]; l++; }
      if (l > best_len) { best_len = l; best_pos = i; }
    }
    return t.substr(best_pos - best_len + 1, best_len);
  }
};

// 3. Palindromic Tree (Eertree) en O(N)
struct PalindromicTree {
  struct Node {
    int next[26], len, link;
    ll cnt = 0;
    Node(int l = 0, int lk = 0) : len(l), link(lk) {
      fill(begin(next), end(next), 0);
    }
  };
  vector<Node> st;
  string s;
  int last = 1;

  PalindromicTree(const string& str) : s(str) {
    st.pb(Node(-1, 0)); // Nodo 0: raiz de longitud impar (-1)
    st.pb(Node(0, 0));  // Nodo 1: raiz de longitud par (0)
    for (int i = 0; i < sz(s); ++i) add(i);
    for (int i = sz(st) - 1; i >= 2; --i)
      st[st[i].link].cnt += st[i].cnt;
  }
  int get_link(int v, int i) const {
    while (i - st[v].len - 1 < 0 || s[i - st[v].len - 1] != s[i])
      v = st[v].link;
    return v;
  }
  void add(int i) {
    int c = s[i] - 'a';
    int cur = get_link(last, i);
    if (!st[cur].next[c]) {
      int nw = sz(st);
      int lk = st[get_link(st[cur].link, i)].next[c];
      st.pb(Node(st[cur].len + 2, max(1, lk)));
      st[cur].next[c] = nw;
    }
    last = st[cur].next[c];
    st[last].cnt++;
  }
  // Cantidad de palindromos DISTINTOS en S: sz(st) - 2
};
```

Ejemplo de Input y Uso (Suffix Automaton):

```
Input: S = "ababa", T = "babab"
Subcadenas distintas de "ababa": 9 ("a", "b", "ab", "ba", "aba", "bab",
"abab", "baba", "ababa")
LCS("ababa", "babab"): "baba" (longitud 4)
```

```cpp
int main() {
  SuffixAutomaton sam("ababa");
  cout << sam.count_distinct_substrings() << "\n";      // 9
  cout << sam.longest_common_substring("babab") << "\n"; // "baba"
}
```

Bloque 6: Grafos, Árboles, Conectividad y Flujos

6.1 Recorridos BFS: Grid BFS (Flood Fill), Multi-Source BFS (Aristas Invertidas) y 0-1 BFS

- **¿Cuándo usar?**:
  - **Grid BFS / Flood Fill con ID**: Recorrer un laberinto/matriz 2D etiquetando cada componente conexa con un `id` único para precalcular métricas de toda la sala (área, perímetro, cuadros `*` en las paredes) y responder Q consultas en $\mathcal{O}(1)$ (ej. CF 598D).
  - **Multi-Source BFS con Inversión de Aristas**: Hallar para **todos los nodos** la distancia mínima hacia **cualquier nodo de un conjunto meta** T. En grafos dirigidos se invierten las aristas ($u \to v$ pasa a $v \to u$) y se meten todas las metas a la cola con `dist = 0` al inicio (ej. CF 1272E).
  - **0-1 BFS (`std::deque`)**: Camino más corto cuando los pesos de las aristas son **únicamente 0 o 1**. Si el peso es 0 haces `push_front` , si es 1 haces `push_back` . Corre en $\mathcal{O}(V+E)$ puro (sin el $\log V$ de Dijkstra).
- **Complejidad**: $\mathcal{O}(V+E)$ (o $\mathcal{O}(N \cdot M)$ en grillas).

```cpp
// 1. Grid BFS / Flood Fill con Identificador de Componentes
const int MAX_R = 1005, MAX_C = 1005;
int n_g, m_g, comp_g[MAX_R][MAX_C], ans_comp[MAX_R * MAX_C];
char grid_g[MAX_R][MAX_C];
int dx4[] = {-1, 1, 0, 0}, dy4[] = {0, 0, -1, 1};

void flood_fill_bfs(int sr, int sc, int id) {
  queue<pii> q;
  q.push({sr, sc});
  comp_g[sr][sc] = id;
  int metric = 0;
  while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d = 0; d < 4; ++d) {
      int nr = r + dx4[d], nc = c + dy4[d];
      if (nr < 0 || nr >= n_g || nc < 0 || nc >= m_g) continue;
      if (grid_g[nr][nc] == '*') {
        metric++; // Ejemplo: contar paredes '*' adyacentes
      } else if (grid_g[nr][nc] == '.' && comp_g[nr][nc] == -1) {
        comp_g[nr][nc] = id;
        q.push({nr, nc});
      }
    }
  }
  ans_comp[id] = metric;
}

// 2. Multi-Source BFS (con aristas invertidas adj_rev si es dirigido)
vi adj_rev[200005];
int dist_ms[200005];

void multi_source_bfs(const vi& sources, int n) {
  fill(dist_ms + 1, dist_ms + n + 1, -1);
  queue<int> q;
  for (int s : sources) { dist_ms[s] = 0; q.push(s); }
  while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj_rev[u]) {
      if (dist_ms[v] == -1) {
        dist_ms[v] = dist_ms[u] + 1;
        q.push(v);
      }
    }
  }
}

// 3. 0-1 BFS en Grafo General o Grilla con Reconstruccion de Camino
vector<pii> adj01[200005]; // {vecino, peso_0_o_1}
int dist01[200005], par01[200005];

void bfs_01(int s, int n) {
  fill(dist01 + 1, dist01 + n + 1, INF_INT);
  fill(par01 + 1, par01 + n + 1, -1);
  deque<int> dq;
  dist01[s] = 0;
  dq.push_front(s);
  while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : adj01[u]) {
      if (dist01[u] + w < dist01[v]) {
        dist01[v] = dist01[u] + w;
        par01[v] = u;
        if (w == 0) dq.push_front(v);
        else dq.push_back(v);
      }
    }
  }
}
vi get_path_01(int target) {
  if (dist01[target] == INF_INT) return {};
  vi path;
  for (int cur = target; cur != -1; cur = par01[cur]) path.pb(cur);
  reverse(all(path));
  return path;
}
```

Ejemplo de Input y Uso (0-1 BFS):

```
Input: N = 4 nodos, M = 4 aristas (u v w con w in {0, 1}):
4 4
1 2 1
1 3 1
2 3 0
3 4 0
Camino mas corto de 1 a 4: 1 -> 2 -> 3 -> 4 (costo total = 1 + 0 + 0 = 1).
```

```cpp
int main() {
  int n = 4;
  adj01[1] = {{2, 1}, {3, 1}};
  adj01[2] = {{1, 1}, {3, 0}};
  adj01[3] = {{1, 1}, {2, 0}, {4, 0}};
  adj01[4] = {{3, 0}};
  bfs_01(1, n);
  vi path = get_path_01(4); // [1, 2, 3, 4], dist01[4] == 1
}
```

6.2 Patrones DFS: Grafo Bipartito, Ciclos, Euler Tour (`tin` / `tout`), DFS en Complemento y Conteo de Triángulos

- **¿Cuándo usar?**:
  - **Bipartito (2-Coloreo)**: Dividir nodos en 2 grupos sin aristas internas (equivalentemente: verificar que **no existan ciclos de longitud impar**).
  - **Detección de Ciclos en Dirigido**: Verificar si hay dependencias circulares usando 3 colores (`0 = no visitado` , `1 = en la pila actual` , `2 = cerrado`).
  - **Euler Tour (`tin` / `tout`)**: Aplanar un árbol en un arreglo 1D para que el subárbol de u sea el intervalo contiguo `[tin[u], tout[u]]` y responder si u es ancestro de v en $\mathcal{O}(1)$.
  - **DFS en Grafo Complemento**: Cuando te dan las M aristas **prohibidas** de un grafo completo de $N = 2 \cdot 10^5$ nodos y debes hallar las componentes conexas de las aristas **permitidas** en $\mathcal{O}((N+M)\log N)$.
  - **Conteo de Triángulos en $\mathcal{O}(M\sqrt{M})$**: Contar todos los ciclos de longitud 3 (u, v, w) en un grafo dispersо orientando las aristas de menor a mayor grado.
- **Complejidad**: Bipartito/Ciclos/Euler Tour: $\mathcal{O}(V+E)$; Complemento: $\mathcal{O}((V+E)\log V)$; Triángulos: $\mathcal{O}(M\sqrt{M})$.

```cpp
// 1. Grafo Bipartito (2-Coloreo con colores 1 y 2)
vi adj_dfs[200005];
int col_bip[200005]; // Inicializar en 0
bool dfs_bipartite(int u, int c) {
  col_bip[u] = c;
  for (int v : adj_dfs[u]) {
    if (col_bip[v] == 0) {
      if (!dfs_bipartite(v, 3 - c)) return false;
    } else if (col_bip[v] == col_bip[u]) return false;
  }
  return true;
}

// 2. Ciclo en Grafo Dirigido (0 = blanco, 1 = gris/activo, 2 = negro)
int state_dir[200005];
bool dfs_dir_cycle(int u) {
  state_dir[u] = 1;
  for (int v : adj_dfs[u]) {
    if (state_dir[v] == 1) return true; // Back-edge encontrada
    if (state_dir[v] == 0 && dfs_dir_cycle(v)) return true;
  }
  state_dir[u] = 2;
  return false;
}

// 3. Euler Tour en Arboles (tin, tout y ancestralidad en O(1))
int tin_e[200005], tout_e[200005], timer_e = 0;
void dfs_euler(int u, int p) {
  tin_e[u] = ++timer_e;
  for (int v : adj_dfs[u])
    if (v != p) dfs_euler(v, u);
  tout_e[u] = timer_e;
}
bool is_ancestor(int u, int v) { // ¿u es ancestro de v?
  return tin_e[u] <= tin_e[v] && tout_e[v] <= tout_e[u];
}

// 4. Componentes Conexas en el Grafo Complemento en O((N + M) log N)
vector<vi> complement_components(int n, vector<vi>& forbidden_adj) {
  for (int i = 1; i <= n; ++i) sort(all(forbidden_adj[i]));
  set<int> unvis;
  for (int i = 1; i <= n; ++i) unvis.insert(i);
  vector<vi> comps;
```

```
  for (int i = 1; i <= n; ++i) {
    if (!unvis.count(i)) continue;
    vi comp;
    queue<int> q;
    q.push(i);
    unvis.erase(i);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      comp.pb(u);
      for (auto it = unvis.begin(); it != unvis.end(); ) {
        int v = *it;
        if (!binary_search(all(forbidden_adj[u]), v)) {
          q.push(v);
          it = unvis.erase(it); // Borra en O(1) amortizado!
        } else ++it;
      }
    }
    comps.pb(comp);
  }
  return comps;
}

// 5. Contar Triangulos (3-ciclos) en un grafo en O(M * sqrt(M))
ll count_triangles(int n, const vector<pii>& edges) {
  vi deg(n + 1, 0), mark(n + 1, 0);
  for (auto [u, v] : edges) { deg[u]++; deg[v]++; }
  vector<vi> dag(n + 1);
  for (auto [u, v] : edges) {
    if (tie(deg[u], u) > tie(deg[v], v)) swap(u, v);
    dag[u].pb(v); // Orienta de menor grado a mayor grado
  }
  ll triangles = 0;
  for (int u = 1; u <= n; ++u) {
    for (int v : dag[u]) mark[v] = u;
    for (int v : dag[u])
      for (int w : dag[v])
        if (mark[w] == u) triangles++;
  }
  return triangles;
}
```

6.3 Caminos Más Cortos: Dijkstra, Bellman-Ford / SPFA (Ciclo Negativo) y Floyd-Warshall

- **¿Cuándo usar?**:
  - **Dijkstra**: Un origen S hacia todos los demás en grafos con **pesos** ≥ 0. $\mathcal{O}((V+E)\log V)$.
  - **Bellman-Ford / SPFA**: Cuando existen **aristas con peso negativo** o el problema pide **detectar y reconstruir un ciclo negativo** (o buscar un ciclo de ganancia positiva negando los pesos). $\mathcal{O}(V \cdot E)$.
  - **Floyd-Warshall**: Todos los pares de caminos más cortos (u, v) en grafos pequeños ($V \leq 500$). Permite también cerrar transitivamente un grafo o hallar el ciclo de menor peso. $\mathcal{O}(V^3)$.

```
// 1. Dijkstra con Reconstruccion de Camino (1-indexed)
const int MAX_V_DIJ = 200005;
vector<pair<int, ll>> adj_dij[MAX_V_DIJ]; // {vecino, peso}
ll dist_dij[MAX_V_DIJ];
int par_dij[MAX_V_DIJ];

void dijkstra(int s, int n) {
  fill(dist_dij + 1, dist_dij + n + 1, INF);
  fill(par_dij + 1, par_dij + n + 1, -1);
  priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
  dist_dij[s] = 0;
  pq.push({0, s});
  while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist_dij[u]) continue;
    for (auto [v, w] : adj_dij[u]) {
      if (dist_dij[u] + w < dist_dij[v]) {
        dist_dij[v] = dist_dij[u] + w;
        par_dij[v] = u;
        pq.push({dist_dij[v], v});
      }
    }
  }
}
vi get_path_dij(int target) {
  if (dist_dij[target] == INF) return {};
  vi path;
  for (int cur = target; cur != -1; cur = par_dij[cur]) path.pb(cur);
  reverse(all(path));
  return path;
}

// 2. Bellman-Ford con Reconstruccion de Ciclo Negativo Alcanzable
// (Si quieres detectar CUALQUIER ciclo negativo en el grafo aunque no sea
// alcanzable desde 's', inicializa todos los d[1..n] = 0 en vez de INF).
struct BFEdge { int u, v; ll w; };
bool bellman_ford_cycle(int s, int n, const vector<BFEdge>& edges,
                        vll& d, vi& neg_cycle) {
  d.assign(n + 1, INF);
  vi p(n + 1, -1);
  d[s] = 0;
  int x = -1;
  for (int i = 1; i <= n; ++i) {
    x = -1;
    for (const auto& e : edges) {
      if (d[e.u] < INF && d[e.u] + e.w < d[e.v]) {
        d[e.v] = max(-INF, d[e.u] + e.w);
        p[e.v] = e.u;
        x = e.v;
      }
    }
  }
  if (x == -1) return false; // No hay ciclo negativo
  for (int i = 0; i < n; ++i) x = p[x]; // Entrar con certeza al ciclo
  neg_cycle.clear();
  for (int cur = x; ; cur = p[cur]) {
    neg_cycle.pb(cur);
    if (cur == x && sz(neg_cycle) > 1) break;
  }
  reverse(all(neg_cycle));
  return true;
}

// 3. Floyd-Warshall (All-Pairs Shortest Path en O(V^3))
void floyd_warshall(int n, vector<vll>& d) {
  // Precondicion: d[i][i] = 0, d[u][v] = min_peso (o INF si no hay arista)
  for (int k = 1; k <= n; ++k)
    for (int i = 1; i <= n; ++i)
      if (d[i][k] < INF)
        for (int j = 1; j <= n; ++j)
          if (d[k][j] < INF)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

Ejemplo de Input y Uso (Dijkstra y Bellman-Ford):

```
Input Dijkstra (4 nodos, 4 aristas):
4 4
1 2 2
2 3 1
1 3 5
3 4 3
Ruta 1 -> 4: [1, 2, 3, 4], costo = 6.
```

```
int main() {
  int n = 4;
  adj_dij[1] = {{2, 2}, {3, 5}};
  adj_dij[2] = {{1, 2}, {3, 1}};
  adj_dij[3] = {{2, 1}, {1, 5}, {4, 3}};
  adj_dij[4] = {{3, 3}};
  dijkstra(1, n);
  vi path = get_path_dij(4); // [1, 2, 3, 4], dist_dij[4] == 6
}
```

6.4 Árboles: Diámetro (Doble BFS), MST (Kruskal y Prim), LCA Binary Lifting y Centroid Decomposition

- **¿Cuándo usar?**:
  - **Diámetro de un Árbol (Doble BFS)**: Hallar el camino más largo entre cualquier par de nodos de un árbol y reconstruir su ruta en $\mathcal{O}(N)$ (el centro del árbol está en la mitad de este camino).
  - **MST (Kruskal / Prim)**: Conectar todos los nodos de un grafo con el menor costo total de aristas en $\mathcal{O}(E \log E)$.
  - **LCA con Binary Lifting**: Responder en $\mathcal{O}(\log N)$ cuál es el **Ancestro Común Más Bajo** entre u y v, cuál es el k**-ésimo ancestro** de u, o cuál es la **distancia/número de aristas entre** u **y** v en un árbol estático.
  - **Centroid Decomposition**: Divide y vencerás sobre árboles. Crea un "árbol de centroides" de altura $\mathcal{O}(\log N)$ para contar caminos que cumplen cierta propiedad (ej. caminos de longitud exactamente K) o actualizar/consultar el nodo marcado más cercano en $\mathcal{O}(\log N)$.
- **Complejidad**: Diámetro $\mathcal{O}(N)$; Kruskal/Prim $\mathcal{O}(E \log E)$; LCA y Centroid Decomposition: Preprocesamiento $\mathcal{O}(N \log N)$, consultas $\mathcal{O}(\log N)$.

```
// 1. Diametro de un Arbol (Doble BFS con reconstruccion de ruta)
vi adj_tr[200005];
int par_tr[200005];

pii bfs_farthest(int start, int n) {
  vi d(n + 1, -1);
  queue<int> q;
  q.push(start); d[start] = 0; par_tr[start] = -1;
  int far = start;
  while (!q.empty()) {
    int u = q.front(); q.pop();
    if (d[u] > d[far]) far = u;
    for (int v : adj_tr[u]) {
      if (d[v] == -1) {
        d[v] = d[u] + 1;
        par_tr[v] = u;
        q.push(v);
      }
    }
  }
  return {far, d[far]};
}
vi get_diameter_path(int n) {
  pii p1 = bfs_farthest(1, n);
  pii p2 = bfs_farthest(p1.fi, n);
  vi path;
  for (int cur = p2.fi; cur != -1; cur = par_tr[cur]) path.pb(cur);
  reverse(all(path));
  return path; // Longitud del diametro en aristas = sz(path) - 1
}

// 2. Kruskal MST (usa struct DSU de la seccion 4.1)
struct KrEdge {
  int u, v; ll w;
  bool operator<(const KrEdge& o) const { return w < o.w; }
};
ll kruskal_mst(int n, vector<KrEdge>& edges, vector<KrEdge>& mst) {
  sort(all(edges));
  DSU dsu(n);
  ll cost = 0;
  for (const auto& e : edges)
    if (dsu.unite(e.u, e.v)) { cost += e.w; mst.pb(e); }
  return (dsu.num_comp == 1) ? cost : -1;
}

// 3. LCA (Lowest Common Ancestor) + K-th Ancestor + Distancia en Arbol
struct LCA {
  int n, LOG;
  vi dep;
  vector<vi> up;

  LCA(int n, int root, const vector<vi>& adj) : n(n) {
    LOG = __lg(max(1, n)) + 1;
    dep.assign(n + 1, 0);
    up.assign(n + 1, vi(LOG, root));
    dfs(root, root, adj);
  }
  void dfs(int u, int p, const vector<vi>& adj) {
    up[u][0] = p;
    for (int j = 1; j < LOG; ++j)
      up[u][j] = up[up[u][j - 1]][j - 1];
    for (int v : adj[u]) {
      if (v == p) continue;
      dep[v] = dep[u] + 1;
      dfs(v, u, adj);
    }
  }
  int kth_ancestor(int u, int k) const {
    for (int j = 0; j < LOG; ++j)
      if ((k >> j) & 1) u = up[u][j];
    return u;
  }
  int lca(int u, int v) const {
    if (dep[u] < dep[v]) swap(u, v);
    u = kth_ancestor(u, dep[u] - dep[v]);
    if (u == v) return u;
    for (int j = LOG - 1; j >= 0; --j)
      if (up[u][j] != up[v][j]) { u = up[u][j]; v = up[v][j]; }
    return up[u][0];
  }
  int dist(int u, int v) const {
    return dep[u] + dep[v] - 2 * dep[lca(u, v)];
  }
};

// 4. Centroid Decomposition (Arbol de Centroides en O(N log N))
struct CentroidDecomp {
  int n;
  vector<vi> adj;
  vi sz_c, par_c;
  vector<bool> rem;
```

```
  CentroidDecomp(int n, const vector<vi>& g)
    : n(n), adj(g), sz_c(n + 1, 0), par_c(n + 1, 0), rem(n + 1, false) {
    build(1, 0);
  }
  int dfs_sz(int u, int p) {
    sz_c[u] = 1;
    for (int v : adj[u])
      if (v != p && !rem[v]) sz_c[u] += dfs_sz(v, u);
    return sz_c[u];
  }
  int get_centroid(int u, int p, int tot) const {
    for (int v : adj[u])
      if (v != p && !rem[v] && sz_c[v] * 2 > tot)
        return get_centroid(v, u, tot);
    return u;
  }
  void build(int u, int p) {
    int tot = dfs_sz(u, 0);
    int c = get_centroid(u, 0, tot);
    par_c[c] = p; // Padre en el arbol de centroides (0 si es la raiz)
    rem[c] = true;
    for (int v : adj[c])
      if (!rem[v]) build(v, c);
  }
};
```

Ejemplo de Input y Uso (LCA y Distancia en Árbol):

```
Input: Arbol de 5 nodos (raiz 1): 1-2, 2-3, 3-4, 3-5.
Consultas: lca(4, 5) -> 3, dist(1, 4) -> 3, kth_ancestor(4, 2) -> 2.
```

```
int main() {
  int n = 5;
  vector<vi> adj(n + 1);
  auto add = [&](int u, int v) { adj[u].pb(v); adj[v].pb(u); };
  add(1, 2); add(2, 3); add(3, 4); add(3, 5);
  LCA tree_lca(n, 1, adj);
  cout << tree_lca.lca(4, 5) << " " << tree_lca.dist(1, 4) << " "
       << tree_lca.kth_ancestor(4, 2) << "\n"; // 3 3 2
}
```

6.5 Conectividad, DAGs y 2-SAT: Topological Sort, Puentes/Articulación, Biconexas, SCC (Tarjan), 2-SAT y Camino Euleriano

- **¿Cuándo usar?**:
  - **Orden Topológico (Kahn BFS)**: Ordenar tareas con dependencias ($u \to v$ significa u antes que v) en un DAG, o detectar si hay un ciclo dirigido (`sz(order) < n`). Si piden el **orden topológico lexicográficamente menor**, cambia `queue<int>` por `priority_queue<int, vi, greater<int>>` .
  - **Puentes y Puntos de Articulación (Tarjan No Dirigido)**: Hallar aristas críticas (`low[v] > tin[u]`) o vértices críticos (`low[v] >= tin[u]`) cuya eliminación desconecta el grafo no dirigido.
  - **SCC (Componentes Fuertemente Conexas - Tarjan Dirigido)**: Comprimir un grafo dirigido con ciclos en un **DAG de componentes** (donde dentro de cada SCC todo nodo llega a todo otro nodo).
  - **2-SAT**: Asignar valores booleanos $\{V, F\}$ a N variables sujetas a cláusulas de tamaño 2 ($a \vee b$, $a \Rightarrow b$, $a = b$, $a \neq b$). Se resuelve en $\mathcal{O}(N + M)$ usando Tarjan SCC.
  - **Camino / Ciclo Euleriano (Hierholzer)**: Recorrer **todas las aristas del grafo exactamente una vez** en $\mathcal{O}(V + E)$.
- **Complejidad**: Todos estos algoritmos toman $\mathcal{O}(V + E)$ tiempo y memoria.

```
// 1. Orden Topologico (Kahn BFS - detecta ciclos si sz(order) < n)
vi topological_sort(int n, const vector<vi>& adj) {
  vi in_deg(n + 1, 0), order;
  for (int u = 1; u <= n; ++u)
    for (int v : adj[u]) in_deg[v]++;
  queue<int> q; // Usar Min-Heap si piden orden lexicografico menor
  for (int i = 1; i <= n; ++i)
    if (in_deg[i] == 0) q.push(i);
  while (!q.empty()) {
    int u = q.front(); q.pop();
    order.pb(u);
    for (int v : adj[u])
      if (--in_deg[v] == 0) q.push(v);
  }
  return order;
}

// 2. Puentes y Puntos de Articulacion en Grafo No Dirigido (Tarjan)
```

```
struct BridgesAndCuts {
  int n, timer = 0;
  vi tin, low;
  vector<bool> is_cut;
  vector<pii> bridges;

  BridgesAndCuts(int n, const vector<vi>& adj)
    : n(n), tin(n + 1, 0), low(n + 1, 0), is_cut(n + 1, false) {
    for (int i = 1; i <= n; ++i)
      if (!tin[i]) dfs(i, -1, adj);
  }
  void dfs(int u, int p, const vector<vi>& adj) {
    tin[u] = low[u] = ++timer;
    int children = 0;
    for (int v : adj[u]) {
      if (v == p) continue;
      if (tin[v]) {
        low[u] = min(low[u], tin[v]);
      } else {
        children++;
        dfs(v, u, adj);
        low[u] = min(low[u], low[v]);
        if (low[v] > tin[u]) bridges.pb({u, v});
        if (p != -1 && low[v] >= tin[u]) is_cut[u] = true;
      }
    }
    if (p == -1 && children > 1) is_cut[u] = true;
  }
};

// 3. Tarjan SCC (Componentes Fuertemente Conexas en Grafo Dirigido)
// Nota: comp_id[u] queda en orden topologico INVERSO (scc_cnt..1)
struct TarjanSCC {
  int n, timer = 0, scc_cnt = 0;
  vi tin, low, comp_id, st;
  vector<bool> in_st;

  TarjanSCC(int n, const vector<vi>& adj)
    : n(n), tin(n + 1, 0), low(n + 1, 0),
      comp_id(n + 1, 0), in_st(n + 1, false) {
    for (int i = 1; i <= n; ++i)
      if (!tin[i]) dfs(i, adj);
  }
  void dfs(int u, const vector<vi>& adj) {
    tin[u] = low[u] = ++timer;
    st.pb(u); in_st[u] = true;
    for (int v : adj[u]) {
      if (!tin[v]) {
        dfs(v, adj);
        low[u] = min(low[u], low[v]);
      } else if (in_st[v]) {
        low[u] = min(low[u], tin[v]);
      }
    }
    if (low[u] == tin[u]) {
      scc_cnt++;
      while (true) {
        int v = st.back(); st.pop_back();
        in_st[v] = false;
        comp_id[v] = scc_cnt;
        if (u == v) break;
      }
    }
  }
};

// 4. 2-SAT Solver en O(N + M) (Variables 0-indexed: 0..n-1)
struct TwoSAT {
  int n;
  vector<vi> adj;
  vector<bool> ans;

  TwoSAT(int n) : n(n), adj(2 * n + 1), ans(n, false) {}
  // Nodo para variable 'x' (0..n-1):
  // Si val == true -> nodo (x + 1); si val == false -> nodo (x + 1 + n)
  int var_node(int x, bool val) const { return val ? (x + 1) : (x + 1 +
n); }
  int neg(int u) const { return (u <= n) ? (u + n) : (u - n); }

  void add_imp(int u, int v) {
    adj[u].pb(v);
    adj[neg(v)].pb(neg(u));
  }
  // (x == val_x) OR (y == val_y)
  void add_or(int x, bool vx, int y, bool vy) {
    int u = var_node(x, vx), v = var_node(y, vy);
    add_imp(neg(u), v);
  }
  void set_val(int x, bool vx) { add_or(x, vx, x, vx); }
  void make_eq(int x, int y) {
    add_or(x, true, y, false);
    add_or(x, false, y, true);
  }
  void make_diff(int x, int y) {
    add_or(x, true, y, true);
    add_or(x, false, y, false);
  }
  bool solve() {
    TarjanSCC scc(2 * n, adj);
    for (int i = 0; i < n; ++i) {
      int u = i + 1, nu = i + 1 + n;
      if (scc.comp_id[u] == scc.comp_id[nu]) return false;
      // En Tarjan, menor comp_id significa mas abajo en el DAG (sink)
      ans[i] = (scc.comp_id[u] < scc.comp_id[nu]);
    }
    return true;
  }
};

// 5. Camino / Ciclo Euleriano Dirigido (Hierholzer en O(V + E))
// Condicion: A lo sumo 1 nodo con out - in == 1 (inicio),
// a lo sumo 1 nodo con in - out == 1 (fin), y el resto in == out.
vi eulerian_directed(int n, int m, const vector<pii>& edges) {
  vector<vi> adj(n + 1);
  vi in(n + 1, 0), out(n + 1, 0);
  for (auto [u, v] : edges) { adj[u].pb(v); out[u]++; in[v]++; }

  int start = 1, plus1 = 0, minus1 = 0;
  for (int i = 1; i <= n; ++i) {
    if (out[i] > 0 && start == 1 && out[start] == 0) start = i;
    if (out[i] - in[i] == 1) { plus1++; start = i; }
    else if (in[i] - out[i] == 1) minus1++;
    else if (in[i] != out[i]) return {};
  }
  if (!((plus1 == 0 && minus1 == 0) || (plus1 == 1 && minus1 == 1)))
    return {};

  vi path, st = {start};
  while (!st.empty()) {
    int u = st.back();
    if (!adj[u].empty()) {
      int v = adj[u].back();
      adj[u].pop_back();
      st.pb(v);
    } else {
      path.pb(u);
      st.pop_back();
    }
  }
  if (sz(path) != m + 1) return {}; // El grafo de aristas no era conexo
  reverse(all(path));
  return path;
}
```

Ejemplo de Input y Uso (2-SAT):

```
Input: 3 variables booleanas (x0, x1, x2).
Clausulas: (x0 = True OR x1 = True), (x0 != x1), (x1 = True => x2 = False).
```

```
int main() {
  TwoSAT sat(3);
  sat.add_or(0, true, 1, true);   // x0 v x1
  sat.make_diff(0, 1);            // x0 != x1
  sat.add_or(1, false, 2, false); // x1 => !x2 (equivale a !x1 v !x2)
  if (sat.solve()) {
    // sat.ans[0], sat.ans[1], sat.ans[2] contiene una asignacion valida
  }
}
```

6.6 Flujos y Matching: Dinic, Min-Cost Max-Flow (MCMF), Kuhn (MCBM) + König, Algoritmo Húngaro, Stoer-Wagner y Blossom

- **¿Cuándo usar?**:
  - **Dinic**: Flujo Máximo / Corte Mínimo en redes con capacidades. Corre en $\mathcal{O}(V^2 E)$ general y $\mathcal{O}(E\sqrt{V})$ **en redes unitarias / Matching Bipartito** (¡el más rápido para Matching Bipartito grande!).
  - **Min-Cost Max-Flow (MCMF)**: Cuando cada arista tiene **capacidad** cap **y costo por unidad** $cost$, y quieres enviar la máxima cantidad de flujo (o exactamente K unidades) de S a T minimizando el costo total.
  - **Kuhn ($\mathcal{O}(V \cdot E)$) + Teorema de König**: Matching Bipartito corto de programar cuando $N, M \leq 2000$. Incluye reconstrucción de **Minimum Vertex Cover** y **Maximum Independent Set** en grafos bipartitos.

- **Algoritmo Húngaro ($\mathcal{O}(N^3)$)**: Problema de Asignación $N \times M$: emparejar cada fila con una columna distinta minimizando (o maximizando) la suma de costos de la matriz. Más rápido y directo que MCMF en matrices densas.
- **Blossom ($\mathcal{O}(V^3)$): Maximum Matching en Grafos Generales (NO bipartitos)** (contiene ciclos impares).

- **Complejidad**: Dinic $\mathcal{O}(V^2E)$ (o $\mathcal{O}(E\sqrt{V})$); MCMF $\mathcal{O}(F \cdot E \log V)$; Húngaro $\mathcal{O}(N^2M)$; Blossom $\mathcal{O}(VE)$.

```
// 1. Dinic (Max Flow + Min Cut + Bipartite Matching en O(E sqrt(V)))
struct Dinic {
  struct Edge { int to, rev; ll cap, flow = 0; };
  int n, s, t;
  vector<vector<Edge>> g;
  vi lvl, ptr;

  Dinic(int n, int s, int t) : n(n), s(s), t(t), g(n + 1), lvl(n + 1), ptr(n
+ 1) {}
  void add_edge(int u, int v, ll cap) {
    g[u].pb({v, sz(g[v]), cap, 0});
    g[v].pb({u, sz(g[u]) - 1, 0, 0}); // Cambiar 0 por cap si es no dirigido
  }
  bool bfs() {
    fill(all(lvl), -1);
    queue<int> q;
    lvl[s] = 0; q.push(s);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (const auto& e : g[u])
        if (e.cap - e.flow > 0 && lvl[e.to] == -1) {
          lvl[e.to] = lvl[u] + 1;
          q.push(e.to);
        }
    }
    return lvl[t] != -1;
  }
  ll dfs(int u, ll pushed) {
    if (pushed == 0 || u == t) return pushed;
    for (int& cid = ptr[u]; cid < sz(g[u]); ++cid) {
      auto& e = g[u][cid];
      if (lvl[u] + 1 != lvl[e.to] || e.cap - e.flow == 0) continue;
      ll tr = dfs(e.to, min(pushed, e.cap - e.flow));
      if (tr == 0) continue;
      e.flow += tr;
      g[e.to][e.rev].flow -= tr;
      return tr;
    }
    return 0;
  }
  ll max_flow() {
    ll flow = 0;
    while (bfs()) {
      fill(all(ptr), 0);
      while (ll pushed = dfs(s, INF)) flow += pushed;
    }
    return flow;
  }
  // Retorna true si 'u' queda del lado de la fuente S en el Min-Cut
  // (Llamar DESPUES de max_flow()):
  bool left_of_min_cut(int u) const { return lvl[u] != -1; }
};

// 2. Min-Cost Max-Flow (MCMF con SPFA/Potenciales)
struct MCMF {
  struct Edge { int to, rev; ll cap, cost, flow = 0; };
  int n;
  vector<vector<Edge>> g;
  vll dist, cap_pushed;
  vi par_u, par_e;
  vector<bool> in_q;

  MCMF(int n)
    : n(n), g(n + 1), dist(n + 1), cap_pushed(n + 1),
      par_u(n + 1), par_e(n + 1), in_q(n + 1) {}

  void add_edge(int u, int v, ll cap, ll cost) {
    g[u].pb({v, sz(g[v]), cap, cost, 0});
    g[v].pb({u, sz(g[u]) - 1, 0, -cost, 0});
  }
  bool spfa(int s, int t) {
    fill(all(dist), INF);
    fill(all(in_q), false);
    queue<int> q;
    dist[s] = 0; cap_pushed[s] = INF;
    q.push(s); in_q[s] = true;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      in_q[u] = false;
      for (int i = 0; i < sz(g[u]); ++i) {
        auto& e = g[u][i];
        if (e.cap - e.flow > 0 && dist[u] + e.cost < dist[e.to]) {
          dist[e.to] = dist[u] + e.cost;
          par_u[e.to] = u; par_e[e.to] = i;
          cap_pushed[e.to] = min(cap_pushed[u], e.cap - e.flow);
          if (!in_q[e.to]) { q.push(e.to); in_q[e.to] = true; }
        }
      }
    }
    return dist[t] != INF;
  }
  // Retorna {flujo_total, costo_minimo} enviando a lo sumo max_f
  pll solve(int s, int t, ll max_f = INF) {
    ll flow = 0, cost = 0;
    while (flow < max_f && spfa(s, t)) {
      ll f = min(max_f - flow, cap_pushed[t]);
      flow += f;
      cost += f * dist[t];
      for (int cur = t; cur != s; cur = par_u[cur]) {
        int p = par_u[cur], idx = par_e[cur];
        g[p][idx].flow += f;
        g[cur][g[p][idx].rev].flow -= f;
      }
    }
    return {flow, cost};
  }
};

// 3. Algoritmo Hungaro (Asignacion N x M de Costo Minimo en O(N^2 * M))
// Matriz 'a' 1-indexed de tamano (n+1) x (m+1) con n <= m.
// Para MAXIMIZAR costo: negar todos los valores de la matriz antes de
llamar.
pair<ll, vi> hungarian(int n, int m, const vector<vll>& a) {
  vll u(n + 1, 0), v(m + 1, 0);
  vi p(m + 1, 0), way(m + 1, 0);
  for (int i = 1; i <= n; ++i) {
    p[0] = i;
    int j0 = 0;
    vll minv(m + 1, INF);
    vector<bool> used(m + 1, false);
    do {
      used[j0] = true;
      int i0 = p[j0], j1 = 0;
      ll delta = INF;
      for (int j = 1; j <= m; ++j) {
        if (!used[j]) {
          ll cur = a[i0][j] - u[i0] - v[j];
          if (cur < minv[j]) { minv[j] = cur; way[j] = j0; }
          if (minv[j] < delta) { delta = minv[j]; j1 = j; }
        }
      }
      for (int j = 0; j <= m; ++j) {
        if (used[j]) { u[p[j]] += delta; v[j] -= delta; }
        else minv[j] -= delta;
      }
      j0 = j1;
    } while (p[j0] != 0);
    do {
      int j1 = way[j0];
      p[j0] = p[j1];
      j0 = j1;
    } while (j0);
  }
  vi match_row(n + 1, 0);
  for (int j = 1; j <= m; ++j) if (p[j] > 0) match_row[p[j]] = j;
  return {-v[0], match_row}; // {costo_minimo, columna_asignada_a_cada_fila}
}

// 4. Blossom (Max Matching en Grafo General No Bipartito en O(V^3))
struct Blossom {
  int n, timer = 0;
  vector<vi> adj;
  vi match, p, base, vis, in_q, in_b;
  queue<int> q;

  Blossom(int n)
    : n(n), adj(n + 1), match(n + 1, 0), p(n + 1, 0),
      base(n + 1, 0), vis(n + 1, 0), in_q(n + 1, 0), in_b(n + 1, 0) {}

  void add_edge(int u, int v) { adj[u].pb(v); adj[v].pb(u); }

  int lca(int u, int v) {
    ++timer;
    while (true) {
      if (u) {
        u = base[u];
        if (vis[u] == timer) return u;
        vis[u] = timer;
        u = p[match[u]];
      }
      swap(u, v);
    }
  }
  void mark_blossom(int u, int v, int b) {
    while (base[u] != b) {
      in_b[base[u]] = in_b[base[match[u]]] = 1;
      p[u] = v;
      v = match[u];
      u = p[v];
    }
  }
  int bfs(int s) {
    fill(all(p), 0); fill(all(in_q), 0);
    for (int i = 1; i <= n; ++i) base[i] = i;
    while (!q.empty()) q.pop();
    q.push(s); in_q[s] = 1;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int v : adj[u]) {
        if (base[u] == base[v] || match[u] == v) continue;
        if (v == s || (match[v] && p[match[v]])) {
          int b = lca(u, v);
          fill(all(in_b), 0);
          mark_blossom(u, v, b);
          mark_blossom(v, u, b);
          for (int i = 1; i <= n; ++i)
            if (in_b[base[i]]) {
              base[i] = b;
              if (!in_q[i]) { q.push(i); in_q[i] = 1; }
            }
        } else if (!p[v]) {
          p[v] = u;
          if (!match[v]) return v;
          v = match[v];
          q.push(v); in_q[v] = 1;
        }
      }
    }
    return 0;
  }
  int solve() {
    int ans = 0;
    for (int i = 1; i <= n; ++i) {
      if (!match[i]) {
        int cur = bfs(i);
        if (cur) {
          ans++;
          while (cur) {
            int nxt = match[p[cur]];
            match[cur] = p[cur];
            match[p[cur]] = cur;
            cur = nxt;
          }
        }
      }
    }
    return ans; // Cantidad de parejas; match[u] indica pareja de u
  }
};
```

Ejemplo de Input y Uso (Dinic Max Flow & Min-Cost Max-Flow):

```
Input Dinic: 4 nodos (Fuente S = 1, Sumidero T = 4).
Aristas (u v cap): (1 2 40), (1 4 20), (2 4 20), (2 3 30), (3 4 10).
Flujo maximo de 1 a 4 = 20 (directo 1->4) + 20 (1->2->4) + 10 (1->2->3->4) =
50.
```

```
int main() {
  Dinic mf(4, 1, 4);
  mf.add_edge(1, 2, 40);
  mf.add_edge(1, 4, 20);
  mf.add_edge(2, 4, 20);
  mf.add_edge(2, 3, 30);
  mf.add_edge(3, 4, 10);
  cout << mf.max_flow() << "\n"; // 50
}
```

6.7 Árbol Generador Mínimo Dirigido (Directed MST / Arborescence)

- **¿Cuándo usar?**: En un grafo dirigido con pesos (positivos o negativos), hallar el árbol generador con menor peso total enraizado en un nodo R, tal que todo nodo sea alcanzable desde R siguiendo aristas dirigidas (Algoritmo de Chu-Liu-Edmonds).
- **Complejidad**: $\mathcal{O}(E \log V)$.

```
// Edmonds' Chu-Liu-Edmonds en O(E log V) con Skew Heaps
struct DirEdge { int u, v; ll w; };
struct SkewNode {
  DirEdge key;
```

```
  SkewNode *l = 0, *r = 0;
  ll delta = 0;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  DirEdge top() { prop(); return key; }
};
SkewNode* skew_merge(SkewNode *a, SkewNode *b) {
  if (!a || !b) return a ? a : b;
  a->prop(); b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = skew_merge(b, a->r)));
  return a;
}
void skew_pop(SkewNode*& a) {
  a->prop(); a = skew_merge(a->l, a->r);
}

// Retorna {costo_total, parent_edge} (-1 si no es conexo)
pair<ll, vi> directed_mst(int n, int root, vector<DirEdge>& edges) {
  struct RollbackDSU {
    vi p, r;
    vector<pair<int, int>> hist;
    RollbackDSU(int n) : p(n), r(n, 0) { iota(all(p), 0); }
    int find(int i) { return p[i] == i ? i : find(p[i]); }
    bool unite(int i, int j) {
      i = find(i); j = find(j);
      if (i == j) return false;
      if (r[i] < r[j]) swap(i, j);
      hist.push_back({j, r[i] == r[j]});
      p[j] = i;
      if (r[i] == r[j]) r[i]++;
      return true;
    }
    int time() { return sz(hist); }
    void rollback(int t) {
      while (sz(hist) > t) {
        auto [j, inc] = hist.back(); hist.pop_back();
        if (inc) r[p[j]]--;
        p[j] = j;
      }
    }
  } uf(n);

  vector<SkewNode*> heap(n, 0);
  for (auto& e : edges) heap[e.v] = skew_merge(heap[e.v], new SkewNode{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n, -1);
  seen[root] = root;
  vector<DirEdge> Q(n), in(n, {-1, -1, 0});
  deque<tuple<int, int, vector<DirEdge>>> cycs;

  for (int s = 0; s < n; ++s) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1, {}}; // Inalcanzable
      DirEdge e = heap[u]->top();
      heap[u]->delta -= e.w; skew_pop(heap[u]);
      Q[qi] = e; path[qi++] = u; seen[u] = s;
      res += e.w; u = uf.find(e.u);
      if (seen[u] == s) { // Ciclo detectado: contraer
        SkewNode* cyc = 0;
        int end = qi, t = uf.time();
        do cyc = skew_merge(cyc, heap[w = path[--qi]]);
        while (uf.unite(u, w));
        u = uf.find(u); heap[u] = cyc; seen[u] = -1;
        cycs.push_front({u, t, {&Q[qi], &Q[end]}});
      }
    }
    for (int i = 0; i < qi; ++i) in[uf.find(Q[i].v)] = Q[i];
  }
  for (auto& [u, t, comp] : cycs) { // Reconstruccion
    uf.rollback(t);
    DirEdge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.v)] = e;
    in[uf.find(inEdge.v)] = inEdge;
  }
  for (int i = 0; i < n; ++i) par[i] = in[i].u;
  return {res, par};
}
```

6.8 Componentes Biconexas y Block-Cut Tree

- **¿Cuándo usar?**: En un grafo no dirigido, descomponer el grafo en bloques (componentes 2-vértice conexas, donde ningún vértice individual puede desconectar el bloque) y puntos de articulación. Construir el **Block-Cut Tree**, un árbol bipartito alternando entre puntos de articulación y componentes biconexas, sobre el cual se responden consultas de caminos seguros y puentes en $\mathcal{O}(\log V)$.
- **Complejidad**: $\mathcal{O}(V + E)$.

```
struct BlockCutTree {
  int n;
  vector<vi> adj, bct_adj;
  vi dfn, low, is_cut, bct_id;
  vector<pii> st;
  int timer = 0, bct_nodes = 0;
  vector<vector<pii>> biconn_edges;

  BlockCutTree(int n) : n(n), adj(n + 1), dfn(n + 1, 0),
    low(n + 1, 0), is_cut(n + 1, 0), bct_id(n + 1, 0) {}

  void add_edge(int u, int v) {
    adj[u].push_back(v); adj[v].push_back(u);
  }
  void dfs(int u, int p = 0) {
    dfn[u] = low[u] = ++timer;
    int children = 0;
    for (int v : adj[u]) {
      if (v == p) continue;
      if (dfn[v]) low[u] = min(low[u], dfn[v]);
      else {
        children++;
        st.push_back({u, v});
        dfs(v, u);
        low[u] = min(low[u], low[v]);
        if (low[v] >= dfn[u]) {
          is_cut[u] = (p != 0 || children > 1);
          biconn_edges.push_back({});
          while (true) {
            auto edge = st.back(); st.pop_back();
            biconn_edges.back().push_back(edge);
            if (edge == pii{u, v}) break;
          }
        }
      }
    }
  }
  void build() {
    for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i);
    int cuts = 0;
    for (int i = 1; i <= n; ++i) if (is_cut[i]) bct_id[i] = ++cuts;
    bct_nodes = cuts;
    bct_adj.resize(cuts + sz(biconn_edges) + 1);

    for (int b = 0; b < sz(biconn_edges); ++b) {
      int block_node = ++bct_nodes;
      set<int> verts;
      for (auto [u, v] : biconn_edges[b]) {
        verts.insert(u); verts.insert(v);
      }
      for (int u : verts) {
        if (is_cut[u]) {
          bct_adj[bct_id[u]].push_back(block_node);
          bct_adj[block_node].push_back(bct_id[u]);
        } else bct_id[u] = block_node;
      }
    }
  }
};
```

6.9 Camino y Ciclo Euleriano (Hierholzer)

- **¿Cuándo usar?**: Encontrar un camino o ciclo que visite **cada arista del grafo exactamente una vez**.
  - **Grafo Dirigido**: Existe ciclo euleriano si para todo nodo $\text{in}(u) = \text{out}(u)$. Existe camino si a lo sumo un nodo tiene $\text{out} - \text{in} = 1$ (inicio) y uno $\text{in} - \text{out} = 1$ (fin).
  - **Grafo No Dirigido**: Existe ciclo si todos los grados son pares; existe camino si exactamente 0 o 2 nodos tienen grado impar.
- **Complejidad**: $\mathcal{O}(V + E)$.

```
// Hierholzer para Grafo Dirigido con multiaristas (1-indexed)
struct EulerianDirected {
  int n, m = 0;
  vector<vector<pii>> adj; // {v, edge_id}
  vi in_deg, out_deg, cur_edge;
  vector<bool> used_edge;

  EulerianDirected(int n) : n(n), adj(n + 1), in_deg(n + 1, 0),
    out_deg(n + 1, 0), cur_edge(n + 1, 0) {}

  void add_edge(int u, int v) {
    adj[u].push_back({v, ++m});
    out_deg[u]++; in_deg[v]++;
  }
  vi find_tour() {
    used_edge.assign(m + 1, false);
    int start = 1, one_out = 0, one_in = 0;
    for (int i = 1; i <= n; ++i) {
      if (out_deg[i] - in_deg[i] == 1) { one_out++; start = i; }
      else if (in_deg[i] - out_deg[i] == 1) one_in++;
      else if (in_deg[i] != out_deg[i]) return {}; // Imposible
    }
    if (!(one_out == 0 && one_in == 0) && !(one_out == 1 && one_in == 1))
      return {};

    for (int i = 1; i <= n; ++i)
      if (out_deg[i] > 0 && one_out == 0) { start = i; break; }

    vi path, st = {start};
    while (!st.empty()) {
      int u = st.back();
      while (cur_edge[u] < sz(adj[u]) && used_edge[adj[u]
[cur_edge[u]].second])
        cur_edge[u]++;
      if (cur_edge[u] < sz(adj[u])) {
        auto [v, id] = adj[u][cur_edge[u]++];
        used_edge[id] = true;
        st.push_back(v);
      } else {
        path.push_back(u);
        st.pop_back();
      }
    }
    reverse(all(path));
    if (sz(path) != m + 1) return {}; // Grafo desconectado
    return path;
  }
};
```

6.10 Flujos Avanzados y Matching: Push-Relabel, Algoritmo Húngaro y Blossom

- **Push-Relabel (Highest-Label Preflow-Push)**: Para redes de flujo densas ($V \leq 5000, E \leq 50000$) donde Dinic sufre por caminos aumentantes largos. $\mathcal{O}(V^2\sqrt{E})$.
- **Algoritmo Húngaro (Hungarian)**: Problema de asignación óptima: emparejar N trabajadores con N tareas con costo total mínimo/máximo en una matriz $N \times N$. $\mathcal{O}(N^3)$ (mucho más rápido y sencillo que modelar con MCMF).
- **Algoritmo de Blossom (Edmonds)**: Emparejamiento máximo en grafos generales **no bipartitos** (resuelve los ciclos impares contrayendo los «blossoms»). $\mathcal{O}(V^2E)$.

```
// 1. Algoritmo Hungaro (Min/Max Weight Bipartite Matching en O(N^3))
// Matriz de costo C de 0..n-1 x 0..n-1.
// Para MAXIMIZAR: pasar costos negativos (-C[i][j]).
template<typename T = ll>
struct Hungarian {
  const T INF = numeric_limits<T>::max() / 2;
  int n;
  vector<vector<T>> cost;
  Hungarian(int n) : n(n), cost(n + 1, vector<T>(n + 1, 0)) {}

  void set_cost(int i, int j, T c) { cost[i + 1][j + 1] = c; }

  // Retorna {costo_minimo, asignacion (0-indexed)}
  pair<T, vi> solve() {
    vector<T> u(n + 1, 0), v(n + 1, 0);
    vi p(n + 1, 0), way(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
      p[0] = i;
      int j0 = 0;
      vector<T> minv(n + 1, INF);
      vector<bool> used(n + 1, false);
      do {
        used[j0] = true;
        int i0 = p[j0], j1 = 0;
        T delta = INF;
        for (int j = 1; j <= n; ++j) {
          if (!used[j]) {
            T cur = cost[i0][j] - u[i0] - v[j];
            if (cur < minv[j]) { minv[j] = cur; way[j] = j0; }
            if (minv[j] < delta) { delta = minv[j]; j1 = j; }
          }
        }
        for (int j = 0; j <= n; ++j) {
          if (used[j]) { u[p[j]] += delta; v[j] -= delta; }
          else minv[j] -= delta;
        }
        j0 = j1;
      } while (p[j0] != 0);
      do {
        int j1 = way[j0];
        p[j0] = p[j1];
```

```
        j0 = j1;
      } while (j0);
    }
    vi match(n);
    for (int j = 1; j <= n; ++j) match[p[j] - 1] = j - 1;
    return {-v[0], match};
  }
};

// 2. Blossom Algorithm (Edmonds Maximum Matching en Grafos Generales
O(V^3))
struct Blossom {
  int n;
  vector<vi> adj;
  vi match, p, base, q;
  vector<bool> in_b, in_q;

  Blossom(int n) : n(n), adj(n + 1), match(n + 1, 0),
    p(n + 1), base(n + 1), in_b(n + 1), in_q(n + 1) {}

  void add_edge(int u, int v) {
    adj[u].push_back(v); adj[v].push_back(u);
  }
  int lca(int root, int u, int v) {
    vector<bool> used(n + 1, false);
    while (true) {
      u = base[u]; used[u] = true;
      if (u == root) break;
      u = p[match[u]];
    }
    while (true) {
      v = base[v];
      if (used[v]) return v;
      v = p[match[v]];
    }
  }
  void mark_blossom(int l, int u) {
    while (base[u] != l) {
      int v = match[u];
      in_b[base[u]] = in_b[base[v]] = true;
      u = p[v];
      if (base[u] != l) p[u] = v;
    }
  }
  void contract_blossom(int s, int u, int v) {
    int l = lca(s, u, v);
    fill(all(in_b), false);
    mark_blossom(l, u); mark_blossom(l, v);
    if (base[u] != l) p[u] = v;
    if (base[v] != l) p[v] = u;
    for (int i = 1; i <= n; ++i) {
      if (in_b[base[i]]) {
        base[i] = l;
        if (!in_q[i]) { in_q[i] = true; q.push_back(i); }
      }
    }
  }
  int bfs(int s) {
    fill(all(in_q), false); fill(all(p), 0);
    iota(all(base), 0);
    q.clear(); q.push_back(s); in_q[s] = true;
    int head = 0;
    while (head < sz(q)) {
      int u = q[head++];
      for (int v : adj[u]) {
        if (base[u] == base[v] || match[u] == v) continue;
        if (v == s || (match[v] && p[match[v]]))
          contract_blossom(s, u, v);
        else if (!p[v]) {
          p[v] = u;
          if (!match[v]) return v;
          if (!in_q[match[v]]) {
            in_q[match[v]] = true; q.push_back(match[v]);
          }
        }
      }
    }
    return 0;
  }
  int max_matching() {
    int ans = 0;
    for (int i = 1; i <= n; ++i) {
      if (!match[i]) {
        int v = bfs(i);
        while (v) {
          int pv = p[v], ppv = match[pv];
          match[v] = pv; match[pv] = v;
          v = ppv;
        }
      }
    }
    for (int i = 1; i <= n; ++i) if (match[i]) ans++;
    return ans / 2;
  }
};

// 3. Push-Relabel (Highest-Label Preflow-Push O(V^2 sqrt(E)))
struct PushRelabel {
  struct Edge { int to, rev; ll cap, flow; };
  int n;
  vector<vector<Edge>> adj;
  vector<ll> excess;
  vi height, count;
  vector<bool> active;
  vector<vi> B;
  int b_max = 0;

  PushRelabel(int n) : n(n), adj(n + 1), excess(n + 1, 0),
    height(n + 1, 0), count(2 * n + 1, 0), active(n + 1, false), B(2 * n +
1) {}

  void add_edge(int from, int to, ll cap) {
    adj[from].push_back({to, sz(adj[to]), cap, 0});
    adj[to].push_back({from, sz(adj[from]) - 1, 0, 0});
  }
  void enqueue(int v) {
    if (!active[v] && excess[v] > 0 && height[v] < n) {
      active[v] = true;
      B[height[v]].push_back(v);
      b_max = max(b_max, height[v]);
    }
  }
  void push(int u, Edge& e) {
    ll push_amt = min(excess[u], e.cap - e.flow);
    if (push_amt == 0 || height[u] <= height[e.to]) return;
    e.flow += push_amt;
    adj[e.to][e.rev].flow -= push_amt;
    excess[u] -= push_amt;
    excess[e.to] += push_amt;
    enqueue(e.to);
  }
  void gap(int k) {
    for (int v = 1; v <= n; ++v) {
      if (height[v] >= k) {
        count[height[v]]--;
        height[v] = max(height[v], n + 1);
        count[height[v]]++;
        enqueue(v);
      }
    }
  }
  void relabel(int v) {
    count[height[v]]--;
    int d = 2 * n;
    for (auto& e : adj[v])
      if (e.cap - e.flow > 0) d = min(d, height[e.to]);
    if (count[height[v]] == 0) gap(height[v]);
    height[v] = d + 1;
    count[height[v]]++;
    enqueue(v);
  }
  ll max_flow(int s, int t) {
    count[0] = n - 1; count[n] = 1;
    height[s] = n; active[s] = active[t] = true;
    for (auto& e : adj[s]) {
      excess[s] += e.cap;
      push(s, e);
    }
    while (b_max >= 0) {
      if (B[b_max].empty()) { b_max--; continue; }
      int v = B[b_max].back(); B[b_max].pop_back();
      active[v] = false;
      for (auto& e : adj[v]) {
        if (excess[v] == 0) break;
        push(v, e);
      }
      if (excess[v] > 0) relabel(v);
    }
    return excess[t];
  }
};
```

Bloque 7: Matemáticas, Teoría de Números, Álgebra y Juegos

7.1 Cribas (Eratóstenes, Divisores, Lineal SPF, Segmentada) y Primalidad $\mathcal{O}(\sqrt{N})$

- **¿Cuándo usar?**:
  - **Criba de Eratóstenes (`sieve`)**: Precalcular qué números hasta 10^7 son primos en $\mathcal{O}(N \log \log N)$.
  - **Criba de Divisores (`harmonic_sieve`)**: Precalcular en $\mathcal{O}(N \log N)$ la cantidad $\tau(x)$ y suma $\sigma(x)$ de divisores para todos los números hasta 10^6.
  - **Criba Lineal SPF (`linear_sieve_spf`)**: Precalcular en $\mathcal{O}(N)$ el Menor Factor Primo (`spf[x]`) hasta 10^7 para **factorizar cualquier número x de una consulta en tiempo $\mathcal{O}(\log x)$**.
  - **Criba Segmentada (`segmented_sieve`)**: Hallar todos los primos en un rango $[L, R]$ donde $R \leq 10^{12}$ pero la longitud del intervalo es pequeña ($R - L + 1 \leq 10^7$).
  - **Primalidad Individual (`is_prime_single`)**: Verificar si un único $N \leq 10^{12}$ es primo en $\mathcal{O}(\sqrt{N})$.
- **Complejidad**: Ver encabezados de cada función.

```
// 1. Criba de Eratostenes en O(MAX_P log log MAX_P)
const int MAX_P = 1000005;
bool is_prime[MAX_P];
void sieve() {
  is_prime[0] = is_prime[1] = false;
  fill(is_prime + 2, is_prime + MAX_P, true);
  for (ll i = 2; i * i < MAX_P; ++i)
    if (is_prime[i])
      for (ll j = i * i; j < MAX_P; j += i) is_prime[j] = false;
}

// 2. Criba Armonica de Divisores en O(N log N)
int num_div[MAX_P];
ll sum_div[MAX_P];
void harmonic_sieve(int n) {
  for (int i = 1; i <= n; ++i)
    for (int j = i; j <= n; j += i) {
      num_div[j]++;
      sum_div[j] += i;
    }
}

// 3. Criba Lineal O(N) con Menor Factor Primo (SPF) y Factorizacion O(log
X)
const int MAX_SPF = 10000005;
int spf[MAX_SPF];
vi primes_list;
void linear_sieve_spf(int n = MAX_SPF - 1) {
  for (int i = 2; i <= n; ++i) {
    if (spf[i] == 0) { spf[i] = i; primes_list.pb(i); }
    for (int p : primes_list) {
      if (p > spf[i] || 1LL * i * p > n) break;
      spf[i * p] = p;
    }
  }
}
vi get_prime_factors(int x) { // Retorna factores primos con multiplicidad
  vi f;
  while (x > 1) { f.pb(spf[x]); x /= spf[x]; }
  return f;
}

// 4. Criba Segmentada en [L, R] para R <= 10^12 y (R - L + 1) <= 10^7
// Retorna vector<bool> donde res[x - L] == true si y solo si x es primo.
vector<bool> segmented_sieve(ll L, ll R) {
  ll lim = sqrt(R);
  vector<bool> mark(lim + 1, false);
  vll pr;
  for (ll i = 2; i <= lim; ++i) {
    if (!mark[i]) {
      pr.pb(i);
      for (ll j = i * i; j <= lim; j += i) mark[j] = true;
    }
  }
  vector<bool> res(R - L + 1, true);
  for (ll p : pr) {
    ll start = max(p * p, (L + p - 1) / p * p);
    for (ll j = start; j <= R; j += p) res[j - L] = false;
  }
  if (L == 1) res[0] = false;
  return res;
}

// 5. Primalidad Individual en O(sqrt(N)) para N <= 10^12
bool is_prime_single(ll n) {
  if (n <= 1) return false;
  if (n <= 3) return true;
  if (n % 2 == 0 || n % 3 == 0) return false;
  for (ll i = 5; i * i <= n; i += 6)
    if (n % i == 0 || n % (i + 2) == 0) return false;
  return true;
}
```

7.2 Primalidad y Factorización Grande: Miller-Rabin (64-bit) y Pollard's Rho

- **¿Cuándo usar?**: Cuando N es enorme ($10^{12} \le N \le 10^{18}$) y la prueba $\mathcal{O}(\sqrt{N})$ da Time Limit Exceeded:
  - **Miller-Rabin**: Verifica de forma **100% determinista** si cualquier `ull` de 64 bits ($N < 2^{64}$) es primo en $\mathcal{O}(\log N)$ usando las 7 bases canónicas.
  - **Pollard's Rho**: Factoriza por completo cualquier entero de 64 bits ($N \le 10^{18}$) en tiempo esperado $\mathcal{O}(N^{1/4} \log N)$.
- **Complejidad**: Miller-Rabin $\mathcal{O}(7 \cdot \log N)$; Pollard's Rho $\mathcal{O}(N^{1/4})$.

```
ull mul_mod64(ull a, ull b, ull m) {
  return (ull)((i128)a * b % m);
}
ull pow_mod64(ull b, ull e, ull m) {
  ull r = 1;
  for (b %= m; e > 0; e >>= 1) {
    if (e & 1) r = mul_mod64(r, b, m);
    b = mul_mod64(b, b, m);
  }
  return r;
}

// 1. Miller-Rabin Determinista para todo entero de 64 bits (< 2^64)
bool miller_rabin(ull n) {
  if (n < 2) return false;
  for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
    if (n % p == 0) return n == p;
  }
  ull d = n - 1, s = 0;
  while ((d & 1) == 0) { d >>= 1; s++; }
  for (ull a : {2ULL, 325ULL, 9375ULL, 28178ULL,
                450775ULL, 9780504ULL, 1795265022ULL}) {
    if (a % n == 0) continue;
    ull x = pow_mod64(a, d, n);
    if (x == 1 || x == n - 1) continue;
    bool composite = true;
    for (ull r = 1; r < s; ++r) {
      x = mul_mod64(x, x, n);
      if (x == n - 1) { composite = false; break; }
    }
    if (composite) return false;
  }
  return true;
}

// 2. Pollard's Rho (Factorizacion completa en O(N^(1/4)))
ull pollard_f(ull x, ull c, ull m) {
  return (mul_mod64(x, x, m) + c) % m;
}
ull pollard_rho(ull n) {
  if ((n & 1) == 0) return 2;
  ull x = 2, y = 2, d = 1, c = rd(1, n - 1);
  while (d == 1) {
    x = pollard_f(x, c, n);
    y = pollard_f(pollard_f(y, c, n), c, n);
    d = std::gcd(x > y ? x - y : y - x, n);
  }
  return (d == n) ? pollard_rho(n) : d;
}
void factorize_64(ull n, map<ull, int>& factors) {
  if (n == 1) return;
  if (miller_rabin(n)) { factors[n]++; return; }
  ull div = pollard_rho(n);
  factorize_64(div, factors);
  factorize_64(n / div, factors);
}
```

Ejemplo de Input y Uso (Pollard's Rho):

```
int main() {
  ull n = 1000000007ULL * 1000000009ULL; // ~ 10^18 (semiprimo)
  map<ull, int> pf;
  factorize_64(n, pf);
  // pf contiene {1000000007: 1, 1000000009: 1}
}
```

7.3 GCD, Diferencia de Cuadrados, Euclides Extendido, Ecuaciones Diofánticas, Ecuación Modular y CRT

- **¿Cuándo usar?**:
  - **GCD tras sumar X (`array_diff_gcd`)**: Cuando sumas un mismo X a todo un arreglo A, dado que $\gcd(a_1+x, a_2+x, \ldots) = \gcd(a_1+x, |a_2-a_1|, |a_3-a_2|, \ldots)$, precalculas el GCD de diferencias una vez y respondes cada X en $\mathcal{O}(\log(\text{val}))$.
  - **Diferencia de Cuadrados**: Expresar $N = a^2 - b^2 = (a-b)(a+b)$ probando divisores $d_1 \cdot d_2 = N$ de igual paridad.
  - **Euclides Extendido y Diofánticas**: Resolver $a \cdot x + b \cdot y = c$ en enteros. Tiene solución si y solo si $\gcd(a,b) \mid c$. Todas las soluciones tienen la forma $(x_0 + k\frac{b}{g}, y_0 - k\frac{a}{g})$.
  - **Teorema Chino del Resto (CRT General)**: Resolver un sistema de congruencias $x \equiv r_i \pmod{m_i}$ incluso cuando los módulos m_i **no son coprimos entre sí**.
- **Complejidad**: $\mathcal{O}(\log(\min(a,b)))$ por operación de GCD/ExtGCD.

```
// 1. GCD de todo el arreglo A despues de sumar X a cada elemento
ll array_diff_gcd(const vll& a, ll x) {
  int n = sz(a);
  if (n == 1) return abs(a[0] + x);
  ll diff_g = 0;
  for (int i = 1; i < n; ++i) diff_g = std::gcd(diff_g, abs(a[i] - a[i -
1]));
  return std::gcd(abs(a[0] + x), diff_g);
}

// 2. Expresar N = a^2 - b^2 (retorna pares {a, b} >= 0)
vector<pll> solve_diff_of_squares(ll n) {
  vector<pll> sols;
  for (ll d1 = 1; d1 * d1 <= n; ++d1) {
    if (n % d1 == 0) {
      ll d2 = n / d1;
      if ((d1 + d2) % 2 == 0) sols.pb({(d1 + d2) / 2, (d2 - d1) / 2});
    }
  }
  return sols;
}

// 3. Euclides Extendido: halla g = gcd(a, b) y (x, y) tal que a*x + b*y = g
ll extgcd(ll a, ll b, ll& x, ll& y) {
  if (b == 0) { x = 1; y = 0; return a; }
  ll x1, y1;
  ll g = extgcd(b, a % b, x1, y1);
  x = y1;
  y = x1 - y1 * (a / b);
  return g;
}

// 4. Ecuacion Diofantica Lineal: a*x + b*y = c
// Retorna true si existe solucion; deja una solucion particular en (x0,
y0).
// Familia general de soluciones: x = x0 + k*(b/g), y = y0 - k*(a/g).
bool find_diophantine(ll a, ll b, ll c, ll& x0, ll& y0, ll& g) {
  g = extgcd(abs(a), abs(b), x0, y0);
  if (c % g != 0) return false;
  x0 *= c / g;
  y0 *= c / g;
  if (a < 0) x0 = -x0;
  if (b < 0) y0 = -y0;
  return true;
}

// 5. Teorema Chino del Resto General (CRT para modulos NO necesariamente
coprimos)
// Recibe vectores 'r' y 'm' (x == r[i] mod m[i]).
// Retorna {x_minimo_no_negativo, lcm_total}, o {-1, -1} si es incompatible.
pll crt_general(const vll& r, const vll& m) {
  ll r0 = 0, m0 = 1;
  for (int i = 0; i < sz(r); ++i) {
    ll r1 = ((r[i] % m[i]) + m[i]) % m[i], m1 = m[i];
    ll x, y;
    ll g = extgcd(m0, m1, x, y);
    if ((r1 - r0) % g != 0) return {-1, -1}; // Sin solucion
    ll lcm = (m0 / g) * m1;
    ll step = (r1 - r0) / g % (m1 / g) * (x % (m1 / g)) % (m1 / g);
    r0 = (r0 + (i128)step * m0) % lcm;
    if (r0 < 0) r0 += lcm;
    m0 = lcm;
  }
  return {r0, m0};
}
```

Ejemplo de Uso (CRT General):

```
int main() {
  // Resolver: x = 2 (mod 3), x = 3 (mod 5), x = 2 (mod 7)
  auto [x, mod_lcm] = crt_general({2, 3, 2}, {3, 5, 7});
  // x = 23, mod_lcm = 105 (porque 23 % 3 = 2, 23 % 5 = 3, 23 % 7 = 2)
}
```

7.4 Aritmética Modular (`mint`), Combinatoria (nCr), Teorema de Lucas y Fórmula de Legendre ($n!$)

- **¿Cuándo usar?**:
  - **`normalize` / `add` / `sub` / `mul` / `binpow` / `inv_fermat` / `inv_extgcd`**: Operaciones modulares seguras y rápidas.
  - **`init_fact` + `nCr` / `nPr`**: Precalcular factoriales e inversos hasta $N = 10^6$ en $\mathcal{O}(N)$ para responder cualquier $\binom{n}{r} \bmod p$ en $\mathcal{O}(1)$.
  - **Teorema de Lucas**: Cuando n, r son gigantescos ($n \le 10^{18}$) pero el módulo primo p es pequeño ($p \le 10^6$): calcula $\binom{n}{r} \bmod p$ en $\mathcal{O}(\log_p n)$.
  - **Fórmula de Legendre**: Hallar la mayor potencia k^x que divide a $n!$ en $\mathcal{O}(\log_p n)$.
- **Complejidad**: Preprocesamiento $\mathcal{O}(N)$, `nCr` en $\mathcal{O}(1)$, Lucas y Legendre en $\mathcal{O}(\log_p N)$.

```
constexpr ll MOD = 1000000007LL;

inline ll norm_mod(ll x, ll m = MOD) {
  x %= m;
  return (x < 0) ? (x + m) : x;
}
inline ll add_mod(ll a, ll b, ll m = MOD) {
  ll res = a % m + b % m;
  return (res >= m) ? (res - m) : (res < 0 ? res + m : res);
}
inline ll sub_mod(ll a, ll b, ll m = MOD) {
  ll res = a % m - b % m;
  return (res < 0) ? (res + m) : res;
}
inline ll mul_mod(ll a, ll b, ll m = MOD) {
  return (ll)((i128)norm_mod(a, m) * norm_mod(b, m) % m);
}
ll binpow(ll base, ll exp, ll m = MOD) {
  if (m == 1) return 0;
  base = norm_mod(base, m);
  ll res = 1;
  while (exp > 0) {
    if (exp & 1) res = mul_mod(res, base, m);
    base = mul_mod(base, base, m);
    exp >>= 1;
  }
  return res;
}
// Inverso cuando m es PRIMO (Fermat):
inline ll inv_fermat(ll a, ll m = MOD) { return binpow(a, m - 2, m); }

// Inverso cuando m es COMPUESTO (requiere gcd(a, m) == 1):
ll inv_extgcd(ll a, ll m) {
  ll x, y;
  ll g = extgcd(a, m, x, y);
  return (g == 1 || g == -1) ? norm_mod(x, m) : -1;
}

// Combinatoria O(N) precalculo, O(1) consulta:
const int MAX_F = 1000005;
ll fact[MAX_F], inv_fact[MAX_F], inv_num[MAX_F];
void init_fact(int n, ll m = MOD) {
  fact[0] = inv_fact[0] = 1;
  inv_num[0] = 0; inv_num[1] = 1;
  for (int i = 1; i <= n; ++i) fact[i] = (fact[i - 1] * i) % m;
  inv_fact[n] = inv_fermat(fact[n], m);
  for (int i = n - 1; i >= 1; --i)
    inv_fact[i] = (inv_fact[i + 1] * (i + 1)) % m;
  for (int i = 2; i <= n; ++i)
    inv_num[i] = (m - (m / i) * inv_num[m % i] % m) % m;
}
inline ll nCr(int n, int r, ll m = MOD) {
  if (r < 0 || r > n || n < 0) return 0;
  return fact[n] * inv_fact[r] % m * inv_fact[n - r] % m;
}
inline ll nPr(int n, int r, ll m = MOD) {
  if (r < 0 || r > n || n < 0) return 0;
  return fact[n] * inv_fact[n - r] % m;
}

// Teorema de Lucas: nCr(n, r) mod p para n, r <= 10^18 y p primo <= MAX_F
// (Requiere haber llamado init_fact(p - 1, p))
ll lucas_nCr(ll n, ll r, ll p) {
  if (r < 0 || r > n) return 0;
  if (r == 0) return 1;
  return lucas_nCr(n / p, r / p, p) * nCr(n % p, r % p, p) % p;
}

// Formula de Legendre: exponente del primo 'p' en la factorizacion de (n!)
ll legendre_prime_in_fact(ll n, ll p) {
  ll cnt = 0;
  while (n > 0) { cnt += n / p; n /= p; }
```

```
  return cnt;
}
```

7.5 Funciones Multiplicativas: Totient de Euler (ϕ) y Función de Möbius (μ)

- **¿Cuándo usar?**:
  - **Totient de Euler** $\phi(n)$: Cuenta cuántos enteros en $[1, n]$ son coprimos con n. Sirve también para reducir exponentes gigantes por el Teorema de Euler: $a^b \equiv a^{b \bmod \phi(m)+\phi(m)} \pmod m$ (para $b \geq \phi(m)$).
  - **Función de Möbius** $\mu(n)$: Clave para **Inclusión-Exclusión sobre divisores / GCD** (ej. contar cuántos pares o subconjuntos de un arreglo tienen $\gcd = 1$ o $\gcd = k$ en $\mathcal{O}(M \log M)$ usando la identidad $\sum_{d|g} \mu(d) = [g == 1]$).
- **Complejidad**: $\phi(n)$ individual en $\mathcal{O}(\sqrt{n})$; Criba de ϕ y μ hasta N en $\mathcal{O}(N \log \log N)$.

```
// 1. Totient de Euler individual en O(sqrt(N))
ll euler_phi(ll n) {
  ll ans = n;
  for (ll d = 2; d * d <= n; ++d) {
    if (n % d == 0) {
      while (n % d == 0) n /= d;
      ans -= ans / d;
    }
  }
  if (n > 1) ans -= ans / n;
  return ans;
}

// 2. Precalculo de phi[1..n] y mu[1..n] en O(N log log N)
int phi_arr[MAX_P], mu_arr[MAX_P];
void init_phi_and_mobius(int n) {
  for (int i = 1; i <= n; ++i) { phi_arr[i] = i; mu_arr[i] = 0; }
  for (int i = 2; i <= n; ++i) {
    if (phi_arr[i] == i) { // i es primo
      for (int j = i; j <= n; j += i) phi_arr[j] -= phi_arr[j] / i;
    }
  }
  mu_arr[1] = 1;
  for (int i = 1; i <= n; ++i)
    for (int j = 2 * i; j <= n; j += i)
      mu_arr[j] -= mu_arr[i];
}
```

7.6 Álgebra Lineal y Polinomios: Exponenciación de Matrices, Eliminación Gaussiana (`double` , Modular, XOR Basis), FFT, FWT, Lagrange, Logaritmo Discreto y Fracciones

- **¿Cuándo usar?**:
  - **Exponenciación de Matrices**: Calcular el término N-ésimo ($N \leq 10^{18}$) de una recurrencia lineal ($f(n) = c_1 f(n-1) + ... + c_k f(n-k)$) o contar caminos de longitud exacta K en un grafo en $\mathcal{O}(D^3 \log K)$.
  - **Eliminación Gaussiana (`double` / Modular)**: Resolver sistemas de ecuaciones lineales $Ax = b$ en $\mathcal{O}(N^3)$ (por ejemplo, valores esperados en grafos con ciclos / cadenas de Markov).
  - **Base de XOR (Gauss Binario)**: Insertar números de 60 bits en $\mathcal{O}(B)$ para saber el **máximo XOR posible de un subconjunto**, si un valor X se puede formar como XOR de algún subconjunto, o el K-ésimo menor XOR posible.
  - **FFT (Fast Fourier Transform)**: Multiplicar dos polinomios de grado N en $\mathcal{O}(N \log N)$ (equivalentemente: calcular para toda suma $k = i + j$ la convolución $\sum a[i] \cdot b[j]$).
  - **FWT / FHT (Fast Walsh-Hadamard Transform)**: Como FFT, pero en vez de sumar índices $(i + j)$, combina los índices con **XOR ($i \oplus j$), AND (i & j) u OR ($i \mid j$)** en $\mathcal{O}(N \log N)$.
  - **Logaritmo Discreto (Baby-Step Giant-Step)**: Hallar el menor $x \geq 0$ tal que $a^x \equiv b \pmod m$ en $\mathcal{O}(\sqrt{m})$.
- **Complejidad**: Ver encabezados de cada bloque.

```
// 1. Exponenciacion de Matrices en O(D^3 log E)
struct Matrix {
  int n;
  vector<vll> m;
  Matrix(int n, ll diag = 0) : n(n), m(n, vll(n, 0)) {
    for (int i = 0; i < n; ++i) m[i][i] = diag;
  }
  Matrix operator*(const Matrix& o) const {
    Matrix res(n, 0);
    for (int i = 0; i < n; ++i)
      for (int k = 0; k < n; ++k) {
        if (!m[i][k]) continue;
        for (int j = 0; j < n; ++j)
          res.m[i][j] = (res.m[i][j] + m[i][k] * o.m[k][j]) % MOD;
      }
    return res;
  }
};
Matrix mat_pow(Matrix b, ll e) {
  Matrix res(b.n, 1); // Identidad
  for (; e > 0; e >>= 1) {
    if (e & 1) res = res * b;
    b = b * b;
  }
  return res;
}

// Fibonacci en O(log N) usando duplicacion rapida: retorna {F_n, F_{n+1}}
pll fib_fast(ll n, ll m = MOD) {
  if (n == 0) return {0, 1};
  auto [a, b] = fib_fast(n >> 1, m);
  ll c = a * (2 * b - a + m) % m;
  ll d = (a * a % m + b * b % m) % m;
  return (n & 1) ? pll{d, (c + d) % m} : pll{c, d};
}

// 2. Base de XOR (Gauss Binario en O(B) por insercion)
struct XorBasis {
  static const int B = 60;
  ll basis[B + 1] = {0};
  int sz_b = 0;

  bool insert(ll x) {
    for (int i = B; i >= 0; --i) {
      if (!((x >> i) & 1LL)) continue;
      if (!basis[i]) { basis[i] = x; sz_b++; return true; }
      x ^= basis[i];
    }
    return false; // x ya era linealmente dependiente
  }
  // Maximo valor posible de (initial_val ^ subconjunto)
  ll max_xor(ll ans = 0) const {
    for (int i = B; i >= 0; --i)
      ans = max(ans, ans ^ basis[i]);
    return ans;
  }
  bool can_form(ll x) const {
    for (int i = B; i >= 0; --i)
      if ((x >> i) & 1LL) x ^= basis[i];
    return x == 0;
  }
};

// 3. Eliminacion Gaussiana en punto flotante O(N^3)
// Matriz aumentada 'a' de tamano N x (M + 1).
// Retorna: 0 (sin solucion), 1 (solucion unica en 'ans'), 2 (infinitas).
int gauss_double(vector<vector<double>> a, vector<double>& ans) {
  int n = sz(a), m = sz(a[0]) - 1;
  vi where(m, -1);
  for (int col = 0, row = 0; col < m && row < n; ++col) {
    int sel = row;
    for (int i = row; i < n; ++i)
      if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
    if (abs(a[sel][col]) < 1e-9) continue;
    swap(a[sel], a[row]);
    where[col] = row;
    for (int i = 0; i < n; ++i)
      if (i != row) {
        double c = a[i][col] / a[row][col];
        for (int j = col; j <= m; ++j) a[i][j] -= a[row][j] * c;
      }
    row++;
  }
  ans.assign(m, 0);
  for (int i = 0; i < m; ++i)
    if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
  for (int i = 0; i < n; ++i) {
    double sum = 0;
    for (int j = 0; j < m; ++j) sum += ans[j] * a[i][j];
    if (abs(sum - a[i][m]) > 1e-9) return 0;
  }
  for (int i = 0; i < m; ++i) if (where[i] == -1) return 2;
  return 1;
}

// 4. Fast Fourier Transform (FFT — Multiplicacion de Polinomios en O(N log N))
using cd = complex<double>;
void fft(vector<cd>& a, bool inv) {
  int n = sz(a);
  for (int i = 1, j = 0; i < n; ++i) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    double ang = 2 * acos(-1.0) / len * (inv ? -1 : 1);
    cd wlen(cos(ang), sin(ang));
    for (int i = 0; i < n; i += len) {
      cd w(1);
      for (int j = 0; j < len / 2; ++j) {
        cd u = a[i + j], v = a[i + j + len / 2] * w;
        a[i + j] = u + v;
        a[i + j + len / 2] = u - v;
        w *= wlen;
      }
    }
  }
  if (inv) for (cd& x : a) x /= n;
}
vll multiply_poly(const vll& a, const vll& b) {
  vector<cd> fa(all(a)), fb(all(b));
  int n = 1;
  while (n < sz(a) + sz(b)) n <<= 1;
  fa.resize(n); fb.resize(n);
  fft(fa, false); fft(fb, false);
  for (int i = 0; i < n; ++i) fa[i] *= fb[i];
  fft(fa, true);
  vll res(sz(a) + sz(b) - 1);
  for (int i = 0; i < sz(res); ++i) res[i] = llround(fa[i].real());
  return res;
}

// 5. Fast Walsh-Hadamard Transform (FWT para Convolucion XOR en O(N log N))
// n debe ser potencia de 2.
void fwt_xor(vll& a, bool inv) {
  int n = sz(a);
  for (int len = 1; 2 * len <= n; len <<= 1) {
    for (int i = 0; i < n; i += 2 * len) {
      for (int j = 0; j < len; ++j) {
        ll u = a[i + j], v = a[i + len + j];
        a[i + j] = u + v;
        a[i + len + j] = u - v;
        if (inv) { a[i + j] /= 2; a[i + len + j] /= 2; }
      }
    }
  }
}

// 6. Logaritmo Discreto (Baby-Step Giant-Step): menor x >= 0 con a^x = b (mod m)
ll discrete_log(ll a, ll b, ll m) {
  a %= m; b %= m;
  ll k = 1, add = 0, g;
  while ((g = std::gcd(a, m)) > 1) {
    if (b == k) return add;
    if (b % g != 0) return -1;
    b /= g; m /= g; ++add;
    k = (k * a / g) % m;
  }
  ll n = sqrt(m) + 1, an = 1;
  for (int i = 0; i < n; ++i) an = (an * a) % m;
  safe_umap<ll, ll> vals;
  for (ll q = 0, cur = b; q <= n; ++q) {
    vals[cur] = q;
    cur = (cur * a) % m;
  }
  for (ll p = 1, cur = k; p <= n; ++p) {
    cur = (cur * an) % m;
    if (vals.find(cur) != vals.end())
      return n * p - vals[cur] + add;
  }
  return -1;
}

// 7. Estructura de Fracciones Exactas (evita errores de precision de double)
struct Frac {
  ll num, den;
  Frac(ll n = 0, ll d = 1) : num(n), den(d) {
    if (den < 0) { num = -num; den = -den; }
    ll g = std::gcd(abs(num), den);
    if (g) { num /= g; den /= g; }
  }
  Frac operator+(const Frac& o) const { return {num*o.den + o.num*den, den*o.den}; }
```

```
  Frac operator-(const Frac& o) const { return {num*o.den - o.num*den,
den*o.den}; }
  Frac operator*(const Frac& o) const { return {num*o.num, den*o.den}; }
  Frac operator/(const Frac& o) const { return {num*o.den, den*o.num}; }
  bool operator<(const Frac& o) const { return (i128)num * o.den <
(i128)o.num * den; }
  bool operator==(const Frac& o) const { return num == o.num && den ==
o.den; }
};
```

Ejemplo de Input y Uso (Base de XOR y FFT):

```
int main() {
  XorBasis xb;
  for (ll x : {3, 10, 5}) xb.insert(x);
  cout << xb.max_xor() << "\n"; // 15 (10 ^ 5 = 15)

  // Multiplicar (1 + 2x) * (3 + 4x) = 3 + 10x + 8x^2
  vll prod = multiply_poly({1, 2}, {3, 4}); // {3, 10, 8}
}
```

7.7 XOR en Rango $\mathcal{O}(1)$, Función `mex`, Grundy y Juego de Nim / Misère Nim

- **¿Cuándo usar?**:
  - `xor_n` / `xor_range`: Calcular $1 \oplus 2 \oplus \ldots \oplus n$ o el XOR de todos los enteros consecutivos en $[L, R]$ en $\mathcal{O}(1)$ aprovechando que el XOR tiene periodo 4.
  - `mex` **y Teorema de Sprague-Grundy**: En cualquier juego imparcial para 2 jugadores donde gana quien hace el último movimiento, cada estado tiene un valor de Grundy $g(u) = \text{mex}(\{g(v) : u \to v\})$. Si el juego se compone de varios subjuegos independientes, el Grundy total es el **XOR de los Grundy individuales** ($g_1 \oplus g_2 \oplus \ldots \oplus g_k$). El primer jugador gana $\Leftrightarrow \text{XOR total} \neq 0$.
  - **Misère Nim**: Igual que Nim normal, pero **pierde** quien toma la última piedra. Si todas las pilas son de tamaño 1, gana el primero $\Leftrightarrow$ hay cantidad **par** de pilas; si al menos una pila es > 1, la condición ganadora es idéntica al Nim normal ($\bigoplus a_i \neq 0$).
- **Complejidad**: `xor_n` y `xor_range` en $\mathcal{O}(1)$; `mex` y `nim` en $\mathcal{O}(N)$.

```
// 1. XOR de 0 a n en O(1)
ll xor_n(ll n) {
  if (n < 0) return 0;
  if (n % 4 == 0) return n;
  if (n % 4 == 1) return 1;
  if (n % 4 == 2) return n + 1;
  return 0;
}
// XOR de todos los enteros en el intervalo cerrado [l, r] en O(1)
ll xor_range(ll l, ll r) {
  return (l > r) ? 0 : (xor_n(r) ^ xor_n(l - 1));
}

// 2. MEX (Minimum Excluded >= 0) en O(N)
int mex(const vi& v) {
  int n = sz(v);
  vector<bool> pres(n + 1, false);
  for (int x : v) if (0 <= x && x <= n) pres[x] = true;
  for (int i = 0; i <= n; ++i) if (!pres[i]) return i;
  return n + 1;
}

// 3. Nim Normal (gana quien toma el ultimo) y Misere Nim (pierde quien toma
el ultimo)
bool nim_normal_win(const vll& piles) {
  ll x = 0;
  for (ll p : piles) x ^= p;
  return x != 0;
}
bool nim_misere_win(const vll& piles) {
  bool all_ones = true;
  ll x = 0;
  for (ll p : piles) {
    if (p > 1) all_ones = false;
    x ^= p;
  }
  return all_ones ? (sz(piles) % 2 == 0) : (x != 0);
}
```

7.8 Criba Segmentada (Primos en Rango $[L, R]$)

- **¿Cuándo usar?**: Hallar todos los números primos en un intervalo $[L, R]$ donde $R \leq 10^{12}$ y la longitud del intervalo $R - L + 1 \leq 10^7$ (donde una criba directa de 1 a R desbordaría la memoria).
- **Complejidad**: $\mathcal{O}((R - L + 1) \log \log R + \sqrt{R} \log \log \sqrt{R})$.

```
// Retorna vector de bool donde is_prime[i] indica si (L + i) es primo
vector<bool> segmented_sieve(ll L, ll R) {
  ll lim = sqrt(R);
  vector<bool> mark(lim + 1, false);
  vll primes;
  for (ll i = 2; i <= lim; ++i) {
    if (!mark[i]) {
      primes.push_back(i);
      for (ll j = i * i; j <= lim; j += i) mark[j] = true;
    }
  }
  vector<bool> is_prime(R - L + 1, true);
  for (ll p : primes) {
    for (ll j = max(p * p, (L + p - 1) / p * p); j <= R; j += p)
      is_prime[j - L] = false;
  }
  if (L == 1) is_prime[0] = false;
  return is_prime;
}
```

7.9 Programación Lineal: Algoritmo Simplex de Dos Fases

- **¿Cuándo usar?**: Optimización continua: **Maximizar** $Z = \mathbf{c}^T\mathbf{x}$ sujeto al sistema de restricciones lineales $A\mathbf{x} \leq \mathbf{b}$ con variables no negativas $\mathbf{x} \geq 0$. Resuelve problemas de flujo con costos, mezclas, producción o asignaciones lineales continuas.
- **Complejidad**: Polinomial en la práctica (rápido para $N, M \leq 100$).

```
// Simplex de Dos Fases con Regla de Bland (Anti-ciclos)
struct Simplex {
  const ld EPS = 1e-9, INF = 1e18;
  int n, m; // n restricciones, m variables
  vi X, Y;
  vector<ld> b, c;
  vector<vector<ld>> a;
  ld Z = 0;

  Simplex(int n, int m) : n(n), m(m), b(n), c(m), a(n, vector<ld>(m)) {}

  void pivot(int x, int y) {
    swap(X[y], Y[x]);
    b[x] /= a[x][y];
    for (int j = 0; j < m; ++j) if (j != y) a[x][j] /= a[x][y];
    a[x][y] = 1.0 / a[x][y];
    for (int i = 0; i < n; ++i) {
      if (i != x && abs(a[i][y]) > EPS) {
        b[i] -= a[i][y] * b[x];
        for (int j = 0; j < m; ++j) if (j != y) a[i][j] -= a[i][y] * a[x]
[j];
        a[i][y] = -a[i][y] * a[x][y];
      }
    }
    Z += c[y] * b[x];
    for (int j = 0; j < m; ++j) if (j != y) c[j] -= c[y] * a[x][j];
    c[y] = -c[y] * a[x][y];
  }

  // Retorna {max_Z, vector_x}. Si sin solucion: {-1, {}}. Si no acotada:
{INF, {}}
  pair<ld, vector<ld>> solve() {
    X.resize(m); iota(all(X), 0);
    Y.resize(n); iota(all(Y), m);
    while (true) {
      int x = min_element(all(b)) - b.begin(), y = -1;
      if (b[x] > -EPS) break;
      for (int j = 0; j < m; ++j) if (a[x][j] < -EPS) { y = j; break; }
      if (y == -1) return {-1, {}}; // Sin solucion factible
      pivot(x, y);
    }
    while (true) {
      int x = -1, y = max_element(all(c)) - c.begin();
      if (c[y] < EPS) break;
      ld mn = INF;
      for (int i = 0; i < n; ++i) {
        if (a[i][y] > EPS && b[i] / a[i][y] < mn) {
          mn = b[i] / a[i][y]; x = i;
        }
      }
      if (x == -1) return {INF, {}}; // Solucion no acotada
      pivot(x, y);
    }
    vector<ld> ans(m, 0);
    for (int i = 0; i < n; ++i) if (Y[i] < m) ans[Y[i]] = b[i];
    return {Z, ans};
  }
};
```

7.10 Berlekamp-Massey y Recurrencias Lineales Modulares

- **¿Cuándo usar?**: Dada una sucesión $a_0, a_1, a_2, \ldots$ de la cual puedes simular o calcular los primeros $\sim 2K$ términos (mediante fuerza bruta o DP para N pequeño), pero necesitas calcular a_N para $N \leq 10^{18}$. Berlekamp-Massey encuentra la relación de recurrencia lineal más corta en $\mathcal{O}(K^2)$, y luego calcula el N-ésimo término en $\mathcal{O}(K^2 \log N)$ sin necesidad de deducir matemáticamente la matriz de transición.
- **Complejidad**: $\mathcal{O}(K^2)$ para hallar la relación, $\mathcal{O}(K^2 \log N)$ para consultar.

```
// Berlekamp-Massey + Cayley-Hamilton para sucesiones modulares
struct BerlekampMassey {
  ll mod;
  BerlekampMassey(ll m = MOD) : mod(m) {}

  ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (!b) { x = 1; y = 0; return a; }
    ll d = extgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
  }
  ll inv(ll a) {
    ll x, y; extgcd(a, mod, x, y);
    return (x % mod + mod) % mod;
  }

  vll BM(const vll& x) {
    vll ls, cur;
    int lf = 0; ll ld = 0;
    for (int i = 0; i < sz(x); ++i) {
      ll t = 0;
      for (int j = 0; j < sz(cur); ++j)
        t = (t + x[i - j - 1] * cur[j]) % mod;
      if ((t - x[i]) % mod == 0) continue;
      if (cur.empty()) {
        cur.resize(i + 1, 0);
        lf = i; ld = (t - x[i]) % mod;
        continue;
      }
      ll k = -(x[i] - t) * inv(ld) % mod;
      vll c(i - lf - 1, 0); c.push_back(k);
      for (int j = 0; j < sz(ls); ++j)
        c.push_back(-ls[j] * k % mod);
      if (sz(c) < sz(cur)) c.resize(sz(cur), 0);
      for (int j = 0; j < sz(cur); ++j)
        cur[j] = (cur[j] + c[j]) % mod;
      if (i - lf + sz(ls) >= sz(cur)) {
        ls = cur; lf = i; ld = (t - x[i]) % mod;
      }
      cur = c;
    }
    for (ll& val : cur) val = (val % mod + mod) % mod;
    return cur;
  }

  vll poly_mul(const vll& p, const vll& q, const vll& h, int m) {
    vll t(2 * m, 0);
    for (int i = 0; i < m; ++i) if (p[i])
      for (int j = 0; j < m; ++j)
        t[i + j] = (t[i + j] + p[i] * q[j]) % mod;
    for (int i = 2 * m - 1; i >= m; --i) if (t[i])
      for (int j = 0; j < m; ++j)
        t[i - j - 1] = (t[i - j - 1] + t[i] * h[j]) % mod;
    t.resize(m);
    return t;
  }

  ll get_kth(const vll& first_terms, ll k) {
    if (k < sz(first_terms)) return (first_terms[k] % mod + mod) % mod;
    vll h = BM(first_terms);
    int m = sz(h);
    if (!m) return 0;
    vll s(m, 0), t(m, 0);
    s[0] = 1;
    if (m != 1) t[1] = 1; else t[0] = h[0];
    for (; k > 0; k >>= 1) {
      if (k & 1) s = poly_mul(s, t, h, m);
      t = poly_mul(t, t, h, m);
    }
    ll ans = 0;
    for (int i = 0; i < m; ++i)
      ans = (ans + s[i] * first_terms[i]) % mod;
    return (ans % mod + mod) % mod;
```

```
  }
};
```

7.11 Interpolación de Lagrange en $\mathcal{O}(D)$ y $\mathcal{O}(D^2)$

- **¿Cuándo usar?**: Evaluar un polinomio de grado D en un punto gigante X ($X \leq 10^9$) conociendo sus valores en puntos consecutivos $y_0 = P(0), y_1 = P(1), ..., y_D = P(D)$ en $\mathcal{O}(D)$. (Muy común en sumatorias de potencias $\sum_{i=1}^{N} i^k$ donde el resultado es un polinomio de grado $k+1$).
- **Complejidad**: $\mathcal{O}(D)$ para puntos consecutivos; $\mathcal{O}(D^2)$ para puntos arbitrarios.

```
// Interpolacion de Lagrange en O(D) para x = 0, 1, ..., D
// 'y' contiene P(0), P(1), ..., P(D). Evalua P(x) mod MOD.
ll lagrange_eval_consecutive(const vll& y, ll x, ll mod = MOD) {
  int n = sz(y); // Grado = n - 1
  if (x < n) return (y[x] % mod + mod) % mod;
  vll pref(n), suff(n), fact(n), inv_fact(n);
  pref[0] = x % mod;
  for (int i = 1; i < n; ++i) pref[i] = pref[i - 1] * ((x - i) % mod) % mod;
  suff[n - 1] = (x - (n - 1)) % mod;
  for (int i = n - 2; i >= 0; --i) suff[i] = suff[i + 1] * ((x - i) % mod) %
mod;

  auto bpow = [&](ll b, ll e) {
    ll r = 1;
    for (; e > 0; e >>= 1, b = b * b % mod) if (e & 1) r = r * b % mod;
    return r;
  };
  fact[0] = 1;
  for (int i = 1; i < n; ++i) fact[i] = fact[i - 1] * i % mod;
  inv_fact[n - 1] = bpow(fact[n - 1], mod - 2);
  for (int i = n - 2; i >= 0; --i) inv_fact[i] = inv_fact[i + 1] * (i + 1) %
mod;

  ll ans = 0;
  for (int i = 0; i < n; ++i) {
    ll num = 1;
    if (i > 0) num = num * pref[i - 1] % mod;
    if (i + 1 < n) num = num * suff[i + 1] % mod;
    ll den = inv_fact[i] * inv_fact[n - 1 - i] % mod;
    if ((n - 1 - i) & 1) den = mod - den;
    ans = (ans + y[i] * num % mod * den) % mod;
  }
  return (ans % mod + mod) % mod;
}
```

Bloque 8: Programación Dinámica (DP) y Geometría Computacional

8.1 DP Clásica y Máscaras: Knapsack 0/1 y Unbounded, LCS, Edit Distance, LIS $\mathcal{O}(N \log N)$, Digit DP y SOS DP

- **¿Cuándo usar?**:
  - **Knapsack 0/1 vs Unbounded**: Seleccionar objetos con peso w_i y valor v_i sin exceder capacidad W. Si cada objeto se usa a lo sumo 1 vez (0/1), el bucle de capacidad va **hacia atrás** (`W` a `w[i]`). Si se puede repetir infinitas veces (Unbounded), va **hacia adelante** (`w[i]` a `W`).
  - **LCS y Edit Distance**: Subsecuencia común más larga o mínimo número de inserciones/borrados/reemplazos para transformar $S_1 \to S_2$ en $\mathcal{O}(N \cdot M)$.
  - **LIS en $\mathcal{O}(N \log N)$**: Subsecuencia estrictamente creciente más larga (`lower_bound` ; usar `upper_bound` para no decreciente $\leq$).
  - **Digit DP**: Contar cuántos números en $[L, R]$ ($R \leq 10^{18}$ o 10^{1000}) cumplen una propiedad sobre sus dígitos, calculando `solve(R) - solve(L - 1)` .
  - **SOS DP (Sum Over Subsets)**: Calcular para todas las máscaras $m \in [0, 2^N - 1]$ la suma de $A[\text{sub}]$ sobre todas las submáscaras $\text{sub} \subseteq m$ en $\mathcal{O}(N \cdot 2^N)$ en lugar de $\mathcal{O}(3^N)$.

```
// 1. Knapsack 0/1 en O(N * W) tiempo y O(W) memoria
ll knapsack_01(int W, const vi& weight, const vll& val) {
  vll dp(W + 1, 0);
  for (int i = 0; i < sz(weight); ++i)
    for (int w = W; w >= weight[i]; --w) // Hacia adelante si es Unbounded
      dp[w] = max(dp[w], dp[w - weight[i]] + val[i]);
  return dp[W];
}

// 2. Edit Distance (Levenshtein) en O(N * M)
int edit_distance(const string& a, const string& b) {
  int n = sz(a), m = sz(b);
  vector<vi> dp(n + 1, vi(m + 1, 0));
  for (int i = 0; i <= n; ++i) dp[i][0] = i;
  for (int j = 0; j <= m; ++j) dp[0][j] = j;
  for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j) {
      if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1];
      else dp[i][j] = 1 + min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j -
1]});
    }
  return dp[n][m];
}

// 3. LIS (Longest Increasing Subsequence) en O(N log N)
int lis_length(const vll& a) {
  vll tails;
  for (ll x : a) {
    auto it = lower_bound(all(tails), x); // upper_bound para <=
    if (it == tails.end()) tails.pb(x);
    else *it = x;
  }
  return sz(tails);
}

// 4. Plantilla de Digit DP (Cuenta numeros en [0, R] con suma de digitos %
K == 0)
string dig_s;
int K_mod;
ll memo_dig[20][200][2];

ll dp_dig(int pos, int rem, bool tight) {
  if (pos == sz(dig_s)) return (rem == 0) ? 1 : 0;
  ll& res = memo_dig[pos][rem][tight];
  if (res != -1) return res;
  res = 0;
  int lim = tight ? (dig_s[pos] - '0') : 9;
  for (int d = 0; d <= lim; ++d)
    res += dp_dig(pos + 1, (rem + d) % K_mod, tight && (d == lim));
  return res;
}
ll solve_digit_dp(ll R, int K) {
  if (R < 0) return 0;
  dig_s = to_string(R); K_mod = K;
  memset(memo_dig, -1, sizeof(memo_dig));
  return dp_dig(0, 0, true);
}

// 5. SOS DP (Sum Over Subsets) en O(N * 2^N)
// Al terminar, F[mask] = suma de A[sub] para todo subconjunto 'sub' de
'mask'.
vll sos_dp(int N, const vll& A) {
  vll F = A;
  for (int i = 0; i < N; ++i)
    for (int mask = 0; mask < (1 << N); ++mask)
      if (mask & (1 << i))
        F[mask] += F[mask ^ (1 << i)];
  return F;
}
```

8.2 Optimizaciones de DP: Convex Hull Trick (Estático y Dinámico), Divide & Conquer DP y Knuth's Optimization

- **¿Cuándo usar cada optimización de DP?**:

Optimización Recurrencia Típica Condición Suficiente Reducción de Complejidad Convex Hull Trick (CHT) $dp[i] = \min_{j<i}\{dp[j] + m_j \cdot x_i\}$ Transición es una recta y = mjx + bj $\mathcal{O}(N2) \to \mathcal{O}(N\log N)$ o $\mathcal{O}(N)$ Divide & Conquer DP $dp[k][i] = \min_{j<i}\{dp[k-1][j] + C(j,i)\}$ Punto óptimo monótono: opt[k][i] ≤ opt[k][i + 1] $\mathcal{O}(K \cdot N2) \to \mathcal{O}(K \cdot N\log N)$ Knuth's Optimization $dp[l][r] = \min_{l<k<r}\{dp[l][k] + dp[k][r]\} + C(l,r)$ opt[l][r − 1] ≤ opt[l][r] ≤ opt[l + 1][r] $\mathcal{O}(N3) \to \mathcal{O}(N2)$

```
// 1. Convex Hull Trick Dinamico (LineContainer - inserta rectas y = k*x + m
//    en cualquier orden y consulta el MAXIMO en x en O(log N)).
//    NOTA: Para consultar el MINIMO, insertar (-k, -m) y devolver -
query(x).
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // Division entera hacia abajo (floor)
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) { // Agrega la recta y = k*x + m
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) const { // Retorna max(k*x + m) en O(log N)
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};

// 2. Divide and Conquer DP Optimization en O(K * N log N)
vll dp_prev, dp_cur;
ll cost_dc(int j, int i); // Definir costo de cubrir [j..i]

void compute_dc(int l, int r, int optl, int optr) {
  if (l > r) return;
  int mid = (l + r) / 2;
  pair<ll, int> best = {INF, -1};
  for (int j = optl; j <= min(mid, optr); ++j)
    best = min(best, {dp_prev[j - 1] + cost_dc(j, mid), j});
  dp_cur[mid] = best.fi;
  int opt = best.se;
  compute_dc(l, mid - 1, optl, opt);
  compute_dc(mid + 1, r, opt, optr);
}

// 3. Knuth's Optimization para DP de Intervalos en O(N^2)
// Resuelve dp[l][r] = min_{l <= k < r} (dp[l][k] + dp[k+1][r]) + C(l, r)
ll solve_knuth(int n, const vector<vll>& C) {
  vector<vll> dp(n + 1, vll(n + 1, 0));
  vector<vi> opt(n + 1, vi(n + 1, 0));
  for (int i = 1; i <= n; ++i) opt[i][i] = i;

  for (int len = 2; len <= n; ++len) {
    for (int l = 1; l + len - 1 <= n; ++l) {
      int r = l + len - 1;
      dp[l][r] = INF;
      int kl = opt[l][r - 1], kr = min(r - 1, opt[l + 1][r]);
      for (int k = kl; k <= kr; ++k) {
        ll val = dp[l][k] + dp[k + 1][r] + C[l][r];
        if (val < dp[l][r]) { dp[l][r] = val; opt[l][r] = k; }
      }
    }
  }
  return dp[1][n];
}
```

Ejemplo de Uso (Convex Hull Trick Dinámico):

```
int main() {
  LineContainer cht;
  cht.add(2, 3);  // y = 2x + 3
  cht.add(-1, 10);// y = -x + 10
  cout << cht.query(4) << "\n"; // max(2*4+3 = 11, -4+10 = 6) -> 11
}
```

8.3 Geometría 2D: `Point` , `Line` , Segmentos, `Polygon` (Área, Centroide, Punto en Polígono, Corte), `Convex Hull` , Rotating Callipers, Minkowski, Semiplanos, `Circle` , Closest Pair y KD-Tree

- **¿Cuándo usar?**:
  - `Pt` **(Punto/Vector 2D)**: Operaciones vectoriales base. Producto punto (`dot(a, b)`) mide ángulos/proyecciones; producto cruz (`cross(a, b)`) mide área con signo y orientación de giro (`> 0` izquierda/CCW, `< 0` derecha/CW, `0` colineal).
  - **Intersección de Segmentos y Distancia Punto-Segmento**: Detectar choques de segmentos o mínima distancia de un punto a una pared.
  - **Polígonos**: Fórmula de Shoelace (`polygon_area2` retorna $2 \times$ Área exacta en enteros si las coordenadas son enteras), centroide, verificar si un punto está dentro de un polígono cualquiera en $\mathcal{O}(N)$ (*Ray Casting*) o dentro de un **polígono convexo en** $\mathcal{O}(\log N)$.

- **Convex Hull (Monotone Chain) + Rotating Callipers**: Envolvente convexa en $\mathcal{O}(N \log N)$ y par de puntos más lejanos (diámetro del conjunto de puntos) en $\mathcal{O}(N)$.
- **Closest Pair of Points (Sweep-Line)**: Hallar la menor distancia entre cualquier par de puntos de un conjunto de N puntos en $\mathcal{O}(N \log N)$.

```
// 1. Estructura Base de Punto / Vector 2D (usar ld para general o ll para
exacto)
struct Pt {
  ld x, y;
  Pt(ld x = 0, ld y = 0) : x(x), y(y) {}
  Pt operator+(const Pt& o) const { return {x + o.x, y + o.y}; }
  Pt operator-(const Pt& o) const { return {x - o.x, y - o.y}; }
  Pt operator*(ld k) const { return {x * k, y * k}; }
  Pt operator/(ld k) const { return {x / k, y / k}; }
  bool operator<(const Pt& o) const {
    return abs(x - o.x) > EPS ? x < o.x : y < o.y - EPS;
  }
  bool operator==(const Pt& o) const {
    return abs(x - o.x) <= EPS && abs(y - o.y) <= EPS;
  }
  ld norm2() const { return x * x + y * y; }
  ld norm() const { return hypot(x, y); }
  Pt perp() const { return {-y, x}; } // Rotacion 90 grados CCW
  Pt rot(ld ang_rad) const {          // Rotacion CCW por 'ang_rad'
    return {x * cos(ang_rad) - y * sin(ang_rad),
            x * sin(ang_rad) + y * cos(ang_rad)};
  }
};

int sgn(ld x) { return (x > EPS) - (x < -EPS); }
ld dot(const Pt& a, const Pt& b) { return a.x * b.x + a.y * b.y; }
ld cross(const Pt& a, const Pt& b) { return a.x * b.y - a.y * b.x; }
// Giro de a -> b -> c: > 0 izquierda (CCW), < 0 derecha (CW), 0 colineal
ld cross(const Pt& a, const Pt& b, const Pt& c) { return cross(b - a, c -
a); }

// 2. Orden Radial (Polar Sort alrededor del origen (0, 0) en O(N log N))
int half_plane(const Pt& p) {
  return (p.y > 0 || (p.y == 0 && p.x > 0)) ? 0 : 1;
}
bool polar_cmp(const Pt& a, const Pt& b) {
  int ha = half_plane(a), hb = half_plane(b);
  return (ha != hb) ? (ha < hb) : (cross(a, b) > 0);
}

// 3. Segmentos y Rectas: Punto en segmento, Distancia e Interseccion
bool on_segment(const Pt& p, const Pt& a, const Pt& b) {
  return sgn(cross(a, b, p)) == 0 && sgn(dot(a - p, b - p)) <= 0;
}
ld dist_point_segment(const Pt& p, const Pt& a, const Pt& b) {
  if (a == b) return (p - a).norm();
  ld d2 = (b - a).norm2();
  ld t = clamp(dot(p - a, b - a) / d2, (ld)0.0, (ld)1.0);
  Pt proj = a + (b - a) * t;
  return (p - proj).norm();
}
bool segments_intersect(const Pt& a, const Pt& b, const Pt& c, const Pt& d)
{
  int d1 = sgn(cross(a, b, c)), d2 = sgn(cross(a, b, d));
  int d3 = sgn(cross(c, d, a)), d4 = sgn(cross(c, d, b));
  if (d1 * d2 < 0 && d3 * d4 < 0) return true;
  return on_segment(c, a, b) || on_segment(d, a, b) ||
         on_segment(a, c, d) || on_segment(b, c, d);
}
// Interseccion de dos rectas infinitas (a->b) y (c->d) (asume no paralelas)
Pt line_intersection(const Pt& a, const Pt& b, const Pt& c, const Pt& d) {
  return a + (b - a) * (cross(c - a, d - c) / cross(b - a, d - c));
}

// 4. Poligonos: Area (Shoelace), Centroide y Punto en Poligono O(N)
ld polygon_area(const vector<Pt>& p) {
  ld a = 0;
  for (int i = 0; i < sz(p); ++i) a += cross(p[i], p[(i + 1) % sz(p)]);
  return abs(a) / 2.0; // Sin dividir entre 2 es siempre entero en coords
enteras
}
// Retorna: 2 si esta en el BORDE, 1 si esta ESTRICTAMENTE DENTRO, 0 si
FUERA
int point_in_polygon(const Pt& q, const vector<Pt>& p) {
  int n = sz(p), wn = 0;
  for (int i = 0; i < n; ++i) {
    Pt a = p[i], b = p[(i + 1) % n];
    if (on_segment(q, a, b)) return 2;
    if (a.y <= q.y) {
      if (b.y > q.y && cross(a, b, q) > EPS) wn++;
    } else {
      if (b.y <= q.y && cross(a, b, q) < -EPS) wn--;
    }
  }
  return wn != 0 ? 1 : 0;
}

// 5. Convex Hull (Monotone Chain de Andrew en O(N log N) — orden CCW)
vector<Pt> convex_hull(vector<Pt> pts) {
  int n = sz(pts), k = 0;
  if (n <= 1) return pts;
  sort(all(pts));
  pts.erase(unique(all(pts)), pts.end());
  n = sz(pts);
  if (n <= 2) return pts;
  vector<Pt> h(2 * n);
  for (int i = 0; i < n; ++i) {
    while (k >= 2 && cross(h[k - 2], h[k - 1], pts[i]) <= EPS) k--;
    h[k++] = pts[i];
  }
  for (int i = n - 2, t = k + 1; i >= 0; --i) {
    while (k >= t && cross(h[k - 2], h[k - 1], pts[i]) <= EPS) k--;
    h[k++] = pts[i];
  }
  h.resize(k - 1);
  return h;
}

// 6. Rotating Callipers: Mayor distancia al cuadrado en un poligono convexo
CCW
ld rotating_callipers_max_dist2(const vector<Pt>& h) {
  int n = sz(h);
  if (n <= 1) return 0;
  if (n == 2) return (h[0] - h[1]).norm2();
  ld max_d2 = 0;
  for (int i = 0, j = 1; i < n; ++i) {
    Pt edge = h[(i + 1) % n] - h[i];
    while (cross(edge, h[(j + 1) % n] - h[i]) > cross(edge, h[j] - h[i]))
      j = (j + 1) % n;
    max_d2 = max({max_d2, (h[i] - h[j]).norm2(),
                  (h[(i + 1) % n] - h[j]).norm2()});
  }
  return max_d2;
}

// 7. Par de Puntos Mas Cercanos (Sweep-Line con std::set en O(N log N))
ld closest_pair_distance(vector<Pt> pts) {
  sort(all(pts));
  set<pair<ld, ld>> active; // Guarda {y, x}
  ld best = 1e18;
  int j = 0;
  for (int i = 0; i < sz(pts); ++i) {
    while (pts[i].x - pts[j].x >= best) {
      active.erase({pts[j].y, pts[j].x});
      j++;
    }
    auto it1 = active.lower_bound({pts[i].y - best, -1e18});
    auto it2 = active.upper_bound({pts[i].y + best, 1e18});
    for (auto it = it1; it != it2; ++it) {
      ld dx = pts[i].x - it->second, dy = pts[i].y - it->first;
      best = min(best, hypot(dx, dy));
    }
    active.insert({pts[i].y, pts[i].x});
  }
  return best;
}
```

Ejemplo de Input y Uso (Convex Hull + Área + Punto en Polígono):

```
Input: 5 puntos: (0,0), (4,0), (4,4), (0,4), (2,2).
Convex Hull descarta (2,2) y deja el cuadrado [(0,0), (4,0), (4,4), (0,4)]
de area 16.
```

```
int main() {
  vector<Pt> pts = {{0, 0}, {4, 0}, {4, 4}, {0, 4}, {2, 2}};
  vector<Pt> hull = convex_hull(pts);
  cout << polygon_area(hull) << "\n";              // 16.0
  cout << point_in_polygon({2, 2}, hull) << "\n";// 1 (estrictamente dentro)
}
```

8.4 Intersección de Semiplanos (Halfplane Intersection)

- **¿Cuándo usar?**: Hallar el polígono convexo formado por la intersección de N semiplanos dirigidos de la forma «a la izquierda del vector $\overrightarrow{pq}$» (o desigualdades lineales $Ax + By + C \leq 0$). Útil en problemas de visibilidad (núcleo de un polígono / galería de arte), encontrar si un conjunto de restricciones 2D tiene región factible, o el corte continuo de una figura convexa.
- **Complejidad**: $\mathcal{O}(N \log N)$.

```
// Semiplanos a la izquierda del rayo p -> p + v
struct Halfplane {
  Pt p, v;
  ld angle;
  Halfplane() {}
  Halfplane(Pt a, Pt b) : p(a), v(b - a), angle(atan2(v.y, v.x)) {}
  bool operator<(const Halfplane& o) const { return angle < o.angle; }
  bool out(Pt q) const { return cross(v, q - p) < -EPS; }
};

Pt halfplane_intersect(Halfplane a, Halfplane b) {
  ld c = cross(a.v, b.v);
  return a.p + a.v * (cross(b.p - a.p, b.v) / c);
}

// Retorna vertices del poligono convexo en orden CCW (vacio si no hay
interseccion)
vector<Pt> halfplane_intersection(vector<Halfplane> h) {
  const ld BOX = 1e9;
  h.push_back({{-BOX, -BOX}, {BOX, -BOX}});
  h.push_back({{BOX, -BOX}, {BOX, BOX}});
  h.push_back({{BOX, BOX}, {-BOX, BOX}});
  h.push_back({{-BOX, BOX}, {-BOX, -BOX}});

  sort(all(h));
  deque<Halfplane> dq;
  deque<Pt> pts;

  for (auto& cur : h) {
    while (!pts.empty() && cur.out(pts.back())) {
      pts.pop_back(); dq.pop_back();
    }
    while (!pts.empty() && cur.out(pts.front())) {
      pts.pop_front(); dq.pop_front();
    }
    if (!dq.empty() && abs(cross(cur.v, dq.back().v)) < EPS) {
      if (dot(cur.v, dq.back().v) < 0) return {}; // Opuestos paralelos
      if (cur.out(dq.back().p)) dq.back() = cur;  // Mismo sentido, mas
restrictivo
      continue;
    }
    if (!dq.empty()) pts.push_back(halfplane_intersect(dq.back(), cur));
    dq.push_back(cur);
  }
  while (!pts.empty() && dq.front().out(pts.back())) {
    pts.pop_back(); dq.pop_back();
  }
  if (sz(dq) < 3) return {};
  pts.push_back(halfplane_intersect(dq.back(), dq.front()));
  return vector<Pt>(all(pts));
}
```

8.5 Suma de Minkowski de Polígonos Convexos

- **¿Cuándo usar?**: Dados dos polígonos convexos P y Q, calcular el polígono convexo resultante de la suma vectorial $P + Q = \{p + q : p \in P, q \in Q\}$.
  - **Detección de Colisión / Distancia**: P y Q se intersectan si y solo si el origen $(0, 0)$ está dentro de la suma de Minkowski $P + (-Q)$.
- **Complejidad**: $\mathcal{O}(|P| + |Q|)$.

```
// Suma de Minkowski P + Q para dos poligonos convexos en orden CCW
vector<Pt> minkowski_sum(vector<Pt> P, vector<Pt> Q) {
  auto reorder = [](vector<Pt>& pts) {
    int pos = 0;
    for (int i = 1; i < sz(pts); ++i)
      if (pts[i].y < pts[pos].y || (pts[i].y == pts[pos].y && pts[i].x <
pts[pos].x))
        pos = i;
    rotate(pts.begin(), pts.begin() + pos, pts.end());
  };
  reorder(P); reorder(Q);
  P.push_back(P[0]); P.push_back(P[1]);
  Q.push_back(Q[0]); Q.push_back(Q[1]);

  vector<Pt> res;
  int i = 0, j = 0;
  while (i + 2 < sz(P) || j + 2 < sz(Q)) {
    res.push_back(P[i] + Q[j]);
    ld c = cross(P[i + 1] - P[i], Q[j + 1] - Q[j]);
    if (c >= -EPS) i++;
    if (c <= EPS) j++;
  }
  return res;
}
```

Bloque 9: Miscelánea y Parsing Especial

9.1 Evaluador de Expresiones Aritméticas (Shunting-Yard Parser)

- **¿Cuándo usar?**: Evaluar en $\mathcal{O}(|S|)$ expresiones matemáticas en un string que contienen números enteros, operadores `+`, `-`, `*`, `/`, paréntesis `( )` y signo menos unario `-x`.
- **Complejidad**: $\mathcal{O}(|S|)$.

```
struct ExpressionParser {
  int priority(char op) {
    if (op < 0) return 3; // Operador unario (-x)
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return -1;
  }
  void apply_op(stack<ll>& st, char op) {
    if (op < 0) {
      ll x = st.top(); st.pop();
      st.push(-x);
    } else {
      ll b = st.top(); st.pop();
      ll a = st.top(); st.pop();
      if (op == '+') st.push(a + b);
      if (op == '-') st.push(a - b);
      if (op == '*') st.push(a * b);
      if (op == '/') st.push(a / b);
    }
  }
  ll evaluate(const string& s) {
    stack<ll> st;
    stack<char> op;
    bool may_be_unary = true;
    for (int i = 0; i < sz(s); ++i) {
      if (isspace(s[i])) continue;
      if (s[i] == '(') {
        op.push('('); may_be_unary = true;
      } else if (s[i] == ')') {
        while (op.top() != '(') {
          apply_op(st, op.top()); op.pop();
        }
        op.pop(); may_be_unary = false;
      } else if (s[i] == '+' || s[i] == '-' || s[i] == '*' || s[i] == '/') {
        char cur_op = s[i];
        if (may_be_unary && (cur_op == '+' || cur_op == '-')) {
          if (cur_op == '-') cur_op = -cur_op;
          else continue;
        }
        while (!op.empty() && (
          (cur_op >= 0 && priority(op.top()) >= priority(cur_op)) ||
          (cur_op < 0 && priority(op.top()) > priority(cur_op)))) {
          apply_op(st, op.top()); op.pop();
        }
        op.push(cur_op); may_be_unary = true;
      } else {
        ll val = 0;
        while (i < sz(s) && isdigit(s[i]))
          val = val * 10 + (s[i++] - '0');
        --i;
        st.push(val); may_be_unary = false;
      }
    }
    while (!op.empty()) {
      apply_op(st, op.top()); op.pop();
    }
    return st.top();
  }
};
```

9.2 Clique Máximo (Bron-Kerbosch Branch & Bound)

- **¿Cuándo usar?**: Encontrar el clique máximo (subgrafo completo más grande) en un grafo no dirigido con $N \leq 60$. (También resuelve el **Conjunto Independiente Máximo** corriendo el algoritmo en el grafo complemento).
- **Complejidad**: $\mathcal{O}(3^{N/3})$ en el peor caso; instantáneo para $N \leq 60$ con podas de bitset.

```
// Max Clique para N <= 64 usando mascaras ull
struct MaxClique {
  int n;
  vector<ull> adj;
  vi best_clique;

  MaxClique(int n) : n(n), adj(n, 0) {}

  void add_edge(int u, int v) {
    adj[u] |= (1ULL << v); adj[v] |= (1ULL << u);
  }
  void expand(ull R, ull P, ull X, vi& cur) {
    if (P == 0 && X == 0) {
      if (sz(cur) > sz(best_clique)) best_clique = cur;
      return;
    }
    if (sz(cur) + __builtin_popcountll(P) <= sz(best_clique)) return;
    int u = __builtin_ctzll(P | X);
    ull cand = P & ~adj[u];
    while (cand) {
      int v = __builtin_ctzll(cand);
      cand &= ~(1ULL << v);
      cur.push_back(v);
      expand(R | (1ULL << v), P & adj[v], X & adj[v], cur);
      cur.pop_back();
      P &= ~(1ULL << v);
      X |= (1ULL << v);
    }
  }
  vi solve() {
    best_clique.clear();
    ull P = (n == 64 ? ~0ULL : (1ULL << n) - 1);
    vi cur;
    expand(0, P, 0, cur);
    return best_clique;
  }
};
```

Bloque 10: Compendio Teórico, Teoremas, Fórmulas e Identidades

10.1 Tabla de Optimización de Programación Dinámica (DP Speedups)

Nombre	Recurrencia Original	Condición Suficiente	De	A
CHT 1	$dp[i] = \min_{j<i}\{dp[j] + b[j] \cdot a[i]\}$	$b[j] \geq b[j+1]$ (si $a[i] \leq a[i+1]$, monótona)	$\mathcal{O}(N^2)$	$\mathcal{O}(N)$
CHT 2	$dp[i][j] = \min_{k<j}\{dp[i-1][k] + b[k] \cdot a[j]\}$	$b[k] \geq b[k+1]$	$\mathcal{O}(KN^2)$	$\mathcal{O}(KN)$
D&Q	$dp[i][j] = \min_{k<j}\{dp[i-1][k] + C[k][j]\}$	$opt[i][j] \leq opt[i][j+1]$ (Desigualdad Cuadrilátero)	$\mathcal{O}(KN^2)$	$\mathcal{O}(KN \log N)$
Knuth	$dp[i][j] = \min_{i<k<j}\{dp[i][k] + dp[k][j]\} + C[i][j]$	$opt[i][j-1] \leq opt[i][j] \leq opt[i+1][j]$	$\mathcal{O}(N^3)$	$\mathcal{O}(N^2)$
Aliens / WQS	$\min dp[N][K]$ con K fijo	$f(k) = dp[N][k]$ es convexa/cóncava en k	$\mathcal{O}(KN)$	$\mathcal{O}(N \log(\text{costo}))$

- **Desigualdad del Cuadrilátero**: $C[a][c] + C[b][d] \leq C[a][d] + C[b][c]$ para todo $a \leq b \leq c \leq d$. Garantiza la monotonicidad de los puntos óptimos $opt[i][j]$.

10.2 Combinatoria y Sumatorias

- **Sumas Polinomiales**:
  - $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$
  - $\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$
  - $\sum_{k=1}^{n} k^3 = \left(\frac{n(n+1)}{2}\right)^2$
  - $\sum_{k=1}^{n} k^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$
  - $\sum_{k=0}^{n} x^k = \frac{x^{n+1}-1}{x-1} \quad (x \neq 1)$
  - $\sum_{k=1}^{n} kx^k = \frac{x-(n+1)x^{n+1}+nx^{n+2}}{(1-x)^2}$
- **Identidad del Palo de Hockey (Hockey-Stick)**: $\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$
- **Identidad de Vandermonde**: $\sum_{k=0}^{r} \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$
- **Coeficiente Multinomial**: $\binom{n}{k_1,k_2,\ldots,k_m} = \frac{n!}{k_1!k_2!\cdots k_m!}$, donde $\sum k_i = n$.
- **Números de Catalan (C_n)**:
  - $C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$, con $C_0 = 1$ y $C_{n+1} = \frac{2(2n+1)}{n+2}C_n$.
  - Primeros valores: $1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796$.
  - Aplicaciones: Secuencias balanceadas de n pares de paréntesis; triangulaciones de un polígono convexo de $n+2$ lados; árboles binarios no etiquetados con n nodos; caminos de Dyck de $(0,0)$ a $(2n,0)$ que nunca caen bajo el eje X.
- **Desarreglos (Derangements D_n — permutaciones sin puntos fijos)**:
  - $D_0 = 1, D_1 = 0, D_n = (n-1)(D_{n-1} + D_{n-2}) = nD_{n-1} + (-1)^n$.
  - Permutaciones con exactamente k puntos fijos: $\binom{n}{k}D_{n-k}$.
- **Números de Stirling de 1ra Especie ($s(n,k)$ o $\begin{bmatrix} n \\ k \end{bmatrix}$)**:
  - Permutaciones de n elementos con exactamente k ciclos disjuntos:
  - $\begin{bmatrix} n \\ k \end{bmatrix} = (n-1)\begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix}$.
- **Números de Stirling de 2da Especie ($S(n,k)$ o $\begin{Bmatrix} n \\ k \end{Bmatrix}$)**:
  - Particionar n elementos en k subconjuntos no vacíos:
  - $\begin{Bmatrix} n \\ k \end{Bmatrix} = k\begin{Bmatrix} n-1 \\ k \end{Bmatrix} + \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix}$.
  - Funciones sobreyectivas de n a k: $k!\begin{Bmatrix} n \\ k \end{Bmatrix} = \sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$.
- **Números de Bell (B_n)**:
  - Número de particiones totales de un conjunto de n elementos: $B_n = \sum_{k=0}^{n} S(n,k) = \sum_{k=0}^{n-1}\binom{n-1}{k}B_k$.
- **Teorema del Número Pentagonal de Euler (Función de Partición $p(n)$)**:
  - $p(n) = \sum_{k\neq 0}(-1)^{k-1}p\left(n - \frac{k(3k-1)}{2}\right) = p(n-1) + p(n-2) - p(n-5) - p(n-7) + \ldots$
- **Lema de Burnside (Conteo bajo simetría)**:
  - $|X/G| = \frac{1}{|G|}\sum_{g\in G}|X^g|$, donde $|X^g|$ es el número de elementos invariantes bajo la acción de g.
  - **Collares de n cuentas con k colores**: $\frac{1}{n}\sum_{d|n}\phi(d)k^{n/d}$.

10.3 Teoría de Números y Aritmética Modular

- **Ecuaciones Diofánticas Lineales**:
  - $ax + by = c$ tiene solución entera si y solo si $g = \gcd(a,b) \mid c$.
  - Si (x_0, y_0) es una solución particular, todas las soluciones son: $x = x_0 + k \cdot \frac{b}{g}, \quad y = y_0 - k \cdot \frac{a}{g} \quad (k \in \mathbb{Z})$.
- **Problema de las Monedas de Frobenius (Chicken McNuggets Theorem)**:
  - Para $\gcd(a,b) = 1$, el mayor número entero que NO puede expresarse como $ax + by$ $(x, y \geq 0)$ es: $g(a,b) = a \cdot b - a - b$.
  - El número exacto de enteros positivos no representables es $\frac{(a-1)(b-1)}{2}$.
- **Teorema de Wilson**:
  - Un entero $p > 1$ es primo si y solo si $(p-1)! \equiv -1 \pmod{p}$.
- **Teorema de Euler y Reducción de Torres de Potencias**:
  - Si $\gcd(a,m) = 1$, $a^{\phi(m)} \equiv 1 \pmod{m}$.
  - En general para cualquier a: $a^b \equiv a^{b \bmod \phi(m))+\phi(m)} \pmod{m}$ si $b \geq \phi(m)$.
- **Fórmula de Inversión de Möbius**:
  - $g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g\left(\frac{n}{d}\right)$.
  - Propiedad clave: $\sum_{d|n}\mu(d) = [n=1]$.
  - Identidad de coprimalidad: $[\gcd(i,j)=1] = \sum_{d|\gcd(i,j)}\mu(d)$.
- **Símbolo de Legendre y Criterio de Euler**:
  - $\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p} \in \{1, -1, 0\}$.
  - a es residuo cuadrático mod p primo impar si $\left(\frac{a}{p}\right) = 1$.
  - Ley de Reciprocidad Cuadrática: $\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{\frac{p-1}{2}\frac{q-1}{2}}$.
- **Teorema de los Dos Cuadrados de Fermat**:
  - Un primo impar p se puede escribir como $a^2 + b^2$ si y solo si $p \equiv 1 \pmod{4}$.
  - Un entero n es suma de dos cuadrados si y solo si todo factor primo $q \equiv 3 \pmod{4}$ aparece con exponente par en su factorización.
- **Ternas Pitagóricas Primitivas**:
  - Todo triplete $a^2 + b^2 = c^2$ con $\gcd(a,b,c) = 1$ se genera mediante: $a = m^2 - n^2, \quad b = 2mn, \quad c = m^2 + n^2$, con $m > n > 0$, $\gcd(m,n) = 1$, y m, n de distinta paridad.
- **Raíces Primitivas**:
  - g es raíz primitiva mod n si el orden multiplicativo de g es $\phi(n)$.

- Existen raíces primitivas solo para $n \in \{1, 2, 4, p^k, 2p^k\}$ con p primo impar.
- Para verificar si g es raíz primitiva mod p: verificar que $g^{(p-1)/q} \not\equiv 1 \pmod{p}$ para todos los factores primos q de $p-1$.

- **Período de Pisano (Fibonacci mod M)**:
  - El período $\pi(M)$ de Fibonacci módulo M siempre existe y es $\leq 6M$.
  - $\pi(10^k) = 1.5 \times 10^k$ para todo $k \geq 3$. $\pi(10^9) = 1.5 \times 10^9$.

10.4 Teoría de Grafos, Árboles y Redes

- **Teorema Matriz-Árbol de Kirchhoff (Matrix-Tree Theorem)**:
  - Sea $L = D - A$ la matriz Laplaciana del grafo ($D_{ii} = \deg(v_i)$, $A_{ij} =$ aristas entre i y j).
  - El número de árboles generadores (spanning trees) del grafo es igual al valor absoluto de cualquier cofactor de L (eliminar fila i y columna i y calcular $\det(L_{ii})$).
  - En grafos dirigidos: el número de arborescencias dirigidas enraizadas en r es el cofactor del Laplaciano dirigido $L^{\text{dir}} = D^{\text{in}} - A$ eliminando la fila y columna r.
- **Fórmula de Cayley y Bosques**:
  - Número de árboles etiquetados en n vértices: n^{n-2}.
  - Número de bosques etiquetados en n vértices con k componentes conexas donde los vértices $1, ..., k$ pertenecen a componentes distintas: $k \cdot n^{n-k-1}$.
- **Código de Prüfer**:
  - Biyección entre árboles etiquetados con n vértices y secuencias de longitud $n-2$ con elementos en $\{1, ..., n\}$.
  - Un vértice con grado d aparece exactamente $d-1$ veces en el código de Prüfer.
- **Teorema de König y Relaciones de Gallai**:
  - En grafos bipartitos: Matching Máximo = Min Vertex Cover. Max Independent Set = V − Matching Máximo.
  - Teorema de Gallai (para grafos generales sin vértices aislados): $\alpha(G) + \tau(G) = V$ (Indep Set + Vertex Cover = V). $\alpha'(G) + \tau'(G) = V$ (Matching Máximo + Edge Cover Mínimo = V).
- **Teorema de Dilworth (Anticadenas en Posets)**:
  - En cualquier conjunto parcialmente ordenado (DAG transitivo), el tamaño de la **anticadena más grande** es igual al número mínimo de cadenas disjuntas necesarias para cubrir todos los elementos.
  - Se resuelve modelando como Maximum Bipartite Matching.
- **Teorema de Tutte-Berge y Matrimonio de Hall**:
  - Condición de Hall para emparejamiento completo de X a Y en bipartito: $|N(S)| \geq |S|$ para todo subconjunto $S \subseteq X$.
  - Fórmula de Tutte-Berge: El tamaño del matching máximo en cualquier grafo es $\frac{1}{2} \min_{U \subseteq V}(|V| + |U| - \text{odd}(G - U))$.
- **Criterio Gráfico de Erdős-Gallai**:
  - Una secuencia de grados $d_1 \geq d_2 \geq ... \geq d_n$ corresponde a un grafo simple no dirigido si y solo si $\sum d_i$ es par y para todo $1 \leq k \leq n$: $\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$.
- **Emparejamiento Estable (Gale-Shapley)**:
  - Si los proponentes son los hombres, el algoritmo genera el emparejamiento estable **óptimo para los hombres** y **pésimo para las mujeres** en $\mathcal{O}(N^2)$.
- **Flujo Máximo con Demandas (Cotas Inferiores $l(u,v) \leq f(u,v) \leq c(u,v)$)**:
  - Crear super-fuente S' y super-sumidero T'.
  - Para cada arista (u, v) con capacidad $[l, c]$, agregar arista en la red con capacidad $c - l$.
  - Para cada vértice u, definir balance $b(u) = \sum_v l(v,u) - \sum_w l(u,w)$.
  - Si $b(u) > 0$, agregar arista (S', u) con capacidad $b(u)$.
  - Si $b(u) < 0$, agregar arista (u, T') con capacidad $-b(u)$.
  - Agregar arista infinita (T, S) con capacidad ∞. Si el flujo máximo desde S' satura todas las aristas que salen de S', existe circulación factible.
- **Fórmula de Euler para Grafos Planares**:
  - $V - E + F = 1 + C$, donde C es el número de componentes conexas ($V - E + F = 2$ si es conexo).
  - Corolarios para grafos simples conexos con $V \geq 3$: $E \leq 3V - 6$ (si no tiene triángulos, $E \leq 2V - 4$). Siempre existe al menos un vértice con grado ≤ 5.

10.5 Geometría y Distancias

- **Teorema de Pick**:
  - Para un polígono simple con vértices en coordenadas enteras (retícula): Área $= I + \frac{B}{2} - 1$ donde I es el número de puntos enteros estrictamente interiores y B es el número de puntos enteros en el borde.
  - Para un segmento de (x_1, y_1) a (x_2, y_2), el número de puntos en el borde es $\gcd(|x_2 - x_1|, |y_2 - y_1|)$.
- **Fórmula de Herón y Radios de Triángulos**:
  - Semiperímetro: $s = \frac{a+b+c}{2}$.
  - Área: $A = \sqrt{s(s-a)(s-b)(s-c)}$.
  - Radio del incírculo (inradius): $r = \frac{A}{s}$.
  - Radio del circuncírculo (circumradius): $R = \frac{abc}{4A}$.
- **Distancia de Gran Círculo (Fórmula de Haversine)**:
  - Distancia entre dos puntos en una esfera de radio R con latitud/longitud (ϕ_1, λ_1) y (ϕ_2, λ_2) en radianes: $d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$.

10.6 Teoría de Juegos y XOR

- **Teorema de Sprague-Grundy**:
  - Todo juego imparcial de información perfecta bajo juego normal es equivalente a un montón de Nim con tamaño igual a su valor de Grundy $G(u) = \text{mex}\{G(v) : u \to v\}$.
  - Para la suma disjunta de juegos independientes: $G(S_1 + S_2 + ...) = G(S_1) \oplus G(S_2) \oplus ...$
  - El estado es **ganador** para el primer jugador si y solo si la suma XOR es $\neq 0$.
- **Juego de Nim**:
  - Posición ganadora $\Leftrightarrow x_1 \oplus x_2 \oplus ... \oplus x_n \neq 0$.
  - **Regla de Misère Nim** (gana quien toma la última ficha): Se juega idéntico al Nim normal con suma XOR $\neq 0$, EXCEPTO cuando todos los montones tienen tamaño ≤ 1: en ese caso, la posición es ganadora si y solo si el número de montones de tamaño 1 es **par**.
- **Juego de Moore ($ ext{Nim}_k$)**:
  - En cada turno, el jugador puede retirar fichas de entre 1 y k montones distintos.
  - Condición de victoria: Sumar cada columna binaria de los tamaños de los montones en base 10; la posición es perdedora si y solo si la suma de cada columna binaria es un múltiplo de $k+1$.

10.7 Reglas Lógicas de 2-SAT

Cláusula Deseada	Implicaciones en el Grafo de 2-SAT
p debe ser Verdadero	$\neg p \implies p$
p debe ser Falso	$p \implies \neg p$
Al menos uno ($p \vee q$)	$\neg p \implies q$ y $\neg q \implies p$
No ambos $\neg(p \wedge q) \equiv (\neg p \vee \neg q)$	$p \implies \neg q$ y $q \implies \neg p$
Equivalentes ($p = q$)	$p \implies q$, $q \implies p$, $\neg p \implies \neg q$, $\neg q \implies \neg p$
Distintos / XOR ($p \neq q$)	$p \implies \neg q$, $\neg p \implies q$, $q \implies \neg p$, $\neg q \implies p$
A lo sumo uno entre $x_1, ..., x_n$	Crear prefijos $p_i = p_{i-1} \vee x_i$: $x_i \implies p_i$, $p_{i-1} \implies p_i$, $p_{i-1} \implies \neg x_i$.

- **Solubilidad**: El sistema 2-SAT es satisfacible si y solo si para ninguna variable x, x y $\neg x$ caen en la misma Componente Fuertemente Conexa (SCC). La asignación se obtiene eligiendo el valor cuya SCC aparece después en el orden topológico.

10.8 Tabla de Constantes, Primos y Módulos Frecuentes

Tipo	Constante / Primo	Propiedades Clave
Estándar	$10^9 + 7 = 1000000007$	Primo común en Codeforces/ICPC ($2 \times 10^9 + 14$ entra en signed 32-bit).
Estándar 2	$10^9 + 9 = 1000000009$	Ideal para segundo módulo en doble rolling hash con $10^9 + 7$.
Primos Hash	$10^9 + 21, 10^9 + 33, 10^9 + 87$	Primos grandes para evitar hacks en tablas hash.
NTT Primo 1	$998244353 = 119 \cdot 2^{23} + 1$	Raíz primitiva $g = 3$. Soporta transformadas de tamaño hasta $2^{23} \approx 8.38 \times 10^6$.
NTT Primo 2	$1004535809 = 479 \cdot 2^{21} + 1$	Raíz primitiva $g = 3$. Soporta tamaños hasta $2^{21} \approx 2.09 \times 10^6$.
NTT Primo 3	$469762049 = 7 \cdot 2^{26} + 1$	Raíz primitiva $g = 3$. Soporta tamaños hasta $2^{26} \approx 6.71 \times 10^7$.
64-bit Prime	$10^{18} + 9 = 1000000000000000009$	Primo de 64 bits para pruebas con Miller-Rabin.
64-bit Prime	$10^{18} + 31 = 1000000000000000031$	Segundo primo de 64 bits.
Bases Miller-Rabin	$\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$	Conjunto determinista para probar primalidad de cualquier $N < 2^{64}$.
Primos Gemelos	$(10^9 + 7, 10^9 + 9)$	Pareja de primos gemelos útiles para hashes combinados.

Bloque 11: Python 3 para ICPC — Trucos, BigInt, Bases y Librería Estándar

¿Cuándo usar Python en competencia?: - Cuando el problema requiere **BigInt** (miles de dígitos sin límite de 64 o 128 bits). - Cuando toque convertir o parsear **bases arbitrarias** o formatear binario/hexadecimal. - Cuando requieras **fechas, días transcurridos y calendarios** exactos (`datetime`). - Cuando se necesiten **fracciones exactas** (`Fraction`) o **100+ decimales** (`Decimal`). - Cuando parsear strings en C++ tome decenas de líneas y en Python salga con `re` o `eval`. - Cuando una DP con estados complejos (strings o tuplas) se resuelva en 5 líneas con `@cache`.

11.1 Template Base, Fast I/O, Límites de Recursión y Digits en BigInt

```
import sys
from math import gcd, isqrt, comb, perm, factorial
from collections import defaultdict, Counter, deque
from fractions import Fraction
from decimal import Decimal, getcontext
from functools import cache
import bisect
import itertools
import re

# Aumentar limite de recursion para DFS profunda
sys.setrecursionlimit(10**6)

# OBLIGATORIO en Python 3.11+ para BigInt > 4300 digitos a string
sys.set_int_max_str_digits(10**6)

def solve():
    # Lectura ultra rapida de todos los tokens
```

```
    input_data = sys.stdin.read().split()
    if not input_data:
        return
    it = iter(input_data)
    # n = int(next(it))

if __name__ == '__main__':
    solve()
```

11.2 Conversión de Bases Arbitrarias (Base $A \leftrightarrow B$) y Formateo

- **¿Cuándo usar?**: Convertir números entre cualquier base del rango $[2, 36]$ (o hasta 62) sin implementar división manual.

```
# 1. De cualquier Base B a Entero Decimal: int(string, base)
num = int("1A3F", 16)       # Hexadecimal a int -> 6719
num = int("101101", 2)      # Binario a int -> 45
num = int("Z9", 36)         # Base 36 a int -> 1269

# 2. De Decimal a Binario, Octal, Hexadecimal nativo:
s_bin = bin(45)[2:]         # "101101"
s_oct = oct(45)[2:]         # "55"
s_hex = hex(45)[2:].upper() # "2D"

# Con f-strings y padding de ceros (ej. 8 bits):
f_bin = f"{45:08b}"         # "00101101"
f_hex = f"{45:04X}"         # "002D"

# 3. De Decimal a cualquier Base B (2 <= B <= 62):
CHARS = ("0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
         "abcdefghijklmnopqrstuvwxyz")

def to_base(n: int, b: int) -> str:
    if n == 0:
        return "0"
    res = []
    neg = n < 0
    n = abs(n)
    while n > 0:
        res.append(CHARS[n % b])
        n //= b
    if neg:
        res.append('-')
    return "".join(reversed(res))

# Conversion directa Base A -> Base B:
def base_a_to_b(s: str, base_a: int, base_b: int) -> str:
    return to_base(int(s, base_a), base_b)
```

11.3 Aritmética Gigante (BigInt), Potencia Modular y Raíz Entera

- **¿Cuándo usar?**: Multiplicación, suma y potencias de números de cientos de dígitos. `pow(b, e, mod)` se ejecuta en C a nivel interno en $\mathcal{O}(\log e)$.

```
# Potencia Modular BigInt en O(log e): pow(b, e, mod)
# Mucho mas veloz que (b ** e) % mod
ans = pow(3, 10**18, 10**9 + 7)

# Division entera con negativos:
# En Python -7 // 3 da -3 (hacia -infinito).
# Para truncar hacia cero como en C++: int(-7 / 3) -> -2

# Raiz Cuadrada Entera Exacta: isqrt(N)
# Retorna floor(sqrt(N)) sin perder precision de punto flotante
r = isqrt(10**50)  # 10^25 exacto (double fallaria aqui)
```

11.4 Fracciones Exactas (`Fraction`) y Decimales de Alta Precisión (`Decimal`)

- **¿Cuándo usar?**:
  - `Fraction`: Cálculos de probabilidad o geometría donde se exige respuesta irreducible P/Q.
  - `Decimal`: Problemas con 100 o más dígitos de precisión donde `long double` sufre redondeos.

```
# 1. Fracciones Exactas (simplifica automaticamente)
from fractions import Fraction

f1 = Fraction(3, 6)         # 1/2
f2 = Fraction("0.25")       # 1/4
res = f1 + f2               # 3/4
print(f"{res.numerator}/{res.denominator}")

# 2. Decimales con precision configurable (ej. 100 digitos)
from decimal import Decimal, getcontext
getcontext().prec = 100     # 100 digitos significativos

pi_approx = Decimal(22) / Decimal(7)
raiz_2 = Decimal(2).sqrt()
```

11.5 Manejo de Fechas y Calendarios (`datetime`)

- **¿Cuándo usar?**: Contar días entre dos fechas, saber qué día de la semana cayó una fecha o sumar días considerando años bisiestos.

```
import datetime

# 1. Crear fecha (Ano, Mes, Dia)
d1 = datetime.date(2024, 2, 28)
d2 = datetime.date(2026, 10, 25)

# 2. Diferencia en dias:
dias = (d2 - d1).days       # 970 dias exactos

# 3. Sumar dias a una fecha:
futuro = d1 + datetime.timedelta(days=100)

# 4. Dia de la semana: 0 = Lunes, 6 = Domingo
dia_sem = d1.weekday()

# 5. Formatear fecha:
texto = d1.strftime("%Y-%m-%d, %A")
```

11.6 DP con Memoización Automática (`@cache` / `cache_clear`)

- **¿Cuándo usar?**: Programación dinámica top-down con estados compuestos (tuplas, strings o enteros con offsets negativos). Evita matrices manuales y llamadas a `memset`.

```
from functools import cache

# DP de Mochila o Digit DP con memoizacion automatica:
@cache
def dp(i: int, rem_weight: int) -> int:
    if i == n or rem_weight == 0:
        return 0
    # Opcion 1: No tomar
    res = dp(i + 1, rem_weight)
    # Opcion 2: Tomar
    if weight[i] <= rem_weight:
        res = max(res, val[i] + dp(i + 1, rem_weight - weight[i]))
    return res

# Limpiar memoizacion entre casos de prueba:
dp.cache_clear()
```

11.7 Extracción de Números en Texto Sucio con Expresiones Regulares (`re`)

- **¿Cuándo usar?**: Cuando la entrada tiene formatos irregulares o molestos (ej. `Point(x=12, y=-45) -> [100, -5]`).

```
import re

s = "pos: (x=15, y=-42), vel=[+100, -5], count=8"
# Extrae TODOS los enteros con o sin signo en 1 linea:
nums = [int(x) for x in re.findall(r'-?\d+', s)]
# nums = [15, -42, 100, -5, 8]
```

11.8 Frecuencias y Álgebra de Multiconjuntos (`Counter`)

- **¿Cuándo usar?**: Conteo de frecuencias, multiconjuntos y hallar los K elementos más frecuentes.

```
from collections import Counter

c1 = Counter([1, 2, 2, 3])
c2 = Counter([2, 3, 3, 4])

# Operaciones algebraicas de multiconjuntos:
suma  = c1 + c2     # Suma conteos: {2: 3, 3: 3, 1: 1, 4: 1}
resta = c1 - c2     # Resta (mantiene solo positivos): {1: 1, 2: 1}
inter = c1 & c2     # Minimo conteo (interseccion): {2: 1, 3: 1}
union = c1 | c2     # Maximo conteo (union): {1: 1, 2: 2, 3: 2, 4: 1}

# Top K elementos mas frecuentes en O(N log K):
top2 = c1.most_common(2) # [(2, 2), (1, 1)]
```

11.9 Agrupación y Grafos Instantáneos (`defaultdict`)

- **¿Cuándo usar?**: Listas de adyacencia de grafos o agrupamientos sin necesidad de inicializar tamaños N.

```
from collections import defaultdict

# Grafo sin tamano previo:
adj = defaultdict(list)
adj[u].append(v)

# Agrupar anagramas:
grupos = defaultdict(list)
for w in words:
    grupos["".join(sorted(w))].append(w)
```

11.10 Rotación y Transposición de Matrices en 1 Línea (`zip(*mat)`)

- **¿Cuándo usar?**: Rotar grillas 2D en simulaciones sin bucles anidados manuales.

```
mat = [
    [1, 2, 3],
    [4, 5, 6]
]

# 1. Transponer matriz (columnas a filas):
trans = [list(col) for col in zip(*mat)]

# 2. Rotar 90 grados sentido horario:
rot90 = [list(col[::-1]) for col in zip(*mat)]

# 3. Rotar 90 grados sentido antihorario:
rot90_ccw = [list(col) for col in zip(*mat)][::-1]
```

11.11 Álgebra de Conjuntos Nativos (`set`)

- **¿Cuándo usar?**: Unión, intersección o diferencia de conjuntos en $\mathcal{O}(\min(|A|, |B|))$ sin ordenar previamente como en `std::set_intersection`.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

inter = A & B       # Interseccion: {3, 4}
union = A | B       # Union: {1, 2, 3, 4, 5, 6}
dif   = A - B       # En A pero no en B: {1, 2}
simet = A ^ B       # Diferencia simetrica: {1, 2, 5, 6}
sub   = A <= B      # True si A es subconjunto de B
```

11.12 Bits en Números Gigantes (`bit_count`, `bit_length`)

- **¿Cuándo usar?**: Operaciones de bits sobre números que exceden 64 bits (donde `__builtin_popcountll` en C++ no funciona).

```
n = 10**60

# Cantidad de bits encendidos (popcount) de un BigInt:
unos = n.bit_count()

# Cuantos bits ocupa el numero:
bits_necesarios = n.bit_length()
```

11.13 Búsqueda Binaria con Proyección (`bisect` con `key`) y Ordenamiento Multi-Criterio

- **¿Cuándo usar?**: Búsqueda binaria sobre arreglos de tuplas sin struct ficticio, y ordenamiento con múltiples criterios de desempate.

```
import bisect

# 1. Búsqueda binaria con funcion key (Python 3.10+):
eventos = [(10, "A"), (25, "B"), (40, "C"), (90, "D")]
# lower_bound por el primer elemento (tiempo >= 30):
idx = bisect.bisect_left(eventos, 30, key=lambda x: x[0]) # idx = 2

# 2. Ordenamiento multi-criterio:
# 1) Puntos descendente (-x['pts'])
# 2) Penalidad ascendente (+x['pen'])
# 3) Nombre alfabetico (+x['nom'])
jugadores.sort(key=lambda x: (-x['pts'], x['pen'], x['nom']))
```

11.14 Combinatoria y Generadores Nativos (`comb`, `perm`, `itertools`)

- **¿Cuándo usar?**: Calcular combinaciones gigantes nCr en $\mathcal{O}(\min(k, n-k))$ o generar permutaciones sin `next_permutation`.

```
from math import comb, perm, factorial
from itertools import permutations, combinations, product

# nCr y nPr gigantes sin overflow:
formas = comb(1000, 50)
p = perm(100, 3)
```

```
fact_100 = factorial(100)

# Generacion exhaustiva:
perms = list(permutations([1, 2, 3]))        # 3! formas
combs = list(combinations([1, 2, 3, 4], 2))  # 4C2 formas
cart  = list(product([0, 1], repeat=3))      # 2^3 producto cartesiano
```

11.15 Evaluación de Expresiones Aritméticas (`eval`)

- **¿Cuándo usar?**: Evaluar expresiones aritméticas en strings con paréntesis y precedencia en 1 sola línea.

```
# eval() procesa operadores (+, -, *, //, %, **)
expr = "((15 + 25) * 3) // 4"
resultado = eval(expr)      # 30

# Reemplazar division para forzar division entera:
expr_safe = expr.replace("/", "//")
```